Termpaper

ELL305

ISHANVI PRASAD (2024EE10406)

Sweety (2024MT10535)

Unified Adaptive Memory Manager

for Key–Value Cache Management in Large Language Model Inference on Edge Processors

Contents

1. The Problem: Running Large Language Models on Edge Processors.....6

1.1 Why LLMs on edge devices at all.....6
1.2 How an LLM generates text: prefill and decode.....7
1.3 What the KV cache is.....7
1.4 Why the KV cache is the bottleneck on the edge.....9
1.5 Why the obvious fixes fall short.....10
1.6 Problem statement and design requirements.....11
1.7 Overview of the Work.....11

2. What Already Exists: Research Before Our Solution.....14

2.1 A taxonomy of approaches.....14
2.2 Changing the attention architecture.....15
2.3 Memory management: PagedAttention and vLLM.....15
2.4 Token eviction.....16
2.5 Query-aware selection and retrieval.....19
2.6 Quantisation of the KV cache.....20
2.7 Low-rank compression.....21
2.8 Offloading and flash.....21
2.9 Joint systems.....22
2.10 Hardware: engines and vector processors.....22
2.11 Deployment runtime: compiler-based runtimes.....24
2.12 Coverage matrix.....24
2.13 Summary of the gaps.....24

3. First Approach: UAMM-1.....26

3.1 Components.....26
3.2 What UAMM-1 achieved.....27

4. Gaps in UAMM-1.....28

4.1 Gaps at the level of ideas.....28
4.2 Defects in the implementation.....28
4.3 Evidence: what the gaps cost.....29

5. The Combined System: UAMM-2.....30

5.1 The core idea: a placement is a (tier, codec) pair.....30
5.2 Memory hierarchy and tiers.....31
5.3 The block and its lifecycle.....32
5.4 Codecs: KIVI-style asymmetric quantisation.....33
5.5 The importance signal: H2O reinterpreted.....35
5.6 The utility planner.....36
5.7 Selective attention: reading less.....37
5.8 Hardware: three verified RTL blocks.....38
5.9 The vector kernel for the Ara vector processor.....40
5.10 The runtime seam and the codec slot.....41

6. Project Structure and Code Walkthrough....42

6.1 Directory layout....42
6.2 Python engine, module by module....42
6.3 Worked examples of the main routines....43
6.4 Hardware....45
6.5 Vector kernel....45
6.6 Tests....45
6.7 How to run everything....46

7. Experimental Methodology....47

7.1 What is real and what is modelled....47
7.2 The synthetic trace generator....47
7.3 Metrics....48
7.4 Assumed cost constants....49
7.5 Experimental design....49

8. Simulation Results....51

8.1 Ablation: what each ingredient contributes (E1)....51
8.2 Scaling with context length (E2)....52
8.3 How many blocks must be read? (E3)....53
8.4 Sensitivity of the planner and the importance signal (E4)....54
8.5 Memory budget (E5)....57
8.6 Behaviour over time (E6)....59
8.7 Codec micro-benchmarks (E7)....60
8.8 Attention structure of the workload (E8)....61
8.9 Final placement map....62
8.10 Summary of the simulation findings....63

9. Hardware Results: RTL Simulation, Waveforms and Verification....64

9.1 Verification method....64
9.2 Sweep scorer....64
9.3 Codec pipeline....67
9.4 Block table....68
9.5 Regression, lint and mutation results....68
9.6 Synthesis statistics....69

10. The Ara Vector Kernel: Results....71

10.1 Algorithm in detail....71
10.2 Numerical agreement (host)....72
10.3 RISC-V Vector compile check....72
10.4 Why a fused kernel: bandwidth arithmetic....73

11. Research Papers, Inferences and How We Used Them....75

11.1 Cross-cutting lessons....76

12. Problems UAMM-2 Does Not Yet Address, and Our Solutions....77

12.1 Problems of quantisation and required precision....77
12.2 Problems of selection, placement and planning....80

12.3 Problems of systems and hardware....83
12.4 Order of work....84

13. Putting the Numbers in a Realistic Frame....86

13.1 Why the compression ratios transfer to other geometries....86
13.2 What fits on a phone-class device....86
13.3 Reads per token....86
13.4 What would convince a sceptic....87
13.5 Deployment sketch....87

14. Conclusion....88

Code availability....88

References....14

Appendix A. Glossary....91

Appendix B. Raw Experiment Tables....92

Appendix C. Core Source Listings....95

Appendix D. UAMM-1 Source Code (Original Prototype)....121

1. The Problem: Running Large Language Models on Edge Processors

Large language models (LLMs) have moved from data-centre curiosities to components that product teams now want inside phones, laptops, vehicles, wearables, industrial controllers and embedded boards. The motivations are practical and well understood: **latency** (no network round trip), **privacy** (prompts and documents never leave the device), **availability** (the feature works offline) and **cost** (no per-token cloud bill). This report is about one specific obstacle that stands between that aspiration and a working product: the memory that a running LLM needs for its *attention history*, the **key–value (KV) cache**, and how that memory can be managed on the small, heterogeneous, power-constrained processors that live at the edge.

The report is organised as follows. Chapter 1 states the problem. Chapter 2 describes the algorithms that already exist for it, each on its own. Chapter 3 describes our first approach, UAMM-1, which combined two of them, and Chapter 4 sets out the gaps it left. Chapter 5 presents UAMM-2, the system built to close those gaps, and Chapter 6 explains its directory and every part of its code. Chapters 7 to 10 give the experimental setup, the simulation results, the hardware waveforms and the vector-kernel results. Chapter 11 lists the research papers we drew on, the inference taken from each and how it was used. Chapter 12 describes the problems UAMM-2 still does not address, with the solutions and implementation ideas we worked out for them. Chapter 13 places the results in a realistic frame and Chapter 14 concludes.

1.1 Why LLMs on edge devices at all

An *edge processor* in this report means a processor that runs the model locally rather than in a data centre: a mobile application processor, an embedded RISC-V or Arm SoC, a single-board computer, a small NPU-equipped module. Typical constraints, compared with a data-centre accelerator, are shown in the table below.

Table 1. *Order-of-magnitude comparison between a data-centre accelerator and an edge SoC. Ranges are representative, not measurements of one device.*

Resource	Data-centre accelerator	Edge SoC (typical range)	Consequence
Memory capacity	80–192 GB HBM per accelerator	2–16 GB shared LPDDR, often shared with the OS and other apps	Weights and KV cache compete for the same few gigabytes
Memory bandwidth	2–5 TB/s (HBM)	10–60 GB/s (LPDDR4/5)	Two orders of magnitude less; decode is bandwidth-bound
Compute	Hundreds of TFLOP/s	0.1–40 TOPS (NPU/GPU/vector unit)	Compute is rarely the first limit during decoding
Power / thermal	300–700 W	2–15 W sustained	Every DRAM access costs energy that the battery must

Resource	Data-centre accelerator	Edge SoC (typical range)	Consequence
			pay
Storage tier	NVMe, network storage	eMMC / UFS / SD flash	Flash exists but is 10–100× slower than DRAM
Batching	Dozens of concurrent requests	Usually one user, one sequence	Batching cannot amortise weight reads

The right-hand column is the crux. Almost every technique that makes LLM serving efficient in the cloud relies on batching many requests together so that each read of the weights is shared by many sequences. An edge device typically serves one sequence at a time, so the cost of reading the model is paid by a single user. That makes the work of decoding a single token almost entirely a question of how many bytes must be moved from memory, which is why memory, not arithmetic, is the central resource on the edge.

1.2 How an LLM generates text: prefill and decode

A decoder-only transformer produces text one token at a time. Generation has two phases with very different hardware behaviour.

- **Prefill.** The prompt (say 1,000 tokens) is processed in parallel. All prompt tokens are pushed through every layer at once. The work is dominated by matrix–matrix multiplications and is compute-bound; hardware with strong vector or matrix units handles it well.
- **Decode.** Each new token is produced by one pass through the network. For that single token the processor must read essentially every weight of the model, and in every attention layer it must also read the stored keys and values of every previous token. The work is matrix–vector multiplication, which performs very little arithmetic per byte moved, and is therefore memory-bandwidth-bound.

Because users experience decode as the streaming speed of the answer (tokens per second), and because the decode loop repeats for every token, the number of bytes read per token is the primary determinant of perceived speed on an edge device.

1.3 What the KV cache is

Inside every attention layer, each token produces three vectors from its hidden state: a **query** (q), a **key** (k) and a **value** (v). Attention for the newest token computes a similarity score between its query and the key of every earlier token, converts the scores into weights with a softmax, and returns the weighted sum of the earlier tokens' values:

$$o_t = \Sigma_{i \le t} \, softmax_i(q_t \cdot k_i / \sqrt{d}) \cdot v_i$$

The keys and values of earlier tokens never change once computed (the model is causal). Recomputing them for every new token would multiply the work enormously, so inference engines store them. This store is the **KV cache**. It is the model's working memory of the conversation: every layer, every KV head, every token contributes one key vector and one value vector that must be kept for as long as the token can still be attended to.

The size of that memory follows a simple formula:

$$\textit{bytes per token} = 2\ (K \text{ and } V) \times \textit{layers} \times \textit{KV-heads} \times \textit{head-dim} \times \textit{bytes per element}$$

and the cache for one sequence is that number multiplied by the current context length. Unlike the weights, which are a fixed size, the KV cache **grows with every generated token** and, in a multi-turn session or a document-question task, may become the largest single object in memory.

Table 2. *KV-cache size for four illustrative model geometries (FP16, one sequence). Geometries are representative of model families, not of any specific checkpoint.*

Illustrative geometry	**layers / KV heads / head-dim**	**bytes per token (FP16)**	**4K context**	**32K context**	**128K context**
TinyLlama-like	22 / 4 / 64	22,528	88 MiB	0.69 GiB	2.8 GiB
Llama-3-8B-like	32 / 8 / 128	131,072	512 MiB	4.00 GiB	16.0 GiB
7B MHA-style	32 / 32 / 128	524,288	2,048 MiB	16.00 GiB	64.0 GiB
Llama-3-70B-like	80 / 8 / 128	327,680	1,280 MiB	10.00 GiB	40.0 GiB

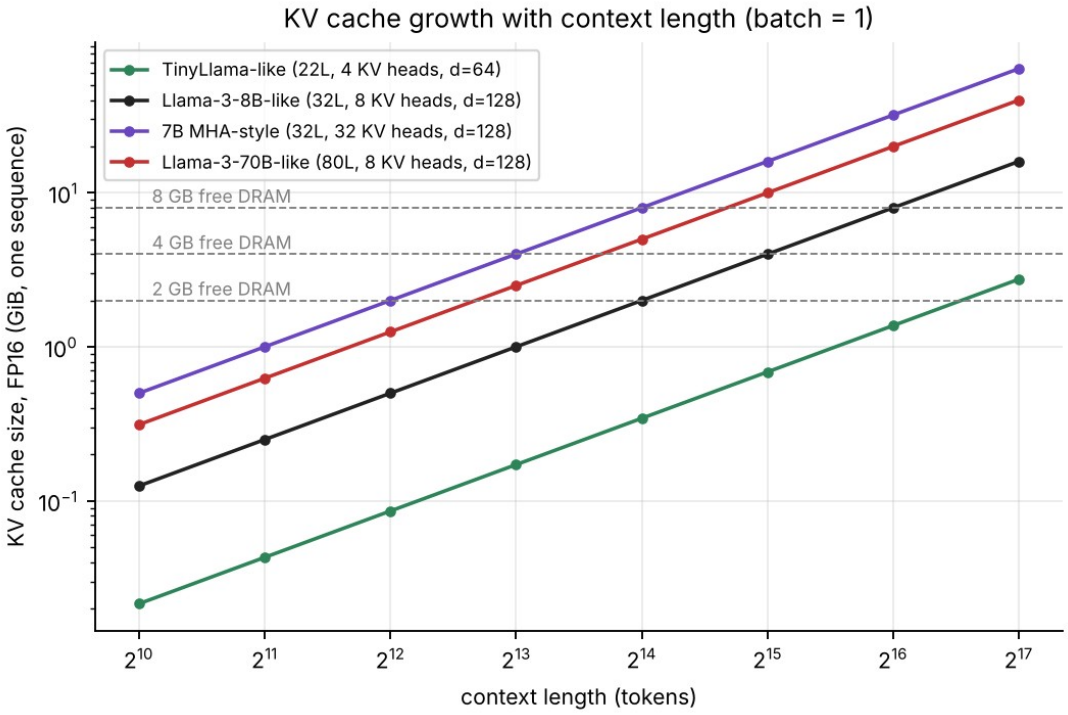


Figure 1. *KV-cache size grows linearly with context. Dashed lines show plausible free-DRAM budgets on edge devices; every geometry except the smallest crosses one of them within the context lengths that applications now request.*

1.4 Why the KV cache is the bottleneck on the edge

1.4.1 Capacity

On a phone with 8 GB of RAM, the operating system and foreground applications may leave 3–5 GB available to an on-device model. A 7–8-billion-parameter model quantised to 4 bits needs roughly 4–5 GB for its weights alone. The weights are fixed, so what remains for the KV cache shrinks to a few hundred megabytes. In the Llama-3-8B-like geometry of Table 2 that budget is exhausted at about 2–3 thousand tokens of FP16 cache, far short of the 32K–128K contexts that long-document and agentic applications need. When memory is exhausted the platform kills the process, or the engine must truncate the context, or the application must refuse the request.

1.4.2 Bandwidth

Capacity is only half of the problem. Standard attention reads **the entire KV cache for every generated token**. Even if the cache fits, its size determines how many bytes are moved per token, and therefore the maximum speed. The next figure shows the upper bound on decode speed imposed by KV reads alone for a 12.8 GB/s memory system, ignoring weight reads and compute. For a 7B-class model with full multi-head attention the bound drops below ten tokens per second at roughly 3K tokens of context, before a single weight has been read.

1.4.3 Energy

Each byte read from DRAM costs on the order of tens to a hundred times more energy than an arithmetic operation on that byte, and flash costs more again. A design that reads the whole cache for every token spends the battery on data movement. Reducing the bytes touched per token is therefore both a speed and a battery-life objective.

1.4.4 Heterogeneous memory

Edge SoCs expose a *hierarchy*: a few tens of kilobytes of tightly coupled SRAM or vector register file next to the compute unit, gigabytes of DRAM behind a shared bus, and gigabytes of flash behind a controller. The hierarchy is an opportunity (cold data can be parked cheaply) and a hazard (a wrong placement decision puts hot data behind a slow link). Data-centre inference engines were not designed around this hierarchy, and most KV-cache research assumes a GPU with high-bandwidth memory and a host CPU as the only second tier.

1.4.5 Why "just buy more memory" is not an answer

For consumer and embedded products the memory is part of the bill of materials, shares a thermal envelope with the processor and is soldered. The memory in a shipped device cannot be upgraded after the fact, whereas the context lengths that applications demand grow every year. Any solution must therefore make the same hardware serve longer contexts and do so within a fixed energy budget.

1.5 Why the obvious fixes fall short

Table 3. *Single-technique fixes and why each leaves part of the problem open.*

Obvious fix	What it gets you	Why it is not sufficient alone
Truncate the context to a fixed window	Bounded memory and bandwidth	Discards information the model may need; quality collapses on tasks that refer back to early content
Store the cache in FP16 but page it to flash	Capacity	A flash read per token is far too slow; without knowing which blocks matter the system stalls
Quantise the entire cache uniformly to 4 bits	≈3× capacity and bandwidth	Applies the same precision to important and unimportant data; still reads everything each token; 2-bit quantisation hurts quality
Evict low-importance tokens permanently	Capacity and bandwidth	Irreversible: a token that becomes relevant later is gone; no precision control for what remains
Use a bigger or smaller model	Moves the problem	Smaller models lose quality; the KV cache of any model still grows with context

Each fix addresses one dimension (capacity, bandwidth, precision, placement) and ignores the others. The central observation behind this work is that these dimensions interact: the best precision for a block depends on where it is stored, how likely it is to be read, and how much room is left. A system that decides them separately will make poor joint decisions.

1.6 Problem statement and design requirements

We therefore pose the problem as follows.

Problem statement

Given an autoregressive LLM decoding one sequence on an edge processor with a small fast memory, a larger DRAM and a slow flash tier, keep the KV cache within a fixed byte budget and reduce the bytes read per token, while keeping the attention output as close as possible to what an unconstrained FP16 dense implementation would produce, without irreversibly discarding information unless explicitly allowed.

From that statement we derived the requirements that shaped the design:

1. **Unit of management is a block, not a token.** Per-token bookkeeping is too costly and defeats memory locality; paging into fixed-size blocks makes allocation, migration and hardware tables simple.
2. **Placement and precision are decided together.** A block is placed in a (tier, codec) pair; the two choices are never made independently.
3. **Importance must be measured, not assumed.** Block priority must come from what attention actually does, not only from age.
4. **Reads must be selective.** The system should be able to skip fetching and decoding blocks that provably cannot matter for the current query.
5. **Loss must be controlled and visible.** Precision may only decrease along an explicit ratchet; permanent drop is opt-in; capacity overflow is reported rather than hidden.
6. **Decisions must be cheap enough for hardware.** The inner scoring loop must be implementable without a divider and verified bit-exactly against a software model.
7. **The design must connect to real runtimes and real vector hardware.** The codecs must be swappable, the compute kernel must target a RISC-V vector unit, and a runtime adapter must isolate the engine from a specific inference framework.

The rest of the report shows how each requirement was met, how it was tested, and where it is still unmet.

1.7 Overview of the Work

We first built **UAMM-1**, our own prototype of a unified adaptive memory manager, which combined PagedAttention-style block allocation, H2O-style importance scoring and a three-tier placement policy, with a small Verilog decision engine. We reviewed the literature on KV-cache management, identified the gaps that remained, and then

redesigned the system as **UAMM-2**, drawing on the most useful ideas from published KV-cache work and from vector-hardware and compiler-runtime tooling.

Table 4. *The combined system at a glance.*

Ingredient	Source	Role in UAMM-2
Paged blocks	PagedAttention / vLLM	Unit of allocation, migration and tracking
Importance from attention	H2O (heavy hitters)	Decayed attention scores aggregated to block importance
Asymmetric KV quantisation	KIVI (and KVQuant ideas)	Per-channel keys and per-token values at 8, 4 and 2 bits
Query-aware block selection	Quest	Min/max key bounds to skip blocks without fetching them
Multi-tier placement, joint codec decisions	InfiniGen, EvicPress, TensorRT-LLM KV compression	(tier, codec) planner with a utility cost and hysteresis
Hardware codec engine	Open hardware KV-compression engines	Reference for group quantisation and outlier handling; kept as a swappable codec slot
Vector processor	Ara RISC-V vector processor	Fused attention-score kernel that reads packed 4-bit keys directly
Portable runtime	TVM-based portable runtimes	Adapter seam through which the engine would be attached to a deployed model
Original UAMM-1	Our prior work	Block table, tiering concept, scorer, kept as an ablation baseline

The combined system adds four things that none of the individual sources provide together:

1. A **(tier, codec) placement** for every block, so that capacity, bandwidth and precision are traded off in one decision.
2. A **utility planner** with hysteresis that prices latency, quality risk and capacity pressure, and a **one-way precision ratchet** that prevents silent quality surprises.
3. **Hardware**: a sweep scorer, a 4-bit codec pipeline and a block table in Verilog, each verified bit-exactly against a Python golden model and mutation-tested.
4. **A vector kernel** for the Ara vector processor that computes attention scores directly on packed 4-bit keys without first expanding them to floating point.

The system was implemented as a Python reference engine (about 1,400 lines), three Verilog modules with self-checking testbenches, a C kernel with a RISC-V Vector path, and a 56-test suite. A set of eight controlled experiments produced the results in Chapters 8 to 10, including waveforms rendered from real simulation dumps of the RTL.

Headline results from the experiments in this report

At 1,536 tokens with a 168 KB total cache budget (FP16 would need 393 KB), the UAMM-1 threshold policy kept 0.367 of the FP16 footprint but permanently dropped 973

of 1,536 tokens on average and retained 0.758 of the attention mass. UAMM-2 with INT8/KIVI-4/KIVI-2 codecs kept all 1,536 tokens in 0.310 of the FP16 footprint and retained 0.939 of the attention mass.

Modelled fetch time per token fell from 0.923 ms (UAMM-1) to 0.112 ms (UAMM-2, all codecs) under the cost constants stated in Chapter 7.

The price is quantisation error: mean relative L2 error of the attention output was 0.166 for UAMM-2 (all codecs) against 0.479 for UAMM-1, whose error is dominated by lost tokens.

All RTL checks (1,088 scorer checks, 400 codec vectors, 6,000 random block-table operations) pass bit-exactly and 12 of 12 injected RTL bugs are caught.

2. What Already Exists: Research Before Our Solution

Before designing anything we surveyed how the community has attacked the KV-cache problem. This chapter describes the **existing algorithms individually**, as their authors presented them, not in any combination. For each one we record the mechanism, what it solves, what it leaves open for an edge device, and the inference we drew that influenced the design. Bibliographic details are in the References; recent preprints should be re-checked against their latest versions before formal submission.

2.1 A taxonomy of approaches

KV-cache research can be organised by *which lever it pulls*. The table below lists the levers; the rest of the chapter takes them in turn.

Table 5. *Levers used by the existing work.*

Lever	Idea	Representative work	Primary saving
Change the model	Fewer KV heads, or a compressed latent KV	MQA, GQA, multi-head latent attention (DeepSeek-V2)	Bytes per token (requires training)
Manage memory better	Page the cache into blocks to remove fragmentation	PagedAttention / vLLM	Wasted capacity
Evict tokens	Keep only tokens judged important	H2O, Scissorhands, StreamingLLM, SnapKV, FastGen, PyramidKV	Capacity and bandwidth (lossy, irreversible)
Select at read time	Keep everything but fetch only relevant blocks	Quest, InfiniGen	Bandwidth
Quantise	Store K and V in fewer bits	KIVI, KVQuant	Capacity and bandwidth
Low-rank / projection	Store low-rank projections of K/V	Palu	Capacity and bandwidth
Offload	Spill to CPU memory or flash and prefetch	FlexGen, InfiniGen, LLM in a flash	Capacity
Joint systems	Combine compression, eviction and tiering	EvicPress, KV-Compress, TensorRT-LLM KV-compression framework	Several at once
Hardware engines	Dedicated logic for compress/decompress or cache control	hardware KV-compression engines, CXL-SpecKV, Ara vector unit	Energy and throughput
Deployment runtimes	Compile and run models portably on many devices	TVM-based compiler runtimes	Portability

2.2 Changing the attention architecture

Multi-query attention (MQA) and grouped-query attention (GQA)

Shazeer's multi-query attention (2019) shares one key head and one value head across all query heads; Ainslie et al.'s grouped-query attention (2023) interpolates, sharing each KV head across a group of query heads. Both reduce the KV cache by the ratio of query heads to KV heads without changing the sequence length dimension. They are now standard in many open models (the 8-KV-head geometry used in Chapter 1 is an example).

Limit. They must be built into the model during training or fine-tuning. For a model already on the device they offer nothing, and the cache still grows linearly with context.

Inference we drew

Architecture-level savings are complementary to ours and multiply with them. Our engine treats the per-token K and V as vectors of whatever size the model produces, so a GQA or latent-attention model simply gives it smaller vectors. We did not attempt to change the model.

Multi-head latent attention

DeepSeek-V2 compresses keys and values into a shared low-dimensional latent vector per token and re-expands them inside the attention computation, cutting the cache by an order of magnitude for that model family. As with GQA, the saving requires the model to be trained with this structure.

2.3 Memory management: PagedAttention and vLLM

PagedAttention / vLLM

Kwon et al., SOSP 2023. Before vLLM, serving systems reserved one contiguous slab of memory per request sized for the *maximum* sequence length, because the final length is unknown. This wasted most of the allocated memory through internal and external fragmentation.

How it works. PagedAttention borrows the idea of virtual-memory paging. The KV cache of each sequence is split into fixed-size **blocks**; a per-sequence block table maps logical blocks to physical blocks scattered anywhere in memory. Blocks are allocated on demand, so waste is limited to the unfilled tail of the last block. The attention kernel gathers keys and values through the block table. Blocks can be shared between sequences (copy-on-write) for common prefixes.

What it solves. Near-zero fragmentation (the authors report under 4 % waste), simple allocation and the ability to share prefixes. It made memory management of the KV cache a solved engineering problem for GPU serving.

What it leaves open for an edge device. PagedAttention solves *how memory is laid out*, not *how much memory is needed* or *which blocks matter*. It has no notion of importance, no compression, no tiers below the GPU/CPU pair, and it assumes every block of a sequence is read for every token.

Inference we drew

Blocks are the right unit. We adopted fixed-size blocks and a block table, and, importantly, made the block the unit that carries metadata (importance, selection frequency, codec, tier, key min/max summary). A block table is also a natural hardware structure; it became one of our three RTL modules.

2.4 Token eviction

A family of methods observes that attention is highly concentrated, so most cached tokens contribute almost nothing to any given step, and drops them. Chapter 8 measures the same concentration on our synthetic workload.

H2O: Heavy-Hitter Oracle

Zhang et al., NeurIPS 2023. H2O notes that attention scores follow a power law: a small set of tokens ("heavy hitters") receives most of the accumulated attention across steps.

How it works. The method keeps a fixed budget of tokens: a window of recent tokens plus the tokens with the highest accumulated attention score. When the budget is full, the lowest-scoring non-recent token is evicted. Scores are the running sum of attention weights that each token has received. Reported experiments retain quality with about 20 % of the cache on the models tested.

What it solves. A principled, cheap importance signal that is computed from attention weights the model already produces, and a simple eviction rule.

What it leaves open for an edge device. H2O decides **only whether a token lives or dies**. There is no quantisation, so what is kept stays at full precision; there is no lower tier, so evicted tokens are gone forever; and a token that is unimportant now but important to a later question cannot be recovered. It also does not reduce bytes per kept token.

Inference we drew

We kept H2O's *signal* (decayed attention mass per token, aggregated to blocks) but changed what it controls. In UAMM-2 the signal drives **placement and precision**, which

are reversible or bounded, while permanent drop is an explicit opt-in. This directly addresses the gap noted above: H2O does eviction but no quantisation.

Scissorhands

Liu et al., NeurIPS 2023. Scissorhands builds on the "persistence of importance" hypothesis: tokens that mattered in the past tend to matter in the future.

How it works. It keeps a budget of tokens, using the history of attention scores to decide which to discard, with compression performed when the budget is exceeded.

What it solves. It reaches large compression with a light-weight rule and no training.

What it leaves open for an edge device. Like H2O it is eviction-only. The persistence assumption is also challenged by workloads whose focus shifts across topics.

Inference we drew

Persistence is a reasonable prior, but it can fail when the query returns to an old topic. Our synthetic workload therefore includes topic phases that revisit older topics, to test whether cold blocks can be promoted back.

StreamingLLM and attention sinks

Xiao et al., ICLR 2024. StreamingLLM observes that the first few tokens of a sequence receive disproportionate attention regardless of content ("attention sinks"), and that removing them breaks the model.

How it works. The method keeps the first few tokens plus a sliding window of recent tokens and evicts the rest, enabling streaming over very long inputs with a bounded cache.

What it solves. A trivial, fast policy that is stable for unbounded streams.

What it leaves open for an edge device. The evicted middle is lost; the method does not improve the model's ability to use information from the evicted span.

Inference we drew

We protect **sink tokens** (the first four) and a **recent window** (32 tokens) unconditionally, hold them in the fastest tier at full precision, and never move or compress them. The protection is also implemented in the RTL scorer (its `protected` input).

SnapKV

Li et al., 2024. SnapKV targets long prompts. It uses an observation window at the end of the prompt to vote on which earlier positions are relevant and compresses the prompt cache accordingly.

How it works. Attention weights from the window to earlier positions are pooled, the highest-voted positions are retained per head, and the rest are dropped during prefill.

What it solves. Substantial prompt-cache reduction with little quality loss on long-context benchmarks.

What it leaves open for an edge device. It is a one-shot decision at the end of prefill and does not adapt as generation proceeds; dropped positions cannot be recovered.

Inference we drew

Importance can be estimated from a small window rather than from all history. We retained the continuously updated H2O-style score (cheap, vectorised) rather than a one-shot vote, but noted that a prefill-time seed for the score is a natural extension (Chapter 12).

FastGen and PyramidKV

Ge et al., 2023; Cai et al., 2024. FastGen profiles each attention head and applies a different eviction policy per head; PyramidKV allocates different budgets to different layers, giving more to lower layers where attention is broad.

How it works. Both use structure in how attention behaves across heads or layers to allocate a fixed budget unevenly.

What it solves. Better quality for the same total budget than uniform schemes.

What it leaves open for an edge device. They again decide only eviction and presume per-layer or per-head access that a flattened single-vector cache does not model.

Inference we drew

Budgets should be non-uniform across layers and heads. Our reference engine flattens K and V per token into a single vector per layer-head group; per-layer and per-head budgets are listed in Chapter 12 as unfinished work, and the block metadata structure was chosen so that a (layer, head) index can be added.

2.5 Query-aware selection and retrieval

Quest

Tang et al., ICML 2024. Quest observes that which tokens matter depends on the **current query**, so a fixed eviction rule is wasteful.

How it works. For each page of the KV cache it stores the element-wise minimum and maximum of the keys. At decode time it computes, from the query and these two vectors, an upper bound on the attention score any key in the page can have, selects the pages with the highest bounds, and runs attention only on them. Nothing is evicted; everything remains stored.

What it solves. Large reductions in memory reads (and therefore decode latency) with little accuracy loss, without losing information.

What it leaves open for an edge device. Quest reduces **bandwidth** but not **capacity**; the whole cache must still be resident, which is exactly what an edge device cannot afford.

Inference we drew

This is the technique that makes tiering practical. If the system can bound which blocks are needed before fetching them, cold blocks can live in slow memory at low precision, because most steps never touch them. We adopted min/max key summaries per block and the bound `Σ_c max(q_c·kmin_c, q_c·kmax_c)`, and our tests verify that it is a true upper bound. Quest's capacity gap is closed by our tiers and codecs.

InfiniGen

Lee et al., OSDI 2024. InfiniGen targets the case where the KV cache does not fit in GPU memory and lives in CPU memory.

How it works. It uses a lightweight speculation, computed from a skewed view of the weights, to predict at layer i which tokens layer $i+1$ will need, and prefetches only those from CPU memory.

What it solves. Large speedups over naive offloading by reading only the needed entries across the slow link.

What it leaves open for an edge device. It handles one slow tier, assumes a GPU/CPU split and does not change precision.

Inference we drew

A slow tier is usable if the fetch set is predicted ahead of time. Our reference engine uses bound-ranking at read time (not prefetch) and lists **prefetching** as a missing piece (Chapter 12). The tier model (SRAM, DRAM hot, DRAM warm, flash) generalises InfiniGen's two-level view.

2.6 Quantisation of the KV cache

Weights are routinely quantised to 4 bits; the KV cache is harder because its statistics differ between keys and values and, in particular, a few key channels carry very large magnitudes.

KIVI

Liu et al., ICML 2024. KIVI studies the element distributions of the KV cache in popular models and reports that keys have a few channels with large magnitudes (outlier channels) while values do not show this pattern.

How it works. It quantises **keys per channel** (one scale and zero-point per channel across a group of tokens) and **values per token** (one scale and zero-point per token across channels), using asymmetric 2-bit integers, and keeps a small recent window in full precision. It needs no fine-tuning.

What it solves. About 2.6× lower peak memory (including weights) at 2 bits with small quality loss on the models tested, enabling larger batches or longer contexts.

What it leaves open for an edge device. KIVI is applied uniformly. Every block gets the same precision whether it is attended a thousand times or never. It does not decide placement, and it still reads the entire quantised cache each step.

Inference we drew

The axis of quantisation matters. We implemented per-channel keys and per-token values and measured the effect directly (Chapter 8): per-channel key quantisation reduced error by 1.9× at 2 and 4 bits on outlier-free data and 5.4× at 2 bits and 5.2× at 4 bits when outlier channels are present, which supports the paper's central design choice. We then used the codec **selectively** per block, which uniform quantisation cannot do.

KVQuant

Hooper et al., NeurIPS 2024. KVQuant pushes KV quantisation toward sub-4-bit precision for very long contexts.

How it works. It combines per-channel key quantisation, quantising keys before the rotary positional embedding, non-uniform datatypes fit per layer, dense-and-sparse handling of outliers, and normalisation of the quantisation centroids.

What it solves. Context lengths of millions of tokens on large models with modest quality loss.

What it leaves open for an edge device. The techniques require calibration data and custom kernels; they are applied uniformly and aimed at GPUs.

Inference we drew

Outliers must be handled explicitly. Our codec supports separate treatment of outlier channels (the key-axis experiment in Chapter 8), and our hardware quantiser follows the simple per-group form because outlier handling is cheaper in the surrounding logic. The more elaborate KVQuant ingredients (pre-RoPE keys, non-uniform datatypes) are listed as future work.

2.7 Low-rank compression

Palu

Chang et al., 2024. Palu observes that the projection matrices that produce K and V can be factorised so that a low-rank latent is cached instead of the full vectors.

How it works. It decomposes the projection weights into low-rank factors, stores the latent, and reconstructs keys and values on the fly with a fused kernel; it adds rank-search and quantisation-friendly transformations.

What it solves. Compression that composes with quantisation, on a pre-trained model, without retraining from scratch.

What it leaves open for an edge device. It adds reconstruction compute and requires modifying how the model is run. The latent still has to be stored for every token.

Inference we drew

Low-rank storage is orthogonal to the codecs and could be added as another codec family. It was not included because it needs access to the model's projection weights, which our reference engine does not have.

2.8 Offloading and flash

FlexGen (Sheng et al., ICML 2023) offloads weights and KV cache across GPU, CPU memory and disk for throughput-oriented batch inference, using a linear-programming search over placement and compression (it also compresses weights and cache to 4 bits). **LLM in a flash** (Alizadeh et al., 2023) shows how to run models larger than DRAM from flash by minimising the number of transferred bytes and the number of reads, with windowing and row–column bundling.

Inference we drew

Flash is usable only if accesses are rare, large and sequential. That is why our flash tier stores only entropy-coded, low-precision blocks and why the planner charges a large fixed cost per flash access. Our experiments (Chapter 8) show that the planner keeps flash reads

low as long as replanning is frequent and the blocks that live in flash are genuinely cold.

2.9 Joint systems

A newer line of work recognises that compression, eviction and tiering interact and tries to decide them together. We reviewed the following; they influenced the design most.

Table 6. *Joint and system-level work reviewed, and how UAMM-2 relates to each.*

Work	What it does	What we took	What we did differently
EvicPress	Jointly chooses compression level and eviction across storage tiers using a utility function	The core idea of a joint utility, with an explicit quality term	Added hysteresis, a precision ratchet, query-aware reads and a verified hardware scorer
KV-Compress / Zipage (paged eviction)	Eviction at block granularity inside a paged cache, with variable budgets per head	Eviction and paging should share the same granularity	Default is lossless tiering; drop is opt-in
TensorRT-LLM KV-cache compression framework	Applies a codec when pages move between tiers (cold-page compression)	Compress at tier transitions, not on the hot path	Added codec ratcheting and per-block measured error tables

None of these is, to our knowledge, a complete edge-oriented stack that includes a verified decision engine in hardware and a vector kernel for a RISC-V vector unit. That gap is where UAMM-2 sits.

2.10 Hardware: engines and vector processors

Hardware KV-compression engines

Open RTL implementations and accelerator papers. Several designs implement the compress/decompress datapath for KV data in hardware.

How it works. A typical engine quantises keys and values in groups (for example 64 values per group) with separate handling of outlier values, using different bit widths for keys (4-bit) and values (2-bit), and may evict a fraction of tokens. Published headline ratios (for example 8×) are often higher than what a benchmark of the same design measures (3–4×), because scales, zero points and outlier lists consume part of the saving.

What it solves. A concrete, synthesisable datapath for compression, useful as an architectural reference.

What it leaves open for an edge device. They contain no policy: nothing decides which blocks to compress, how far, or where to store them. Built-in eviction conflicts with a design that avoids irreversible loss by default.

Inference we drew

Advertised compression ratios must be checked against measured ones including metadata. Our own codec table (Chapter 8) reports measured ratios with scales and zero points counted. The codec interface (`encode`/`decode`/`Payload`) is the slot where such an engine's format can be plugged in.

Ara and Ara2

Cavalcante et al., IEEE TVLSI 2020; Perotti et al., 2023. Ara is an open-source RISC-V Vector (RVV 1.0) coprocessor designed to be attached to the CVA6 application core; Ara2 extends it for multi-lane scaling.

How it works. Ara executes vector instructions on 2–16 parallel lanes, each with its own slice of the vector register file, an FPU and integer units, plus a vector load/store unit that fetches data over AXI. Vector length is configurable at run time; the programming model is standard RVV.

What it solves. Open, synthesisable vector compute with a clean memory interface, suited to a custom SoC that includes a KV manager.

What it leaves open for an edge device. Ara is a compute engine and has no concept of KV blocks, tiers or codecs. An LLM kernel for Ara must be written and benchmarked on Ara itself.

Inference we drew

We placed our **fused QK kernel** on Ara: it loads packed 4-bit key bytes with vector byte loads, widens and converts them, and accumulates with fused multiply–add, without first writing an expanded floating-point key block to memory. This saves exactly the bandwidth that a dequantise-then-dot approach would waste. The kernel compiles for `rv64gcv` and lowers to the expected vector instructions; execution on Ara or a simulator is future work (Chapter 12).

CXL-SpecKV and other cache-engine proposals

arXiv preprints and project pages. CXL-SpecKV describes an FPGA-based cache engine that places KV data on CXL-attached memory with speculative fetch.

How it works. A hardware controller predicts and prefetches KV blocks from a larger, slower memory pool.

What it solves. Shows that a dedicated controller can hide slow-memory latency for KV data.

What it leaves open for an edge device. Targets data-centre CXL fabrics rather than an edge SoC.

Inference we drew

A hardware controller for the cache is a legitimate architecture. In UAMM-2 the per-block scoring and the block table are implemented as RTL so that the planner's inner loop does not occupy the CPU.

2.11 Deployment runtime: compiler-based runtimes

TVM-based compiler runtimes

Chen et al., OSDI 2018 (TVM). Compiler-based runtimes built on TVM are machine-learning compilation frameworks for deploying LLMs on many targets (phones, browsers, GPUs). It builds on TVM and its newer Unity design with dynamic-shape support.

How it works. Models are compiled into target-specific kernels; quantised weights are supported; the runtime exposes an OpenAI-style chat API and manages a KV cache internally.

What it solves. Real portability: the same model can be compiled for several hardware back ends.

What it leaves open for an edge device. Its KV cache is internal to the runtime. We have not verified the details of its KV-cache interface, nor that TVM can target RISC-V vector back ends such as Ara end to end.

Inference we drew

A runtime is an adapter target, not a design dependency. We therefore defined a narrow `RuntimeAdapter` interface (append token, get attended blocks, report) with a working reference adapter, and an `CompilerRuntimeAdapter` skeleton with the integration checklist. This keeps the research engine testable on its own while leaving a clear seam for deployment.

2.12 Coverage matrix

2.13 Summary of the gaps

1. **Eviction without precision control** (H2O, Scissorhands, StreamingLLM, SnapKV): loss is irreversible and kept tokens stay at full size.
2. **Quantisation without selection or placement** (KIVI, KVQuant): uniform precision; the whole cache is still read every token.

3. **Selection without capacity savings** (Quest): reads fewer bytes but needs everything resident.
4. **Offloading without compression** (FlexGen, InfiniGen): one slow tier; no joint precision decision.
5. **Joint ideas without an edge hardware realisation** (EvicPress, TensorRT-LLM framework): no verified decision hardware, no RISC-V vector path.
6. **Hardware engines without a policy layer** (compress engines, Ara): datapaths exist, but nothing decides what to compress and where to put it.

UAMM-2 is an attempt to fill the space between these gaps in one consistent design. The next chapters describe the starting point and the result.

3. First Approach: UAMM-1

Having surveyed the algorithms of Chapter 2, we observed that each solves one part of the problem in isolation: paging manages layout, H2O scores token importance, quantisation shrinks tensors, and selection skips reads. The first system we built, **UAMM-1** (Unified Adaptive Memory Manager, version 1), was an attempt to join the first two into one memory manager for an edge processor. This chapter describes everything UAMM-1 contained.

3.1 Components

3.1.1 Paged block store

The KV cache was divided into fixed-size blocks of consecutive tokens, as in PagedAttention. A block table recorded, for each logical block, the memory level that held it. A block was the unit of allocation, movement and scoring; per-token bookkeeping was avoided.

3.1.2 H2O-style importance

Following H2O, each token accumulated the attention weight it received at every decode step. The importance of a block was a function of the accumulated weights of its tokens. The idea is that a small set of heavy-hitter tokens receives most of the attention, so blocks containing them should be kept close to the processor.

3.1.3 Three-level placement

Three memory levels were modelled: a fast level near the processor, DRAM and a slow non-volatile level (flash). Each block received a combined score from three signals:

$$\textit{score} = 0.60 \cdot \textit{importance} + 0.25 \cdot \textit{recency} + 0.15 \cdot \textit{access frequency}$$

and was assigned a level by two fixed thresholds: a block with score ≥ 0.68 went to the fast level, a block with $0.35 \leq \text{score} < 0.68$ to DRAM, and a block below 0.35 to flash. When a level was full the block was pushed down to the next level (cascade demotion); when no level had room, the block was discarded.

Worked example

A block has importance 0.90, recency 0.50 and access frequency 0.40. Score = $0.60 \times 0.90 + 0.25 \times 0.50 + 0.15 \times 0.40 = 0.54 + 0.125 + 0.06 = 0.725$. Since $0.725 \geq 0.68$ it is placed in the fast level.

A second block has importance 0.20, recency 0.30 and frequency 0.10: score = $0.12 + 0.075 + 0.015 = 0.21 < 0.35$, so it goes to flash. If flash is also full, the block is dropped.

3.1.4 Hardware decision engine

A small Verilog module computed the weighted score and the level decision for one block at a time, intended to offload the arithmetic from the processor. It was checked by a testbench with three input vectors.

3.1.5 Software model

A Python simulator reproduced the policy, advanced the H2O scores token by token, and estimated latency from the level each block resided in.

3.2 What UAMM-1 achieved

UAMM-1 showed that importance-driven tiering is workable: hot blocks stay near the processor, cold ones move away, and the decision can be made cheaply in hardware. It also supplied the structure that UAMM-2 keeps: blocks, a block table, a hot/warm/cold classifier and its RTL scorer. For this reason the original threshold policy was retained in the code as a baseline (`ThresholdPlanner`), so that UAMM-1 and UAMM-2 can be compared on identical inputs.

4. Gaps in UAMM-1

Combining paging and importance scoring was a first step, but UAMM-1 inherited the limits of the algorithms it combined and added defects of its own. This chapter identifies each gap, first at the level of ideas and then at the level of implementation, and gives the evidence for its consequence.

4.1 Gaps at the level of ideas

Table 7. *Gaps at the level of ideas.*

Gap	Explanation	Consequence
No quantisation	H2O-style importance only decides where a block lives or whether it is deleted. Kept data stays at full 16-bit precision. Quantisation methods such as KIVI, which would shrink kept data, were not part of the system.	When the budget is smaller than the FP16 data, the only remedy is deletion
Placement without precision	Importance and placement were linked, but placement and precision were not. A block could not be stored smaller in a slower level.	Slow levels hold full-size blocks and are quickly exhausted
No selective reading	All retained blocks were assumed to be read at every step. There was no query-aware rule, as in Quest, for skipping blocks that cannot matter.	Bandwidth stays proportional to the retained cache
No measure of codec loss	Because no codec existed, nothing estimated how much error a smaller format would cause for a given block.	No principled trade-off between size and accuracy
Irreversible, silent loss	Deletion happened whenever the lowest level was full, without a flag or an opt-in.	Output quality degraded with no visible signal
Isolated hardware	The decision engine scored one block at a time and had no memory of block metadata or a translation table.	The software could not hand a whole table to the hardware

4.2 Defects in the implementation

Table 8. *Implementation defects found on review and the response to each.*

Defect	Why it matters	Response in UAMM-2
The eviction decision was computed but never acted on, and data was deleted silently when the lowest level was full	Output changes without notice	Drop is explicit and opt-in; otherwise overflow is reported as a number
Importance was clamped at 1.0 although H2O scores are sums that grow without bound	Heavy hitters and sinks saturate; the signal stops discriminating	Scores decay each step; block importance is normalised by the 95th percentile
Recency was constant or unused	A quarter of the score carried no information	Recency enters through an exponential average of how often attention fetched the block
No capacity-aware cascade and no hysteresis	Blocks oscillate between levels; each move costs energy	Greedy capacity-aware assignment, minimum residency and a relative-improvement margin

Defect	Why it matters	Response in UAMM-2
Latency charged per token, migrations free	Cost of moving data was invisible	Cost per block per level and codec; migrations accounted separately
O(N) Python loop per token for H2O	Does not scale with context	One vectorised decay per step
Division by 100 in Verilog	Dividers are large and slow	Integer weights 154/64/38 (sum 256) and a shift
Verilog and Python disagreed on protected blocks	Hardware and software make different decisions	RTL honours the protected flag; bit-exact golden model
Three test vectors, no boundary cases	Little confidence in the RTL	17 sweeps with threshold edges, 400 codec vectors, 6,000 random table operations, mutation testing
SmoothQuant listed as a KV-cache solution	It targets weights and activations	Replaced by KV-specific KIVI-style codecs

4.3 Evidence: what the gaps cost

The experiments of Chapter 8 measure the consequence. With a total budget of 168 KB for 1,536 tokens (FP16 would need 393 KB), the UAMM-1 policy keeps 0.37 of the FP16 footprint only by permanently deleting 973 of the 1,536 tokens on average. The remaining blocks then retain only 0.76 of the attention mass, the mean relative error of the attention output is 0.48, and about 12,613 block reads are served from flash. A system with codecs, by contrast, keeps every token at 0.31 of the FP16 footprint with 0.94 recall. The gap is therefore not in the placement rule but in the absence of a way to make data smaller and of a way to read less.

Summary of what was left unresolved

1. No quantisation, so a tight budget forces deletion.
2. Placement and precision decided separately (placement only).
3. No query-aware selection of what to read.
4. No measured error per block, hence no principled quality control.
5. Silent, irreversible loss and several implementation defects.
6. Hardware that was neither connected to a block table nor adequately verified.

5. The Combined System: UAMM-2

This chapter explains the combined design. It begins with the core idea, then walks through the data model, the codecs, the importance signal, the planner, selective attention, the hardware blocks, the vector kernel and the runtime seam.

5.1 The core idea: a placement is a (tier, codec) pair

Every block of the KV cache is described by two coordinates. The **tier** says *where* it is stored (and therefore what a read costs). The **codec** says *how many bits* it occupies (and therefore how big it is and how much error it carries). The planner picks both together. A block that is almost never attended is best stored in flash as a 2-bit entropy-coded blob. A block that is hot but large can stay in DRAM at 4 bits. Protected blocks (sinks, recent window) sit in the fastest tier at full precision. Choosing the two jointly lets the system exchange precision for capacity instead of exchanging data for nothing.

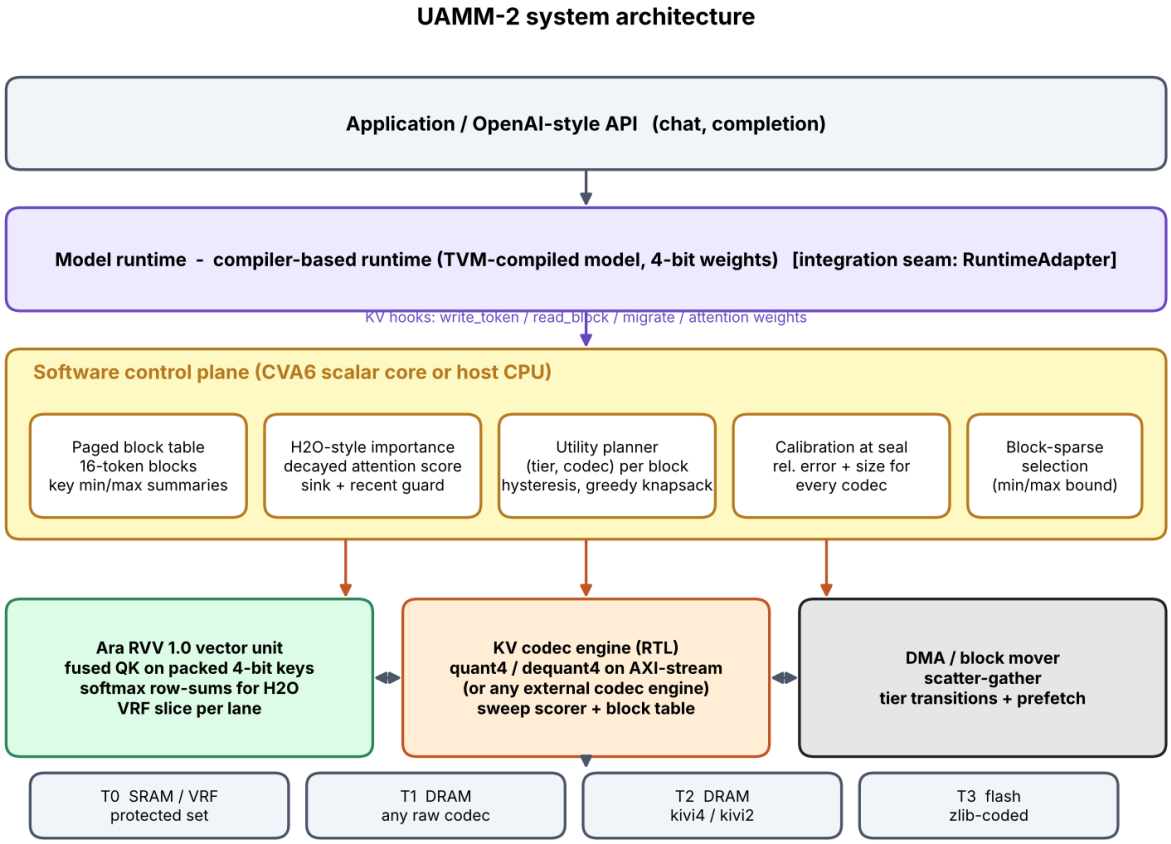


Figure 2. *UAMM-2 system architecture: the decode loop, the paged cache with its tiers and codecs, the importance and planning loop, and the hardware and vector-kernel attachments.*

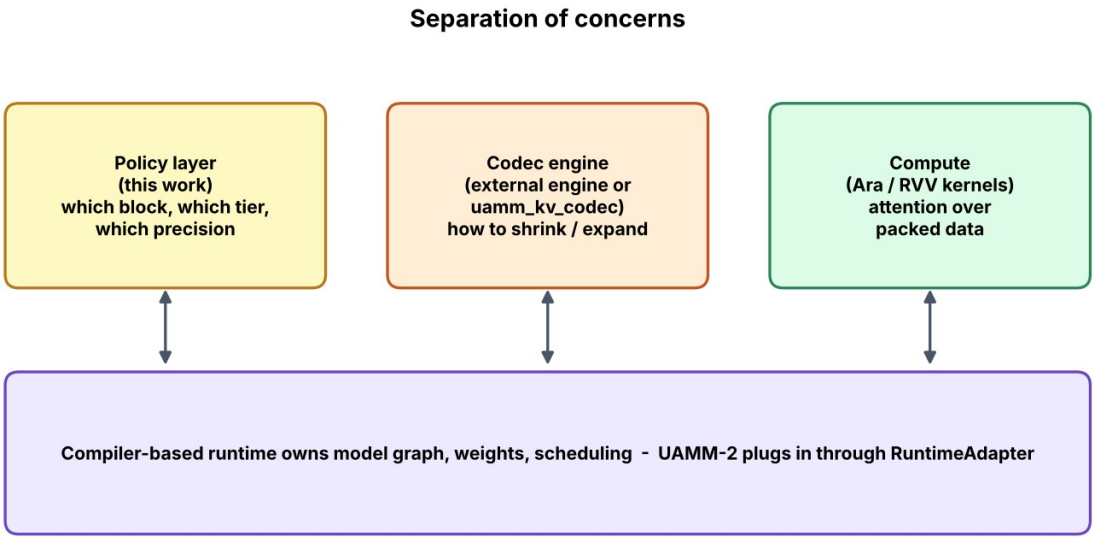


Figure 3. *Separation of concerns. The engine is split so that each part can be tested, replaced or moved to hardware independently.*

5.2 Memory hierarchy and tiers

Table 9. *Tiers used in the simulations. Capacities are deliberately small so that the budget is tight at a few thousand tokens; latencies and bandwidths are assumed values (Chapter 7).*

Tier	Where it models	Capacity in experiments	Base latency / bandwidth (assumed)	Allowed codecs	Intended contents
T0	Near-compute SRAM / vector register file	24 KB	0.5 µs / 100 GB/s	any raw codec	Sink tokens, recent window, unsealed block
T1	DRAM, hot	48 KB	2 µs / 12.8 GB/s	any raw (non-entropy) codec	Frequently fetched blocks; also where 2-bit blocks sit if the planner prices them there
T2	DRAM, warm	32 KB	2 µs / 12.8 GB/s	kivi4, kivi2	Occasionally fetched blocks
T3	Flash	64 KB	100 µs / 1 GB/s	int8z, kivi4z, kivi2z (entropy-coded)	Cold blocks

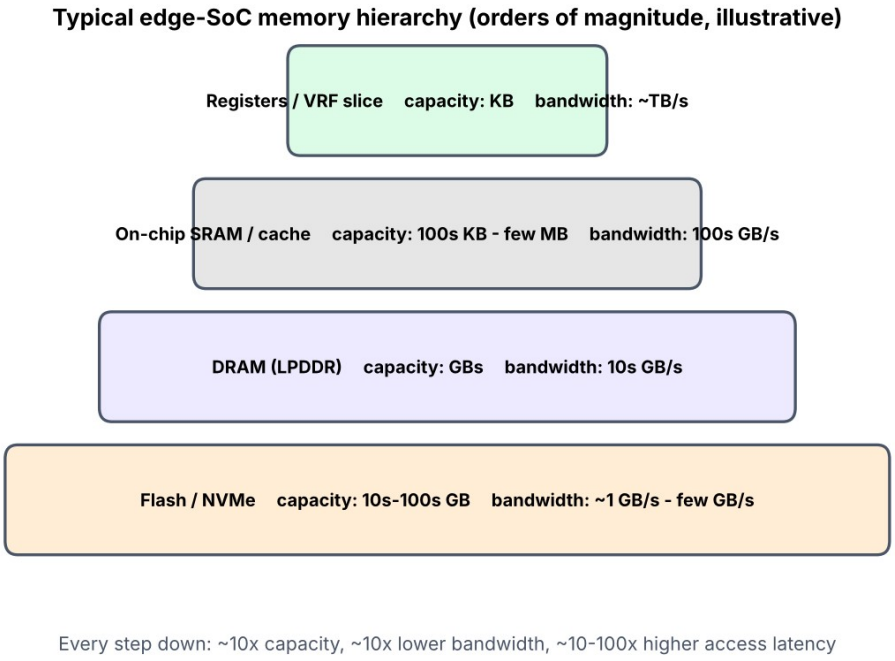


Figure 4. *The memory hierarchy used by the model, with latency and bandwidth classes.*

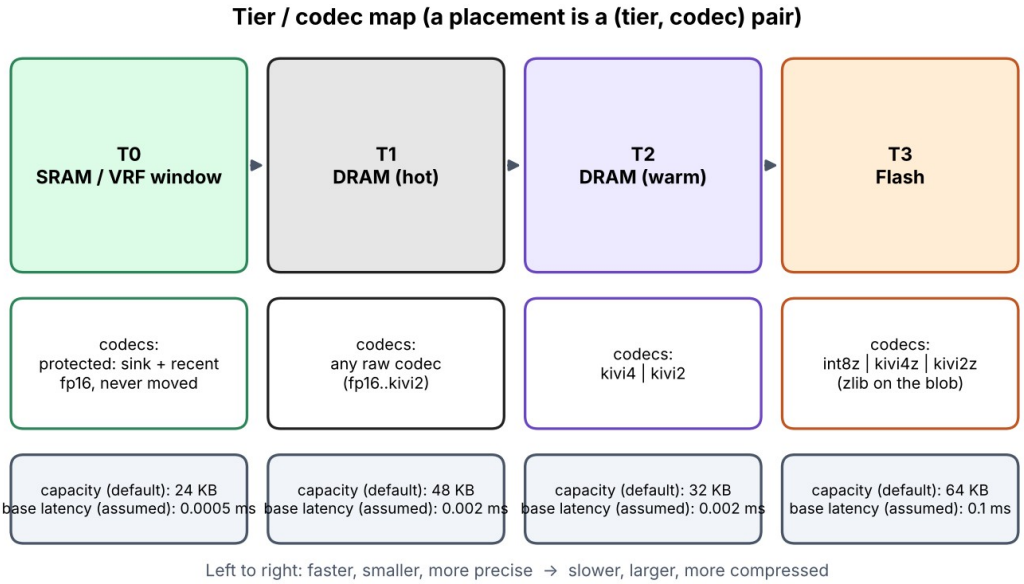


Figure 5. *Which codecs are permitted in which tier. Flash only accepts entropy-coded variants; the warm DRAM tier only accepts 4- and 2-bit codecs.*

The tier restrictions are policy, not physics. They encode two beliefs: that holding full-precision data in a tier labelled "warm" wastes the capacity that tier exists to provide, and that entropy coding (zlib) is only worth its decode time where the access cost is already high.

5.3 The block and its lifecycle

A block holds 16 consecutive tokens (configurable). While it is filling it is *open*: staged as raw vectors in the fastest tier and always read. When the 16th token

arrives it is **sealed**: the K and V vectors are rounded to FP16 (this rounded copy is the ground truth from then on), encoded, and the following summaries are computed and kept resident because they are tiny:

- `kmin`, `kmax`: element-wise minimum and maximum of the block's keys (64 values each in the experiments);
- `err_table[codec]`: the measured relative squared error obtained by actually encoding and decoding this block with every codec;
- `size_table[codec]`: the measured byte size of the block under every codec;
- `importance`, `hit_ema`, `access_count`, `last_move_step`, `protected`: scheduling state.

Calibration by real encoding, rather than by a formula, is deliberate. The error of a codec depends strongly on the block's content (for example on outlier channels), and the planner should know what *this* block will lose, not an average. The cost is one extra encode per codec at seal time, which is a background cost in a real system.

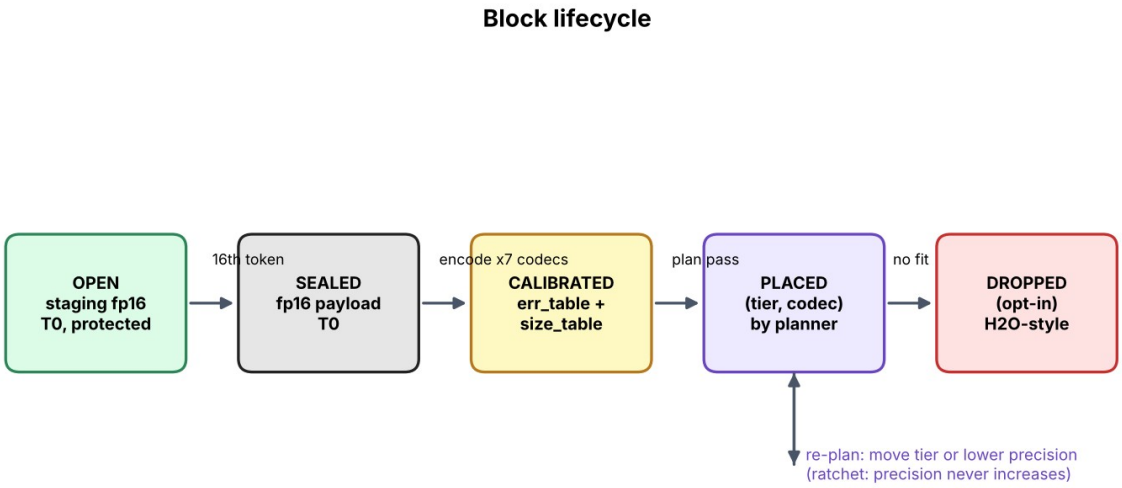


Figure 6. *Lifecycle of a block: open, sealed with calibration, and then moved or re-encoded by the planner; protected blocks never move.*

5.4 Codecs: KIVI-style asymmetric quantisation

The codec layer turns a block of T tokens × d channels of keys and values into a byte payload and back. Four precision families exist: **fp16** (reference), **int8**, **kivi4** and **kivi2**; each non-FP16 family also has an entropy-coded variant with suffix `z` (zlib over the serialised blob), used on flash.

5.4.1 Asymmetric affine quantisation

For a set of values with minimum m and maximum M, and b bits (so L = 2^b − 1 levels), the scale and zero are s = (M − m)/L and z = m. A value x is stored as q = clip(round((x − z)/s), 0, L) and reconstructed as $\hat{x} = q \cdot s + z$. Scales and zeros are stored as FP16 (scale floored at 2^−10 to avoid division by zero). “Asymmetric”

means that the zero point is the true minimum rather than zero, which uses all levels even when the data are not centred on zero.

5.4.2 Keys per channel, values per token

Following KIVI, **keys are quantised per channel**: for each of the d channels one scale and one zero are computed across the block's tokens. **Values are quantised per token**: each token's value vector is split into groups of 32 channels and each group has its own scale and zero. Per-channel quantisation of keys isolates the few channels with large magnitude so they do not stretch the range of the others; values have no such outlier structure and are naturally dequantised token by token as attention weights multiply them.

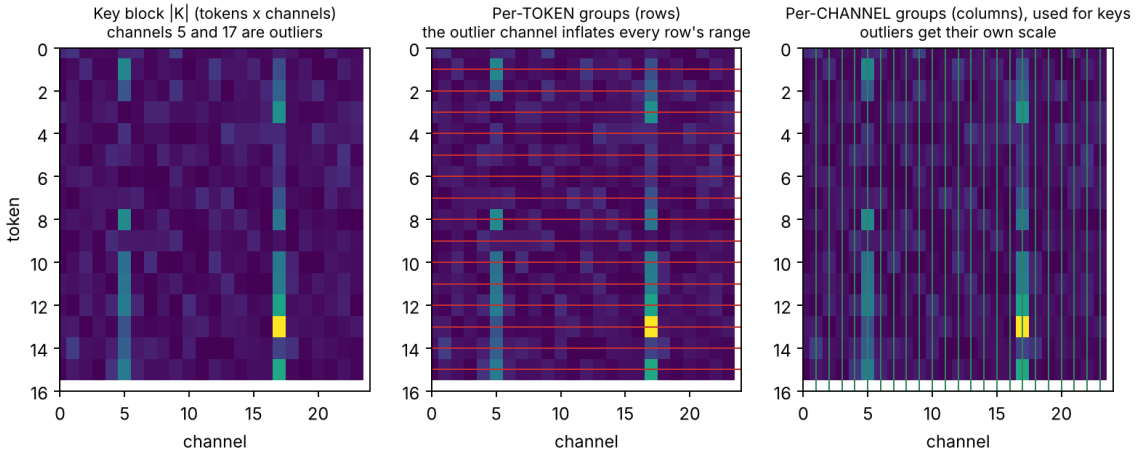


Figure 7. *Quantisation axes. Keys share parameters down a channel (across tokens); values share parameters across a group of channels within one token.*

5.4.3 Packing

Quantised integers are packed little-endian into bytes: two 4-bit values per byte (element 0 in the low nibble) or four 2-bit values per byte. The same convention is used by the RTL nibble packer and by the RVV kernel, so a block produced by the Python codec can be consumed unchanged by the hardware and the kernel.

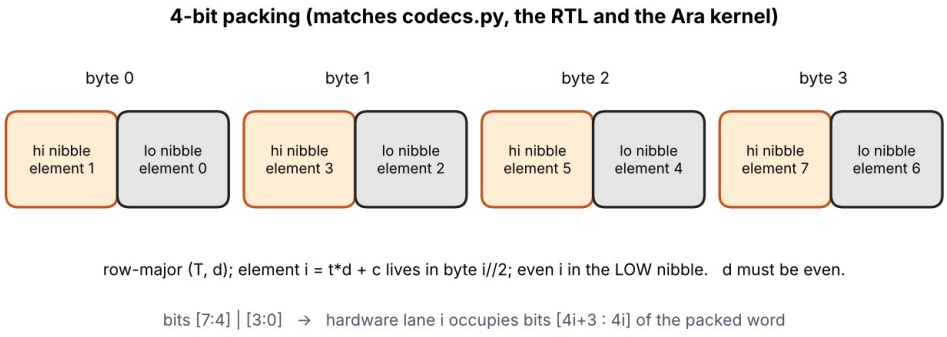


Figure 8. *Bit packing of 4-bit and 2-bit values into bytes. The even element is in the low bits.*

5.4.4 Payload layout and measured sizes

The serialised payload is `[packed K][packed V][K scales][K zeros][V scales][V zeros]`. For one block of 16 tokens × 64 channels (2,048 K elements + 2,048 V elements) the measured sizes are given in the table below: metadata is not free, which is why a 4-bit block is 0.344 of FP16 rather than 0.25.

Table 10. *Measured size and reconstruction error per codec for a 16 × 64 block of K and V (mean over test blocks; synthetic data, see Chapter 7). The z variants reduce size slightly on flash-like data; for 4-bit and 2-bit they add a few bytes of overhead on random-like blocks.*

Codec	Bytes / block (avg)	Ratio to FP16	Rel. sq. error (no outlier channels)	Rel. sq. error (with outlier channels)
fp16	4096	1.000	4.30e-08	4.24e-08
int8	2432	0.594	1.80e-05	1.62e-05
kivi4	1408	0.344	5.18e-03	4.68e-03
kivi2	896	0.219	1.30e-01	1.18e-01
int8z	2407	0.588	1.80e-05	1.63e-05
kivi4z	1419	0.346	5.17e-03	4.66e-03
kivi2z	905	0.221	1.31e-01	1.20e-01

5.4.5 Precision ratchet

Re-encoding a lossy block decodes it and encodes it again, so errors compound. The cache therefore enforces a **one-way ratchet**: a block can be moved to a codec of equal or lower precision, never higher. Promoting a 4-bit block to a "fast tier" changes where it lives, not how accurate it is. The only same-precision transformations allowed are toggling the entropy wrapper, which is a lossless blob operation. This rule makes the quality effect of any sequence of planner decisions bounded and easy to reason about; it also means the planner must be cautious about early demotion (see Chapter 12).

5.5 The importance signal: H2O reinterpreted

Each decode step produces attention weights over the tokens that were actually read. The engine updates a score per token: it multiplies all scores by a decay (0.95 by default) and then adds the new attention weight to the tokens that received it. A block's importance is the aggregate (mean by default) of its tokens' scores, divided by the 95th percentile of block scores and clipped to [0, 1]. Normalising by the 95th percentile prevents the enormous score of the attention-sink block from forcing every other block's importance toward zero. A second signal, `hit_ema`, is an exponential moving average of whether the block was selected for fetching at each step.

The **heat** of a block combines them: heat = ½ · hit_ema + ½ · importance. Importance reflects long-run attention mass; hit_ema reflects recent fetch behaviour. Using both gives the planner a stable and a responsive signal.

How this differs from H2O

In H2O the score decides which tokens are *deleted*. Here it decides where a block is *stored* and at what precision. The signal is the same; the consequence of acting on it is reversible-or-bounded instead of irreversible. Permanent drop remains available (`allow_drop`, importance below 0.02) but is off in all headline experiments.

5.6 The utility planner

Every `plan_interval` steps (default 8) the planner re-evaluates every sealed, unprotected block. It considers every (tier, codec) candidate that the tier rules allow and whose precision is not higher than the block's current precision, and computes an expected cost in milliseconds-equivalent:

$$cost(b,\ t,\ c) = p_access(b) \cdot [read_ms(t,\ c,\ bytes) + decode_ms(c,\ bytes)] + \lambda \cdot heat(b) \cdot err_b(c) + \kappa \cdot (bytes\ /\ cap_t) \cdot (1 + 4 \cdot fill_t)$$

- **Latency term.** p_access is the block's `hit_ema` (floored at 0.01); the read cost is the tier's fixed latency plus bytes over bandwidth plus decode time. A block that is rarely read pays little for living in flash.
- **Quality term.** λ (default 2.0) converts measured codec error, weighted by the block's heat, into the same unit as latency. A hot block with a high-error codec is penalised; a cold block is almost free to compress.
- **Congestion term.** κ (default 0.05) charges for the share of the tier that the block would occupy, scaled up as the tier fills, so that the planner does not exhaust the fast tiers early.

Blocks are visited in descending heat so that the hottest blocks get first choice of fast tiers. Each takes the cheapest candidate that fits the remaining capacity (with 3 % headroom). If nothing fits, the block stays where it is and the overflow is counted. This greedy procedure is a heuristic, not an optimal knapsack solution.

5.6.1 Hysteresis

Without damping, a block whose cost estimate fluctuates would bounce between tiers, paying migration cost each time. Two rules stop that: a block may not move again within `min_residency_steps` (default 16) of its last move, and it only moves if the best alternative is cheaper than staying by more than `hysteresis_margin` (default 25 %) of the best cost. Chapter 8 measures the effect.

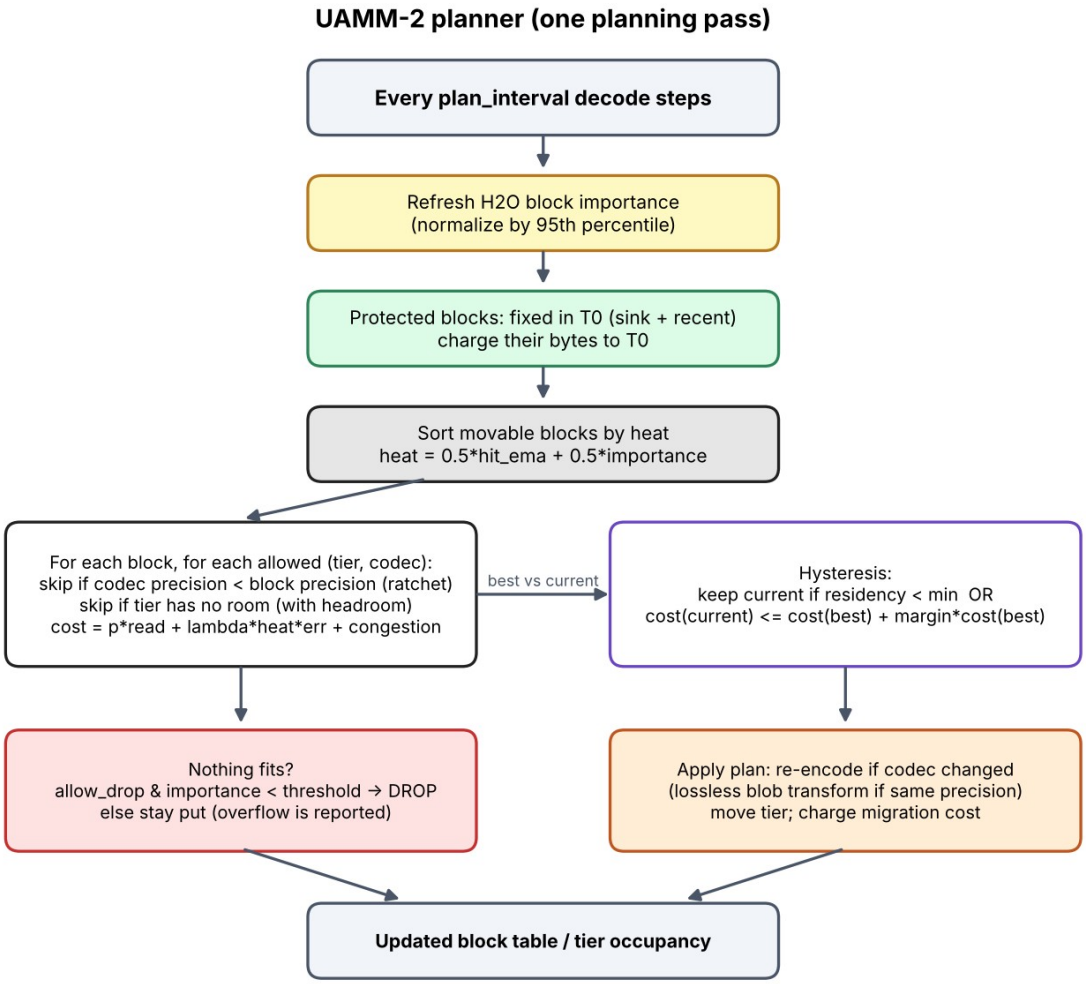


Figure 9. *Planner flow: candidate enumeration, cost evaluation, greedy capacity-aware selection and the hysteresis check.*

5.7 Selective attention: reading less

At each decode step, protected blocks (sinks, recent window and the open block) are always read. Of the remaining sealed blocks, the engine reads only the `attn_block_budget` (default 24) blocks with the largest upper bound on the query–key score, computed from the resident kmin/kmax summaries:

$$bound(b) = \Sigma_c \; max(\; q_c \cdot kmin_c \;,\; q_c \cdot kmax_c \;) \geq q \cdot k \quad for\ every\ key\ k\ in\ b$$

The bound holds because each product q_c · k_c, for k_c between kmin_c and kmax_c, is at most the larger of the two endpoint products. A block with a low bound cannot contain a key that scores highly against this query, so it is neither fetched nor decoded. The test suite checks the bound property on random data. Selected blocks are decoded and attention is computed over the union of protected and selected tokens.

This is where capacity savings and bandwidth savings combine. Because skipped blocks are never touched, they can be stored in the cheapest (slowest, smallest) tier and codec without most steps ever paying for that choice; and because tier and codec

are chosen on measured heat, the blocks that are touched are the ones that the system placed where they are cheap to read.

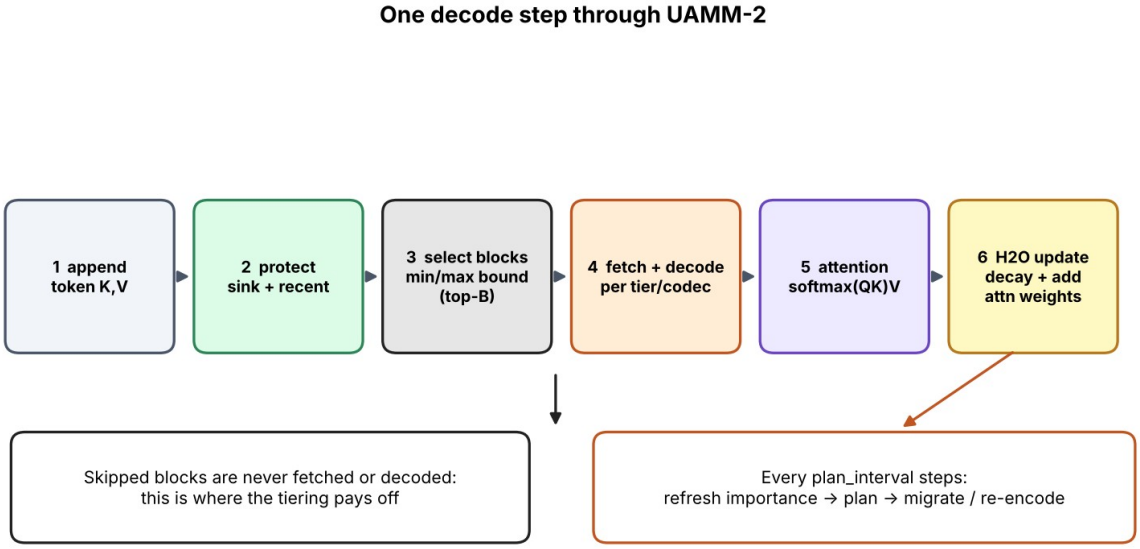


Figure 10. *One decode step: bound ranking, fetch of selected blocks, attention, H2O update and (every few steps) a planning pass.*

5.8 Hardware: three verified RTL blocks

Three modules were written in synthesisable Verilog-2001. Each has a bit-exact Python golden model (`uamm2/hwmodel.py`), generated test vectors and a self-checking testbench run in Verilator. They accelerate the *inner loops* of the system; the utility planner itself remains software.

5.8.1 Sweep scorer (`uamm_sweep_scorer`)

The scorer is the hardware heat classifier derived from UAMM-1. Block metadata is held in a small SRAM as a 25-bit word `{protected, frequency[7:0], recency[7:0], importance[7:0]}`. A `start` pulse latches an 8-bit memory-pressure value and launches a **sweep**: every block is read (one per cycle), passes through a 2-stage pipeline, and emerges as a result record `{idx, score, tier, evict}`. The score is computed without a divider:

$$score = (\ 154 \cdot imp + 64 \cdot rec + 38 \cdot frq\ ) >> 8 \qquad (154 + 64 + 38 = 256)$$

The tier is hot if the block is protected or score ≥ 174, warm if score ≥ 89, otherwise cold. A block is flagged for eviction only if it is **not** protected, the latched pressure is at least 243, and its score is below 89. A sweep over 64 blocks has latency NBLK + 3 cycles and completes with a `done` pulse; a `start` received while busy is ignored.

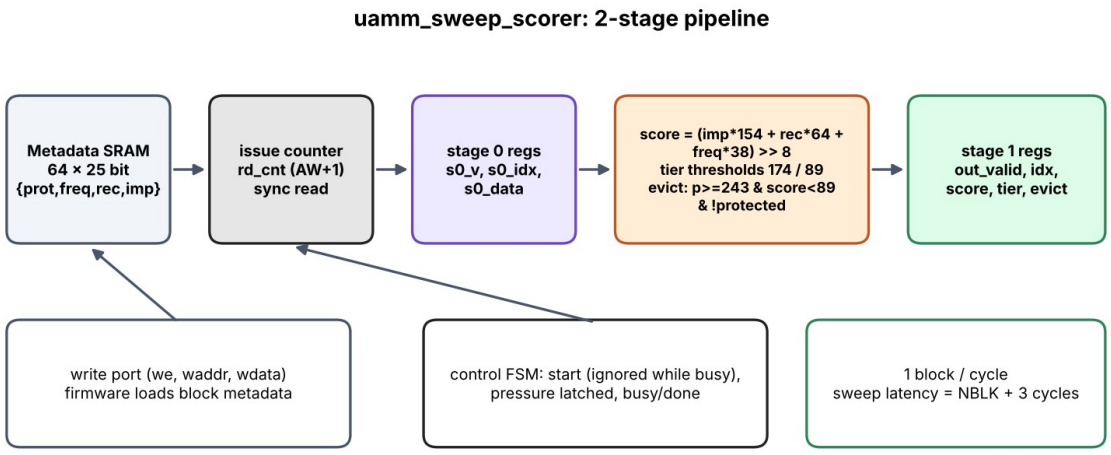


Figure 11. *Sweep scorer micro-architecture: metadata SRAM, weighted sum, threshold comparators, protected override and eviction logic.*

5.8.2 KV codec pipeline (`uamm_kv_quant4`, `uamm_kv_dequant4`)

The quantiser processes 16 byte-wide lanes per beat. Software or firmware supplies per-group parameters derived from the group's min and max: the zero point `zp`, a Q4.12 reciprocal `inv_q12 = round(15·4096/range)` and a Q8.8 step `step_q8 = round(range·256/15)`. Quantisation is $q = \text{clamp}(((x - zp) \cdot inv + 2048) >> 12, 0, 15)$ with $x < zp$ clamped to zero; dequantisation is $x' = \min(255, zp + ((q \cdot step + 128) >> 8))$. Each module has one pipeline register. The output is packed with lane i in nibble i, matching the software packing. The intended location is on the AXI stream between memory and Ara's vector load/store unit, so that (de)quantisation happens in the load path rather than in vector instructions.

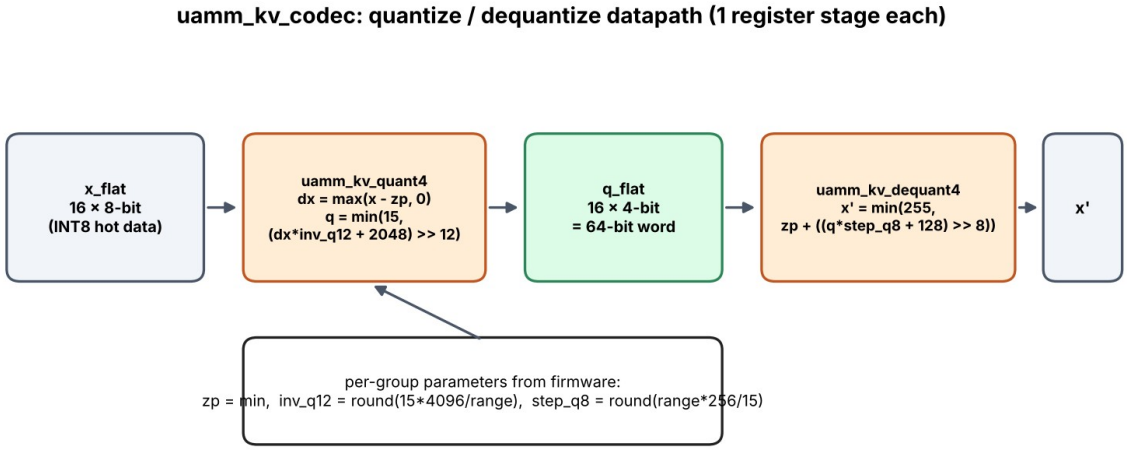


Figure 12. *Codec datapath: sixteen subtract–multiply–round–clamp lanes feeding a nibble packer, and the inverse path.*

5.8.3 Block table (`uamm_block_table`)

A direct-mapped translation table maps a logical block index (8 bits, 256 entries) to `{tier[1:0], codec[1:0], phys[11:0]}` with a separate valid bit that is cleared by reset.

A lookup is a one-cycle synchronous read that returns the entry and a `hit` flag. A read in the same cycle as a write to the same address returns the **old** entry, a property fixed by design and verified by test.

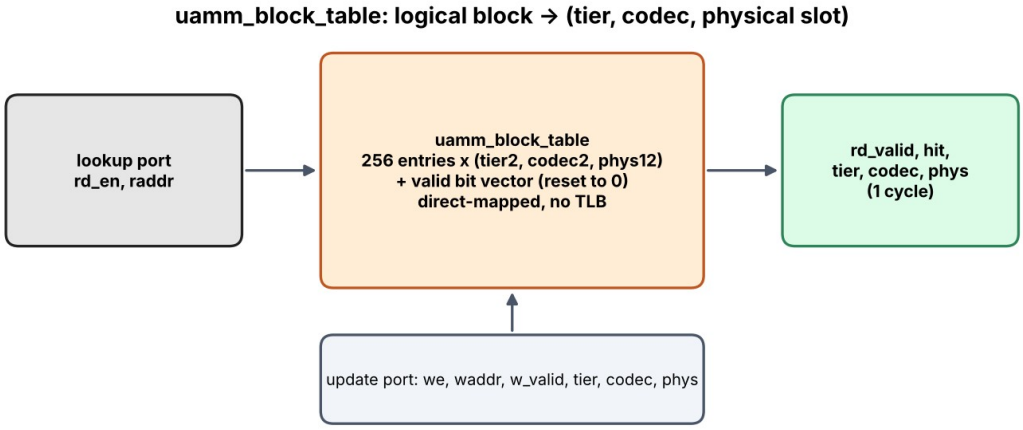


Figure 13. *Block-table structure: entry array, valid bits and the update and lookup ports.*

5.9 The vector kernel for the Ara vector processor

Ara is a RISC-V Vector coprocessor. Attention scores for a block of 4-bit keys can be computed without first dequantising the keys into floating point. Because the key for token t at channel c is x = q · s_c + z_c, the dot product with a query splits into:

$$q \cdot k_t = \Sigma_c \, (\, q_c \cdot s_c \,) \cdot Q[t,c] \;+\; \Sigma_c \, q_c \cdot z_c$$

The first term uses the integer codes Q and a query vector pre-scaled by the per-channel scales (computed once per block); the second term is a constant for the block (the **bias**). The kernel therefore folds scales and zeros into the query once, then, for each token, loads packed bytes with a vector byte load, extracts the low and high nibbles (even and odd channels), zero-extends and converts them to float, and accumulates with fused multiply–add into two pre-scaled query halves, finishing with a vector reduction. No expanded floating-point key block is ever written to memory.

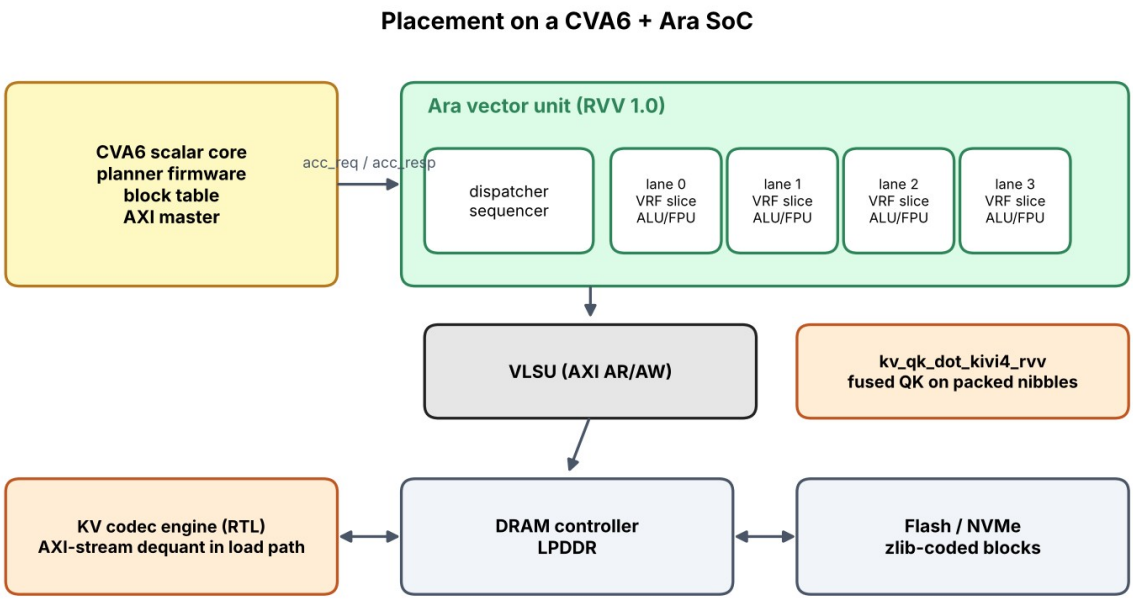


Figure 14. *Integration of the codec engine and the fused kernel with Ara and the memory system.*

A scalar version of the kernel with identical data layout runs on the host and is checked against NumPy (Chapter 10). The RVV version uses the standard intrinsics and compiles for `rv64gcv`. It has not been executed (Chapter 12).

5.10 The runtime seam and the codec slot

A narrow `RuntimeAdapter` interface (write token, read block, migrate block, report attention importance) separates the engine from any inference framework. `ReferenceAdapter` implements it on the NumPy engine and is exercised in the tests. `RuntimeAdapter skeleton for a compiler-based runtime` is a skeleton whose docstring lists the integration steps: map to the runtime's KV pages, obtain attention weights (fused attention kernels do not normally expose them, so a row-sum output or a side kernel is needed), perform synchronised tensor movement, and confirm that the compiler can target RVV.

The codec layer exposes `encode(k, v, codec, v_group) → Payload` and `decode(Payload) → (k, v)`. The group quantiser and outlier handling of a hardware compression engine, or any other codec, can be wired in by implementing the same interface and registering a codec name.

6. Project Structure and Code Walkthrough

This chapter explains how the project is organised and what each part of the code does, so that a reader can find any behaviour described in this report.

6.1 Directory layout

```
uamm2_project/
  README.md  requirements.txt
  uamm2/                 Python reference engine
    config.py              all parameters and the assumed cost constants
    codecs.py              quantisation, packing, entropy coding, calibration
    paged_cache.py         blocks, sealing, re-encoding, dropping
    tiers.py               tier usage and the latency model
    h2o.py                 attention-based importance
    attention.py           block selection, sparse attention, dense oracle
    planner.py             utility planner and the original threshold planner
    simulator.py           decode loop, metrics, per-step trace
    workload.py            synthetic query/key/value generator
    ablation.py cli.py __main__.py     variants and command line
    hwmodel.py             bit-exact models of the RTL + test-vector writer
    ara.py                 host build of the C kernel; RISC-V compile check
    runtime.py             runtime adapter interface, reference adapter, skeleton
  hw/
    rtl/                   uamm_sweep_scorer.v  uamm_kv_codec.v  uamm_block_table.v
    tb/                    self-checking testbenches (+trace writes VCD files)
    vectors/               generated stimulus and expected outputs (hex)
    Makefile               test, lint, vectors, clean
  ara/                     kv_kernels.c / .h : scalar and RISC-V vector kernel
  tests/                   56 pytest tests
  experiments/             run_experiments.py, mutation_test.py, figure scripts, results/
  docs/                    waveform dumps, synthesis reports, captured tool output
  report/                  scripts that build this document
```

6.2 Python engine, module by module

Table 11. *Python modules.*

File	Main items	What it does
`config.py`	`Config`, `TierSpec`	Holds geometry (64 channels, 16-token blocks), protection (4 sinks, 32 recent), planner settings (interval 8, hysteresis 0.25, residency 16, $\lambda = 2$), codec set and the assumed latency, bandwidth and decode constants. Validates inputs on creation.
`codecs.py`	`encode`, `decode`, `calibrate`, `pack_bits`, `parse_codec`	Turns a block of keys and values into a byte payload: asymmetric quantisation (keys per channel, values per token in groups of 32), bit packing, optional zlib. `calibrate` encodes a block with every codec and returns its error and size tables. Codec names: fp16, int8, kivi4, kivi2 and their `z` forms.
`paged_cache.py`	`KVBlock`, `PagedKVCache`	`append_token` fills the open block; at 16 tokens `_seal` rounds to FP16, stores the payload, computes key min/max and calibrates. `reencode` changes codec (lossless for the entropy wrapper, lossy otherwise); `drop` removes a block.
`tiers.py`	`usage`, `read_cost_ms`, `move_cost_ms`, `decode_cost_ms`	Accounts bytes per tier and computes the modelled read, decode and migration costs. No data actually moves.
`h2o.py`	`H2OImportance`	`update` decays all token scores and adds the attention weight

File	Main items	What it does
		each token received; `refresh` aggregates to blocks (mean, max or sum) and normalises by the 95th percentile.
`attention.py`	`select_blocks`, `sparse_attention`, `dense_oracle`	Ranks candidate blocks by the key-bound formula and returns the top budget; computes attention over the read blocks; the oracle computes full FP32 attention for the error metrics.
`planner.py`	`UtilityPlanner`, `ThresholdPlanner`, `codec_allowed`	The utility planner enumerates allowed (tier, codec) pairs, evaluates the cost, visits blocks by heat, applies capacity and hysteresis and returns a plan. The threshold planner reproduces the original policy for comparison.
`simulator.py`	`Simulator.run`	For every token: generate q/k/v, append, update protection, select blocks, run sparse and dense attention, record metrics, update hit rates and importance, and every 8 steps plan and apply. Writes a per-step trace.
`workload.py`	`SyntheticWorkload`	Generates sinks, outlier channels, topic phases and local recency (Chapter 7).
`ablation.py`, `cli.py`	`variants`, `main`	Defines the ablation variants; commands `simulate`, `ablate`, `hwvectors`.
`hwmodel.py`	`scorer_model`, `quant4_model`, `dequant4_model`, `write_vectors`	Integer-exact copies of the RTL arithmetic and the generator of boundary and random vectors that the testbenches read.
`ara.py`	`AraKernels`, `rvv_compile_check`	Builds the C kernel for the host and calls it through ctypes to score packed 4-bit keys; compiles the vector path for a RISC-V target.
`runtime.py`	`RuntimeAdapter`, `ReferenceAdapter`	Interface to an inference runtime (write token, read block, migrate block, report attention); a working reference implementation and a skeleton for a compiler-based runtime.

6.3 Worked examples of the main routines

6.3.1 Sealing and calibration (`paged_cache.py`, `codecs.py`)

When the sixteenth token of a block arrives, `_seal` runs. (1) The staged K and V rows are stacked and rounded to FP16; this rounded copy is the reference from then on. (2) `kmin` and `kmax` are taken over the 16 key rows. (3) `calibrate` encodes the block with each enabled codec (fp16, int8, kivi4, kivi2 and the `z` forms), decodes it again and records the relative squared error $\|x - \hat{x}\|^2 / \|x\|^2$ and the byte size per codec. (4) The block is stored as an FP16 payload in tier 0. For a typical 16×64 block the size table reads about 4,096 B (fp16), 2,432 B (int8), 1,408 B (kivi4) and 896 B (kivi2), and the error table about 4×10^{-8}, 2×10^{-5}, 5×10^{-3} and 0.13.

6.3.2 Selecting blocks (`attention.py`)

For a query q and a candidate block with key minimum kmin and maximum kmax, `select_blocks` computes bound(b) = Σ_c max(q_c · kmin_c, q_c · kmax_c). Suppose d = 2, q = (2, −1), and a block with kmin = (0, 1), kmax = (3, 4). The bound is max(0, 6) + max(−1, −4) = 6 − 1 = 5; every key inside the box [0,3] ×

[1,4] has $q \cdot k \leq 5$. The candidates are ranked by the bound and the best `attn_block_budget` are read; the protected blocks are always added.

6.3.3 Updating importance (`h2o.py`)

After each step the weights w_i of the tokens that were read are available from the softmax. `update` multiplies all token scores by the decay (0.95) and adds w_i to the tokens read. Every plan interval `refresh` computes the mean token score per block, divides by the 95th percentile over blocks and clips to [0, 1]. A block that is not read decays: after 20 steps its score is $0.95^{20} \approx 0.36$ of its former value.

6.3.4 One planning decision (`planner.py`)

Consider a block with hit_ema 0.30 and importance 0.50, so heat = 0.40, with tables err(kivi4) = 0.005, err(kivi2) = 0.13, size(kivi4) = 1,408 B and size(kivi2) = 896 B. With $\lambda = 2$ the quality terms are $2 \times 0.40 \times 0.005 = 0.004$ ms-equivalent and $2 \times 0.40 \times 0.13 = 0.104$. The latency terms are the access probability times the read cost. Reading the 4-bit block from DRAM costs 0.002 ms latency + 1,408 B / 12.8 GB/s (0.0001 ms) + decode 0.0002 ms $\approx$ 0.0023 ms, times 0.30 = 0.0007; from flash it costs about 0.1 + 0.0014 + 0.0002 $\approx$ 0.1016 ms, times 0.30 = 0.030. Total (DRAM, kivi4) $\approx$ 0.0047; (flash, kivi2) $\approx$ 0.030 + 0.104 = 0.134. The planner therefore prefers DRAM at 4 bits, unless DRAM is full, in which case the congestion term and the capacity check push the block to the next feasible pair. Finally the hysteresis rule applies: if the block already sits in a pair whose cost is within 25 % of the best, or moved fewer than 16 steps ago, it stays.

6.3.5 Decode loop (`simulator.py`)

1. Generate q, k, v for token t and append the token to the cache (a block is sealed on the 16th token).
2. Mark protected blocks: those holding the first four tokens, the last 32 tokens, or the open block.
3. Rank the other sealed blocks by the key bound and select the budget; read protected plus selected blocks and compute sparse attention.
4. Compute the dense FP32 attention over all tokens (the oracle) and record error, recall and modelled fetch time.
5. Update each selected block's hit rate, and the token scores with the step's attention weights.
6. Every eighth step refresh importance, run the planner, apply the plan (re-encode and move) and record tier bytes, codec counts and counters.

6.4 Hardware

Table 12. *Hardware files.*

File	What it does
`uamm_sweep_scorer.v`	Metadata SRAM (25-bit words) and a 2-stage pipeline. A `start` pulse latches the pressure value and streams all blocks through `score = (154·imp + 64·rec + 38·frq) >> 8`, the tier decision (protected override, thresholds 174 and 89) and the eviction flag (pressure ≥ 243, score < 89, not protected).
`uamm_kv_codec.v`	`uamm_kv_quant4`: 16 lanes of subtract, multiply by the Q4.12 reciprocal, round, clamp to 15, pack nibbles. `uamm_kv_dequant4`: multiply by the Q8.8 step, round, add zero point, saturate at 255.
`uamm_block_table.v`	256-entry table of `{tier, codec, phys}` with valid bits and a one-cycle lookup returning `hit`.
`tb_*.v`	Read the generated vectors, drive the module, compare every output and print PASS or FAIL; with `+trace` they write a VCD.
`Makefile`	`make test` regenerates vectors and runs the three benches in Verilator; `make lint` runs `-Wall` lint.

6.5 Vector kernel

`kv_qk_prepare` folds the per-channel scale into the query (split into even and odd channels) and returns the bias $\Sigma\ q \cdot \text{zero}$. `kv_qk_dot_kivi4_scalar` loops over tokens and packed bytes with two multiply–adds per byte. `kv_qk_dot_kivi4_rvv` does the same with vector instructions (byte load, nibble extraction, widen, convert, fused multiply–add with tail-undisturbed policy, reduction). `kv_qk_dot_kivi4` selects the vector path when compiled for a vector target.

6.6 Tests

Table 13. *Test files.*

File	Tests	What is checked
`test_codecs.py`	10	Pack/unpack round trip and nibble order; FP16 exactness; error and size ordering across codecs; entropy variants lossless; per-channel keys beat per-token on outlier channels; constant input; bad inputs
`test_cache_h2o_attention.py`	8	Block sealing and partial tail; ratchet and lossless entropy switch; the selection bound is a true upper bound; sparse equals dense when everything is read; heavy hitters rank higher; decay forgets
`test_system.py`	12	Combined system fits without data loss; threshold policy must drop; combined system beats it on quality here; capacities respected; protected blocks stay hot and lossless; precision never increases; hysteresis lowers churn; drop only when enabled; adapter round trip; skeleton honesty
`test_ara_hw.py`	6	C kernel against NumPy; rejects other codecs; vector path compiles; golden scorer properties; codec error bounded; full RTL regression in Verilator

6.7 How to run everything

```
pip install numpy pytest verilator        # g++ with C++20 needed for the RTL tests
python -m uamm2 simulate --tokens 1024     # one configuration
python -m uamm2 ablate --tokens 1536       # UAMM-1 vs UAMM-2 variants
python -m pytest -q tests                  # 56 tests, ~40 s
make -C hw test ; make -C hw lint          # RTL regression and lint
python experiments/run_experiments.py      # all studies -> experiments/results/*.json
python experiments/mutation_test.py        # RTL mutation testing
bash docs/gtkwave/view_waves.sh 3          # open a waveform with signals preloaded (1-6)
```

7. Experimental Methodology

This chapter states precisely what was simulated, what was real, how quality was measured and which constants were assumed. The goal is that a reader can judge exactly what each number in Chapters 8 to 10 means.

7.1 What is real and what is modelled

Table 14. *Status of each element of the evaluation.*

Component	Status in the experiments
KV tensors, codecs, packing, entropy coding	**Real computation.** Blocks are actually quantised, bit-packed, optionally zlib-compressed, decoded and used for attention.
Attention output and quality metrics	**Real computation** on decoded tensors, compared with a dense FP32 attention oracle over all tokens at the same step.
Block selection (min/max bound), importance scores, planner decisions	**Real computation** (the same code that would run on a device).
Q, K, V stream	**Synthetic**: produced by a parametrised trace generator, described in 6.2, not by a neural network.
Tier latencies, bandwidths, decode costs	**Modelled constants** (6.4). The reported latency is a model output, not a measurement.
RTL behaviour	**Real simulation** with Verilator, compared bit-exactly with Python golden models.
RVV kernel	Scalar variant executed on the host; RVV variant compiled to RISC-V vector instructions but not executed.

7.2 The synthetic trace generator

An LLM was not run for the experiments in this report. Instead a trace generator produces, for each decode step, a query vector and a key and value vector with the structural properties that KV-cache managers are designed around. The generator has 64 channels (`head_dim`) and exposes these features:

- **Attention sinks.** The first four tokens, and 3 % of later tokens at random, receive an extra component along a fixed “heavy” direction; the query also has a component along it. These tokens attract disproportionate attention, as heavy hitters and sinks do in real models.
- **Outlier key channels.** One in sixteen channels of the keys is scaled by 6× (4 of 64 channels). This is the structure that motivates per-channel key quantisation.
- **Topic phases.** Four unit topic directions are defined. Keys in each phase of 96 tokens carry that phase's topic. The query's topic follows the phase but periodically (every third phase) returns to an older topic, so that previously cold blocks become hot again and must be promoted.
- **Local recency.** Part of the query is the direction of one of the 12 most recent keys, producing local attention.

- **Sharpness.** The query is scaled by a factor of 4, which controls how peaked the softmax is; value vectors are independent Gaussian noise.

Because the generator is designed rather than learned, its absolute numbers must not be read as what a particular LLM would give. In particular, values that are independent noise average to a small-norm vector, which inflates the *relative* L2 error of the output; the comparison of variants against each other on the same stream is the meaningful quantity. Chapter 8 includes a measurement of how concentrated the attention on this generator is (Section 8.8), so that the reader can compare it with concentration reported for real models.

7.3 Metrics

Table 15. *Metrics reported in the experiments.*

Metric	Definition	Why
`resident_vs_fp16`	Total bytes held across tiers divided by the FP16 size of all tokens	Memory saving (lower is better)
`dropped_tokens`	Tokens permanently removed	Irreversible information loss
`attn_rel_l2`	‖o_sys − o_dense‖ / ‖o_dense‖ for the attention output at each step after a warm-up of 64 steps (mean and 95th percentile)	Output fidelity vs a dense FP32 oracle over the same (FP16-rounded) data
`attn_mass_recall`	Fraction of the oracle's attention probability that falls on the tokens the system read	How much of what attention wants was available
`sim_fetch_ms_per_token`	Modelled time to fetch and decode all blocks read in a step (mean)	Latency of reading the KV cache under the assumed constants
`flash_block_reads`	Count of block reads served from flash	Proxy for slow-tier traffic
`migrations`, `reencodes`	Cumulative counts of tier moves and codec changes	Churn (energy and bandwidth cost of management)
`overflow_total`	Sum over tiers of the maximum bytes by which a tier budget was exceeded	Feasibility: non-zero means the data did not fit

Interpretation guard. The attention-mass recall depends only on which blocks are read and not on their precision, so it is identical for all UAMM-2 variants that use the same selection logic; the codec choice shows up in the relative L2 error. A non-zero error floor remains even with FP16 everywhere, because block-sparse attention omits some tokens; the FP16-only variant (error 0.092 at 1,536 tokens) measures that floor.

Accounting rule. `resident_vs_fp16` and the tier budgets count the bytes of each block's encoded payload, including its scales and zero points. They do **not** count the small per-block summaries that the engine keeps resident for selection (key minimum and maximum, 64 values each) or the scheduling state. Stored in FP16, the key summaries would add about 256 bytes per block, i.e. about 25 KB for the 96 blocks of a 1,536-token run, which is roughly 6 % of the FP16 cache size and 15 % of the

168 KB budget. The results below should be read with that omission in mind; Chapter 12 lists it among the open items.

7.4 Assumed cost constants

The following constants are placeholders for an edge SoC and are not measurements. They define the latency model used by the planner and the simulator. Conclusions about *ratios* between variants depend on them; they should be replaced by values measured on the target before any absolute claim is made.

Table 16. *Default configuration of the experiments.*

Constant	Value	Meaning
T0 latency / bandwidth	0.5 µs / 100 GB/s	Near-compute SRAM or vector register file
T1, T2 latency / bandwidth	2 µs / 12.8 GB/s	LPDDR-class DRAM
T3 latency / bandwidth	100 µs / 1 GB/s	Flash
Decode cost, fp16 / int8 / kivi4 / kivi2	0 / 0.05 / 0.12 / 0.20 ns per byte	Software dequantisation cost model
zlib decode cost	1.0 ns per byte	Additional cost for the `z` variants
Tier capacities T0..T3	24, 48, 32, 64 KB (total 168 KB)	FP16 KV for 1,536 tokens is 393 KB
Block size, v_group, sink, recent window	16 tokens, 32 channels, 4 tokens, 32 tokens	Geometry and protection
Attention block budget	24 blocks per step	Non-protected blocks fetched
Planner: plan interval, hysteresis, residency, λ, κ, headroom	8 steps, 0.25, 16 steps, 2.0, 0.05, 3 %	See Chapter 5
H2O decay, aggregation, hit_ema	0.95, mean, 0.9	Importance signal

λ was chosen on seeds 7–9 during development; all reported experiments use other seeds (11–15), so the default configuration is evaluated on data it was not tuned on.

7.5 Experimental design

Table 17. *Experiments performed.*

ID	Question	Variable	Seeds	Tokens
E1	Does each ingredient matter? (ablation)	Policy and codec set: UAMM-1 threshold; planner with FP16 only; INT8+KIVI-4; all codecs	5	1,536
E2	How does the system scale with context?	Context 512–3,072 at a fixed budget	3	512–3,072
E3	How many blocks must be read?	Attention block budget 4–48	3	1,536
E4	How sensitive are the planner and importance knobs?	λ, hysteresis margin, residency, plan interval, decay, aggregation, block size	3	1,536
E5	How does memory budget change the picture?	Tier budgets ×0.5 to ×3.0; and codec sets with and without entropy coding	3	1,536

ID	Question	Variable	Seeds	Tokens
E6	What happens over time?	Per-step traces of tiers, codecs, churn, error	1	1,536
E7	How good are the codecs, and does the key axis matter?	Codec and outlier presence; per-channel vs per-token keys	200 random blocks per cell	—
E8	How concentrated is attention on the generator?	Dense attention statistics at sampled steps	1	1,536

Values are reported as mean ± standard deviation across seeds. With three seeds the standard deviation is itself a rough estimate; differences smaller than about one standard deviation should not be treated as significant. The experiment code is in `experiments/run_experiments.py`, listed in the appendices, and all raw numbers are tabulated in Appendix B.

8. Simulation Results

All numbers in this chapter come from `experiments/run_experiments.py` (raw data in `experiments/results/*.json`) under the configuration of Chapter 7. Latency figures are model outputs under the assumed constants. Errors are measured against a dense FP32 oracle on the synthetic trace. Where we interpret a result, we say so.

8.1 Ablation: what each ingredient contributes (E1)

Table 18. *Ablation at 1,536 tokens (mean ± std over 5 seeds).*

Variant	Resident / FP16	Dropped tokens	Rel. L2 error	Mass recall	Fetch ms / token (model)	Flash reads	Overflow (B)
UAMM-1 threshold policy, FP16	0.367 ± 0.0102	973 ± 16	0.479 ± 0.0346	0.758 ± 0.0164	0.923 ± 0.0259	12613 ± 374	0 ± 0
Planner, FP16 only	1 ± 0	0 ± 0	0.0921 ± 0.0191	0.939 ± 0.0117	1.04 ± 0.006	14364 ± 86	258048 ± 1832
Planner, INT8 + KIVI-4	0.371 ± 0.00265	0 ± 0	0.122 ± 0.0137	0.939 ± 0.0117	0.116 ± 0.00216	970 ± 32	0 ± 0
Planner, INT8 + KIVI-4 + KIVI-2	0.31 ± 0.00133	0 ± 0	0.166 ± 0.0203	0.939 ± 0.0117	0.112 ± 0.00203	915 ± 30	0 ± 0

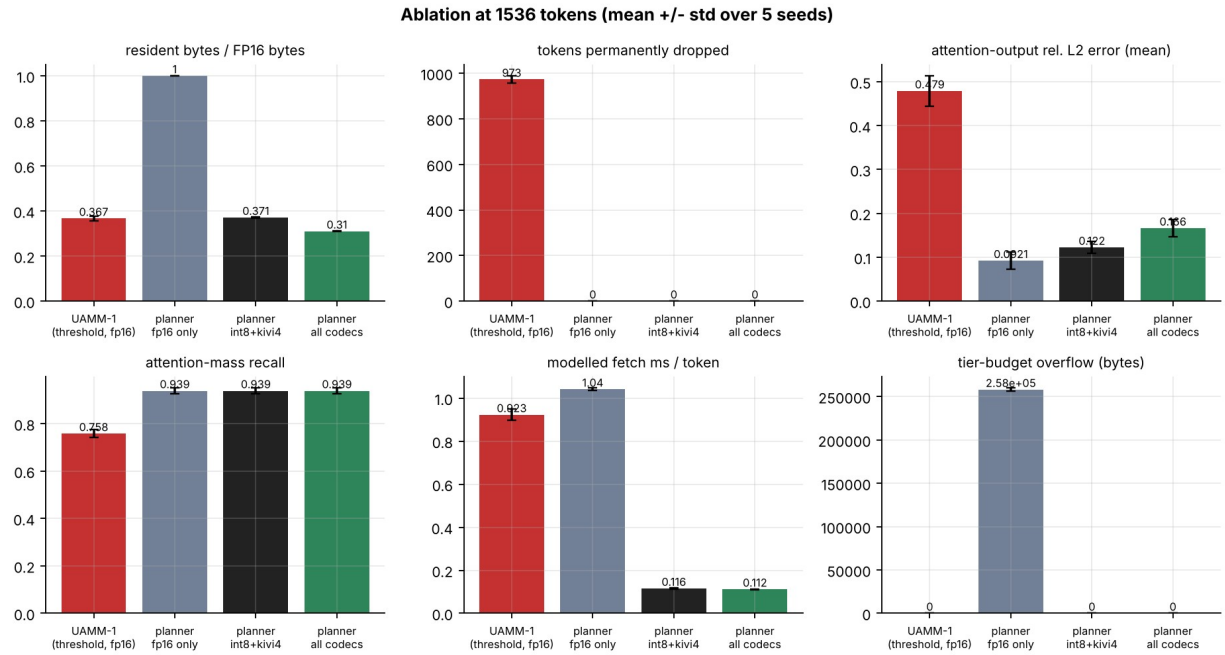


Figure 15. *Ablation results (mean ± std, 5 seeds). The FP16-only planner cannot fit the data in the budget (non-zero overflow); the UAMM-1 threshold policy fits only by dropping most tokens.*

Reading the table.

- **UAMM-1 fits by deleting.** With only FP16 blocks and a 168 KB budget the threshold policy must remove on average 973 of 1,536 tokens (63 %). Attention-

mass recall falls to 0.758 and the output error reaches 0.479; the reads that remain are frequently from flash (about 12,600 flash block reads).

- **The planner alone cannot create space.** With FP16 only, the planner drops nothing but cannot fit: 258 KB of overflow, i.e. the data exceeds the stated budget by that much. The lower error (0.092) is the floor due to block-sparse selection with exact data. This variant shows that placement alone is insufficient: without a codec, a budget below the data size is infeasible without deletion.
- **Codecs make the budget feasible.** Adding INT8 and 4-bit codecs reduces residency to 0.371 of FP16, with no overflow and no drops. Error rises to 0.122 (vs the 0.092 floor): the price of quantisation.
- **2-bit codec is a knob.** Allowing 2-bit blocks reduces residency further to 0.310 (a further 16 % saving) at an error of 0.166, i.e. 36 % more error than INT8+KIVI-4.
- **Latency model.** Modelled fetch time per token falls from 0.92 ms (UAMM-1) to 0.116 ms (INT8+KIVI-4) and 0.112 ms (all codecs). Most of that difference is slower-tier traffic: flash block reads drop from about 12,600 to about 915–970, i.e. 13×. The FP16-only planner is slower than UAMM-1 (1.04 ms) because overflowing blocks must live in flash with no compression.

What the ablation says

UAMM-1's weakness is not placement, it is the lack of a way to make data smaller. A joint (tier, codec) choice turns a budget that was infeasible without data deletion into a feasible one at a cost of bounded, measured quantisation error. Every ingredient has a role: removing the codecs reintroduces overflow or deletion; removing the planner loses the control over where precision is spent.

8.2 Scaling with context length (E2)

Table 19. *Context scaling with the 168 KB budget fixed (mean of 3 seeds). UAMM-2 columns use all codecs unless stated.*

Tokens	Err UAMM-1	Err UAMM-2 (all)	Recall UAMM-1	Recall UAMM-2	Dropped (UAMM-1)	Resident/ FP16 (UAMM-2)	Fetch ms UAMM-1	Fetch ms UAMM-2	Overflow B: INT8+K4 / all
512	0.001	0.016	0.999	0.999	0	0.538	0.473	0.035	0 / 0
768	0.100	0.042	0.954	0.990	245	0.445	0.728	0.049	0 / 0
1024	0.241	0.071	0.881	0.976	496	0.376	0.846	0.072	0 / 0
1536	0.505	0.159	0.750	0.945	971	0.310	0.919	0.111	0 / 0
2048	0.683	0.270	0.657	0.913	1461	0.266	0.948	0.152	25,899 / 0
3072	0.985	0.515	0.532	0.830	2469	0.241	0.946	0.178	132,309 / 28,501

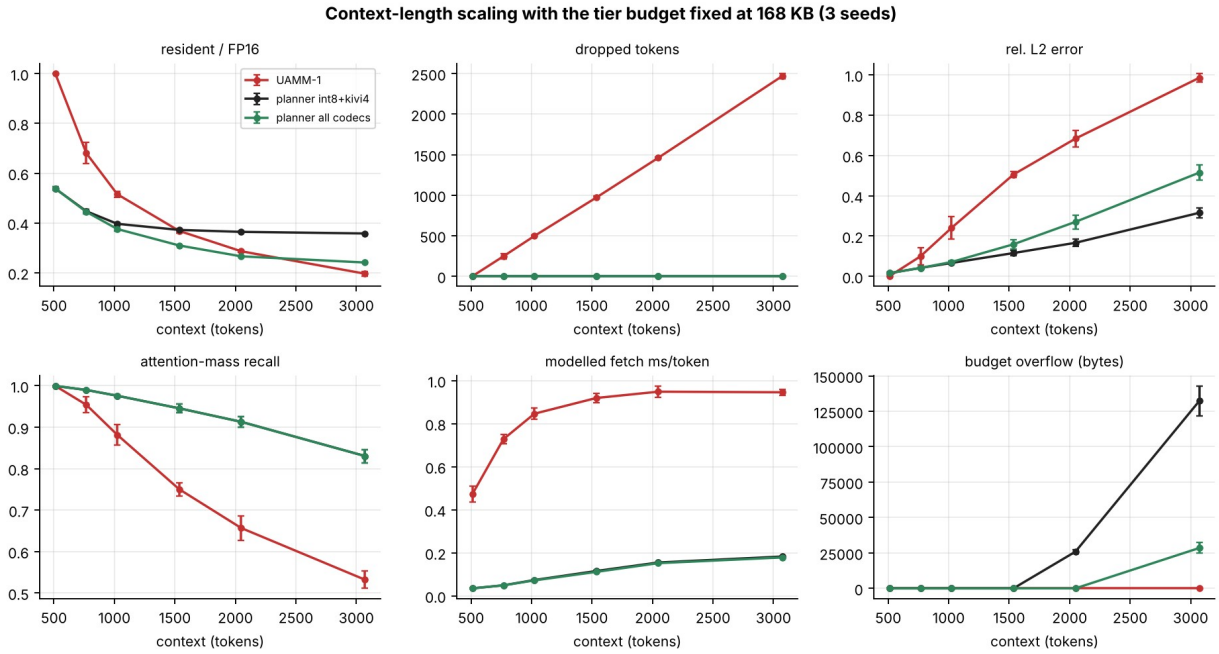


Figure 16. *Context scaling at a fixed budget. UAMM-1's error and drops grow quickly beyond 768 tokens; UAMM-2 degrades gradually and keeps every token until the budget is exceeded near 3,072 tokens.*

- **Short contexts.** At 512 tokens the cache fits at FP16 (UAMM-1 resident ratio 1.0, error 0.0013). UAMM-2 still compresses (resident 0.538) and is 14× faster in the model (0.035 ms vs 0.473 ms) but incurs a small error (0.016). That is a policy consequence of the cost function: when latency is priced and the quality term is small, the planner lowers precision even without capacity pressure. If the application prefers exactness when memory is available, λ should be raised or compression gated on pressure (Chapter 12).
- **Medium contexts.** At 1,024 tokens UAMM-1 has dropped 496 tokens (error 0.241, recall 0.882); UAMM-2 holds all tokens with error 0.071 and recall 0.976.
- **Long contexts.** At 2,048 tokens UAMM-2 (all codecs) still fits (resident 0.266, error 0.270, recall 0.913). At 3,072 tokens it overflows its budget by 28.5 KB (INT8+KIVI-4 only: 132 KB) and error is 0.515 vs 0.985 for UAMM-1, which has dropped 2,469 of 3,072 tokens. Beyond this point the fixed budget is simply too small for the data even at 2-bit; opt-in drop or a larger budget would be needed.
- **Recall.** Recall falls with context in both systems but for different reasons: in UAMM-1 because tokens are missing, in UAMM-2 because the fixed block budget of 24 covers a shrinking fraction of the history (see E3).

8.3 How many blocks must be read? (E3)

Table 20. *Effect of the attention block budget at 1,536 tokens (mean ± std, 3 seeds).*

Block budget B	Mass recall	Rel. L2 error	Fetch ms / token	Resident / FP16
4	0.568 ± 0.013	1.101 ± 0.016	0.036 ± 0.003	0.269 ± 0.001

Block budget B	Mass recall	Rel. L2 error	Fetch ms / token	Resident / FP16
8	0.744 ± 0.017	0.663 ± 0.015	0.062 ± 0.002	0.272 ± 0.003
12	0.841 ± 0.019	0.443 ± 0.027	0.087 ± 0.001	0.277 ± 0.001
16	0.895 ± 0.017	0.308 ± 0.022	0.098 ± 0.003	0.285 ± 0.002
24	0.945 ± 0.011	0.159 ± 0.023	0.111 ± 0.002	0.310 ± 0.001
32	0.966 ± 0.007	0.101 ± 0.008	0.132 ± 0.004	0.337 ± 0.004
48	0.986 ± 0.003	0.070 ± 0.002	0.278 ± 0.024	0.376 ± 0.002

Selecting 24 blocks (about a quarter of the roughly 96 blocks at 1,536 tokens) retains 94.5 % of the attention mass with an error of 0.159; 48 blocks reach 98.6 % at a cost that is 2.5× higher (0.278 ms vs 0.111 ms) because more blocks are fetched from slower tiers. The marginal return of a block falls quickly, as the sorted block mass (Section 8.8) predicts. Note the interaction with residency: a larger budget reads many cold blocks, which pushes the planner to keep more of them in higher precision (resident 0.376 at B=48 vs 0.310 at B=24).

8.4 Sensitivity of the planner and the importance signal (E4)

8.4.1 The quality weight λ

Table 21. *Effect of the quality weight* λ *at 1,536 tokens (3 seeds).*

λ	Resident / FP16	Rel. L2 error	Fetch ms / token	Flash reads	Migrations
0.0	0.243 ± 0.000	0.577 ± 0.020	0.098 ± 0.003	724 ± 38	573 ± 7
0.25	0.264 ± 0.002	0.289 ± 0.016	0.101 ± 0.001	763 ± 16	610 ± 18
0.5	0.284 ± 0.002	0.200 ± 0.028	0.106 ± 0.003	836 ± 44	631 ± 11
1.0	0.295 ± 0.001	0.181 ± 0.031	0.108 ± 0.003	860 ± 48	639 ± 17
2.0	0.310 ± 0.001	0.159 ± 0.023	0.111 ± 0.002	908 ± 37	630 ± 15
4.0	0.315 ± 0.003	0.152 ± 0.025	0.197 ± 0.031	2189 ± 455	567 ± 6
8.0	0.325 ± 0.003	0.138 ± 0.012	0.245 ± 0.003	2912 ± 45	567 ± 24
16.0	0.343 ± 0.004	0.134 ± 0.013	0.237 ± 0.004	2792 ± 60	614 ± 20
32.0	0.366 ± 0.006	0.127 ± 0.009	0.253 ± 0.007	3036 ± 102	726 ± 53

As λ rises the planner protects quality: error falls from 0.577 at $\lambda = 0$ (maximum compression, resident 0.243) to 0.127 at $\lambda = 32$, at the price of larger residency (0.366). Latency is not monotone: up to $\lambda = 2$ fetch time is nearly flat at about 0.10–0.11 ms, then jumps to 0.20–0.25 ms at $\lambda \geq 4$ and flash reads triple. The cause, visible in the planner's cost, is that choosing larger encodings for a block can push other blocks out of DRAM into flash, whose fixed 100 µs access cost dwarfs the transfer savings. The default $\lambda = 2$ sits at the knee: the error is within 25 % of the best seen (0.159 vs 0.127) while fetch time is less than half that of $\lambda \geq 4$.

8.4.2 Hysteresis, residency and replanning interval

Table 22. *Hysteresis margin (3 seeds).*

Hysteresis margin	Migrations	Re-encodes	Resident / FP16	Rel. L2 error	Fetch ms / token
0.0	1871 ± 45	247 ± 3	0.286 ± 0.004	0.208 ± 0.030	0.112 ± 0.002
0.1	784 ± 27	240 ± 3	0.292 ± 0.004	0.181 ± 0.027	0.111 ± 0.004
0.25	630 ± 15	226 ± 2	0.310 ± 0.001	0.159 ± 0.023	0.111 ± 0.002
0.5	593 ± 14	188 ± 4	0.359 ± 0.006	0.111 ± 0.013	0.116 ± 0.003
1.0	463 ± 26	180 ± 3	0.373 ± 0.005	0.110 ± 0.013	0.115 ± 0.003

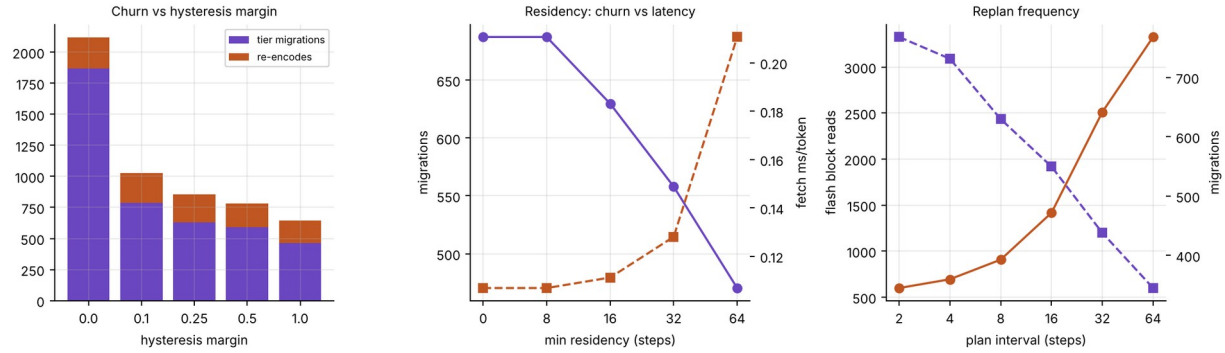


Figure 17. *Left: churn versus hysteresis margin. Middle: minimum residency. Right: planning interval.*

Without hysteresis (margin 0) the planner performs 1,871 migrations; with the default margin of 0.25 it performs 630, a reduction of 66 % at essentially unchanged fetch time (0.1118 vs 0.1112 ms). The trade-off is that a larger margin also suppresses precision demotions (re-encodes fall from 247 to 180), so residency grows (0.286 to 0.373) and error falls (0.208 to 0.110). Hysteresis is therefore both a churn limiter and a quality-conservative bias.

The minimum-residency rule has no effect below 16 steps because the planning interval is 8 (identical rows for 0 and 8): residency is only meaningful when it exceeds the interval. At 64 steps migrations fall to 470, but fetch time doubles (0.211 ms) and flash reads reach 2,384, because blocks that have become hot cannot be promoted in time. Likewise the replanning interval: planning every 2 steps gives 599 flash reads and 0.090 ms but 768 migrations, whereas every 64 steps gives 3,326 flash reads and 0.272 ms with only 345 migrations. The planner must keep up with the changes in attention focus; the cost of keeping up is the planning work itself, which is why the hardware sweep scorer exists.

8.4.3 H2O decay, aggregation and block size

Table 23. *H2O decay (3 seeds).*

H2O decay	Resident / FP16	Rel. L2 error	Fetch ms / token	Re-encodes
0.5	0.285 ± 0.003	0.202 ± 0.018	0.112 ± 0.002	244 ± 3
0.8	0.285 ± 0.003	0.202 ± 0.018	0.111 ± 0.002	244 ± 2

H2O decay	Resident / FP16	Rel. L2 error	Fetch ms / token	Re-encodes
0.9	0.286 ± 0.003	0.202 ± 0.018	0.112 ± 0.002	244 ± 2
0.95	0.310 ± 0.001	0.159 ± 0.023	0.111 ± 0.002	226 ± 2
0.98	0.329 ± 0.005	0.135 ± 0.010	0.108 ± 0.002	210 ± 1
0.995	0.368 ± 0.004	0.114 ± 0.009	0.110 ± 0.002	173 ± 2

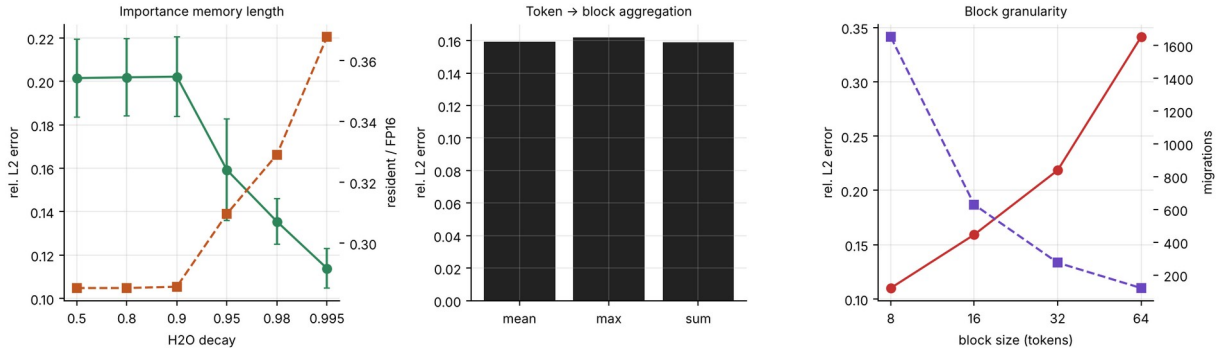


Figure 18. *Left: H2O decay. Middle: token-to-block aggregation. Right: block size (with the block budget scaled so the number of tokens fetched stays constant).*

A longer memory of attention (decay 0.995) makes the planner more conservative about compressing blocks that were important at some point: error falls from 0.202 (decay ≤ 0.9) to 0.114 and residency rises from 0.285 to 0.368, at essentially constant fetch time (0.110–0.112 ms). Decay therefore moves the system along the same memory-versus-quality axis as λ but at no latency cost in this configuration. Mean and sum aggregation are equivalent (sum is normalised by the same percentile; blocks have equal size); max aggregation gives slightly lower residency (0.301) and similar error (0.162).

Table 24. *Block size (3 seeds; block budget scaled to keep ≈384 non-protected tokens fetched).*

Block size (tokens)	Resident / FP16	Rel. L2 error	Fetch ms / token	Migrations	Overflow (B)
8	0.387 ± 0.008	0.110 ± 0.014	0.216 ± 0.009	1652 ± 86	0 ± 0
16	0.310 ± 0.001	0.159 ± 0.023	0.111 ± 0.002	630 ± 15	0 ± 0
32	0.264 ± 0.003	0.219 ± 0.036	0.063 ± 0.002	275 ± 19	0 ± 0
64	0.264 ± 0.013	0.341 ± 0.010	0.038 ± 0.002	121 ± 6	14336 ± 0

Block size is a three-way trade-off. Small blocks (8 tokens) isolate hot tokens well, giving the lowest error (0.110), but have more metadata, more migrations (1,652) and much higher fetch time (0.216 ms, since each block pays a fixed latency). Large blocks (64) cut migrations to 121 and fetch time to 0.038 ms but raise error to 0.341 and overflow the budget by 14 KB: a block must be placed and compressed as a unit, so a few hot tokens drag a whole large block into a fast tier. A block of 16 tokens is a reasonable compromise and also fits the hardware tables.

8.5 Memory budget (E5)

Table 25. *Memory-budget sweep at 1,536 tokens (mean of 3 seeds).*

Budget	UAMM-1 resident	UAMM-1 dropped	UAMM-1 error	UAMM-1 fetch ms	UAMM-2 resident	UAMM-2 error	UAMM-2 fetch ms	UAMM-2 overflow B
×0.5 (84 KB)	0.219	1200	0.855	0.757	0.288	0.220	0.301	33,109
×0.75 (126 KB)	0.299	1077	0.661	1.001	0.293	0.180	0.141	0
×1.0 (168 KB)	0.368	971	0.505	0.919	0.310	0.159	0.111	0
×1.5 (252 KB)	0.521	736	0.280	0.733	0.323	0.131	0.099	0
×2.0 (336 KB)	0.625	576	0.171	0.612	0.330	0.115	0.088	0
×3.0 (504 KB)	0.830	261	0.086	0.502	0.324	0.152	0.049	0

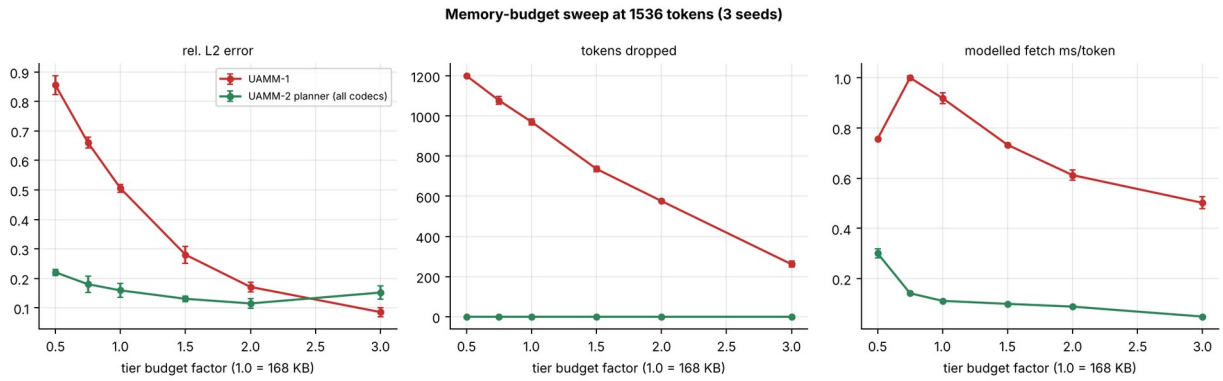


Figure 19. *Budget sweep. UAMM-1 needs roughly three times the baseline budget to bring error below 0.09 and still drops 261 tokens; UAMM-2 reaches comparable error without drops at much smaller budgets.*

- **Equal-quality comparison.** UAMM-2 at ×0.75 of the budget (126 KB) has error 0.180 and drops nothing; UAMM-1 needs ×2.0 (336 KB) to reach a similar error (0.171) and still drops 576 tokens. In the model, UAMM-2 matches UAMM-1's error with 2.7× less memory.
- **Infeasibility.** At ×0.5 (84 KB) UAMM-2 overflows by 33 KB (the data do not fit even at its most compressed); UAMM-1 does not report overflow but drops 1,200 tokens and has error 0.855.
- **Non-monotonic quality.** UAMM-2's error is 0.115 at ×2.0 but 0.152 at ×3.0, even though more memory is available. At ×3.0 flash is almost unused (3 reads) and the planner spends the headroom on speed (fetch 0.049 ms vs 0.088 ms) by choosing smaller encodings. This is the cost function working as designed, but it is a reminder that the optimiser, not the budget, decides the trade-off, and that a quality floor (Chapter 12) is needed to guarantee monotonic behaviour.

8.5.1 Which codecs should be enabled? (E5b)

Table 26. *Codec-set study at 1,536 tokens (3 seeds).*

Enabled codecs	Resident / FP16	Rel. L2 error	Fetch ms / token	Overflow (B)
FP16 only	1.000 ± 0.000	0.081 ± 0.018	1.040 ± 0.007	256,683 ± 965
FP16 + INT8	0.606 ± 0.000	0.083 ± 0.018	0.460 ± 0.014	104,704 ± 0
+ KIVI-4	0.372 ± 0.002	0.116 ± 0.013	0.115 ± 0.003	0 ± 0
+ KIVI-2 (all)	0.310 ± 0.001	0.159 ± 0.023	0.111 ± 0.002	0 ± 0
FP16 + KIVI-4 only	0.364 ± 0.000	0.132 ± 0.013	0.112 ± 0.002	0 ± 0
FP16 + KIVI-2 only	0.346 ± 0.015	0.308 ± 0.018	0.475 ± 0.012	0 ± 0

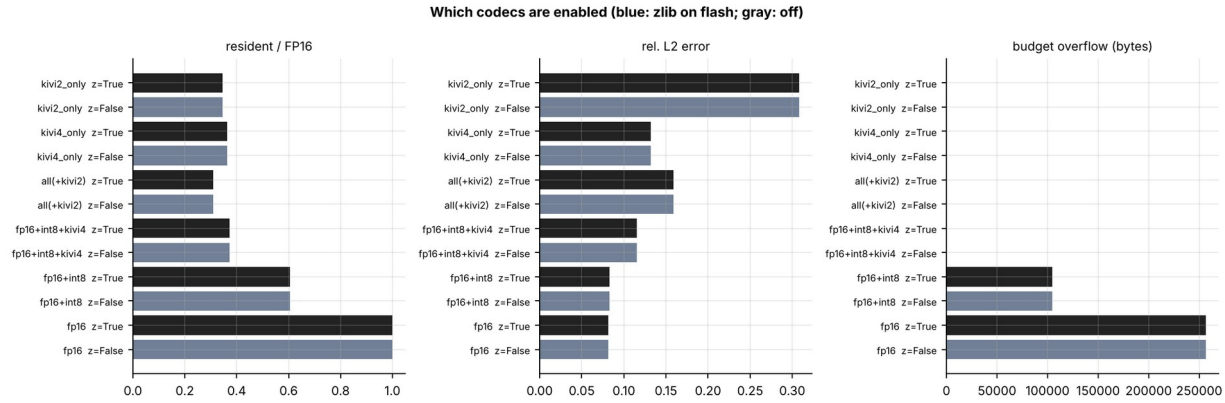


Figure 20. *Codec-set study. Blue bars enable entropy-coded flash variants, gray bars disable them; the two are indistinguishable on this synthetic data.*

- **Feasibility needs 4-bit.** FP16 alone (overflow 257 KB) and FP16+INT8 (overflow 105 KB) cannot fit the budget; a 4-bit codec is the first that does.
- **KIVI-4 alone is competitive.** FP16 + KIVI-4 reaches 0.364 resident at error 0.132 and fetch 0.112 ms, close to the full set on speed with lower error than the full set (0.159); the full set saves a further 15 % memory.
- **KIVI-2 alone is a poor choice.** With only FP16 and KIVI-2 available the planner reaches 0.346 resident, but error doubles (0.308) and fetch time is 4× worse (0.475 ms, about 6,200 flash reads). The reason is structural: the tier rules force warm and cold storage to hold a 2-bit block, and without intermediate precision the planner cannot place blocks that matter more at reasonable cost.
- **Entropy coding gave no benefit here.** Results with and without zlib variants are identical, which means the planner never preferred a `z` variant. On the synthetic data, zlib on already quantised random-like values saves almost nothing (Table of codec sizes in Chapter 5). Real KV tensors have more redundancy and the result may differ; this is an open question rather than a conclusion.

8.6 Behaviour over time (E6)

The following figures are time series from one run (seed 11, 1,536 tokens) of UAMM-1 and UAMM-2.

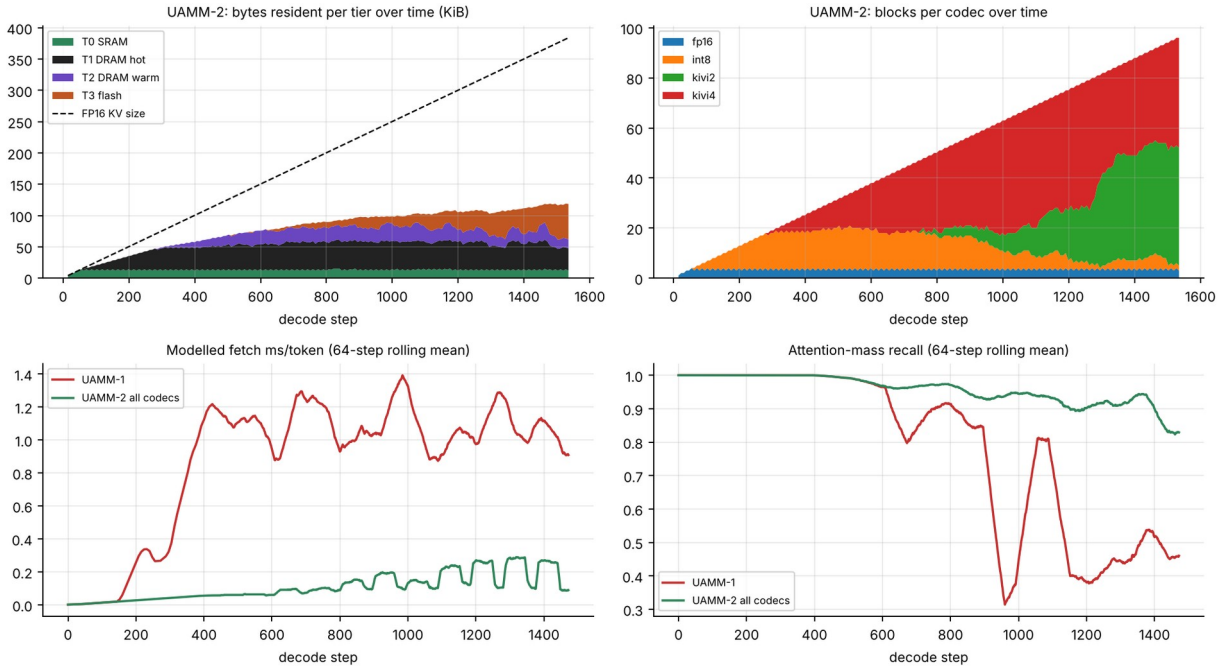


Figure 21. *Top left: bytes resident per tier. Top right: number of blocks by codec. Bottom: rolling modelled fetch time and attention-mass recall for both systems.*

The tiers fill in order: first T0 (protected blocks), then T1, then T2 and T3 as the cache outgrows the faster tiers. The codec histogram shows the ratchet in action: INT8 blocks dominate early, 4-bit blocks take over as pressure rises, and 2-bit blocks appear mainly in the last third of the run (after about 1,100 steps) when the lower tiers are full. FP16 blocks are only the protected set (about 3 blocks). UAMM-1's rolling fetch time rises to about 1.2 ms after about 400 steps and recall oscillates between 0.3 and 0.9, falling as blocks are discarded; UAMM-2's fetch time stays below 0.3 ms and its recall stays above 0.8 throughout.

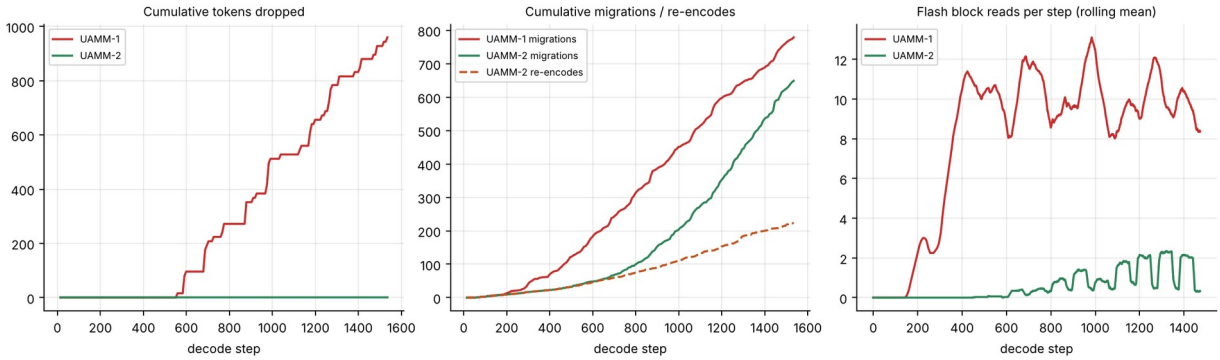


Figure 22. *Cumulative dropped tokens, cumulative migrations and re-encodes, and flash reads per step.*

UAMM-1's dropped-token count rises steadily once the budget is reached and never recovers. UAMM-2's counter stays at zero. Migrations accumulate for both, with UAMM-2 also performing re-encodes (each is a decode and an encode and has a modelled cost). Flash reads for UAMM-1 are persistent and bursty; those for UAMM-2 are rare and concentrated where the query returns to an old topic.

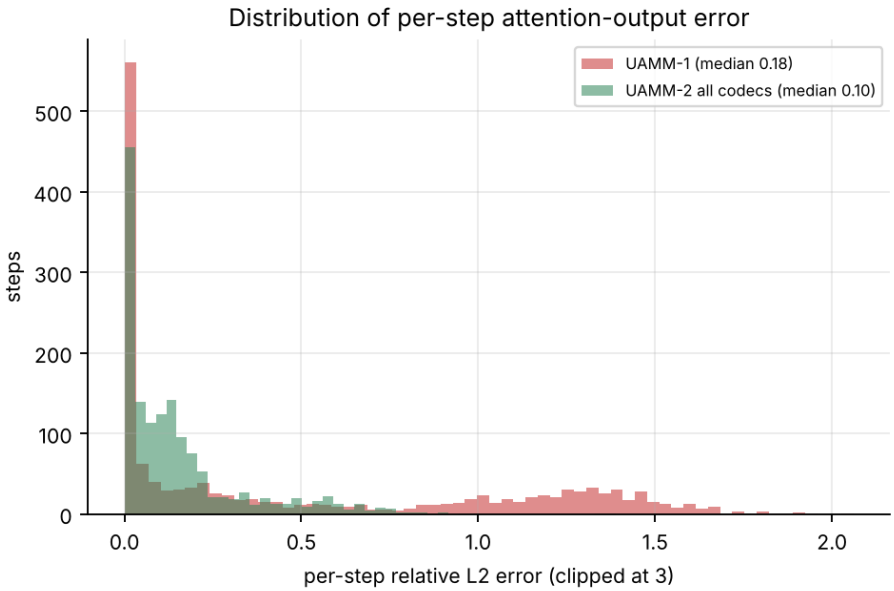


Figure 23. *Distribution of per-step attention-output error. UAMM-1 has a heavy tail of large-error steps (those that needed tokens it had already deleted); UAMM-2 is concentrated at lower values.*

Per-step error medians are 0.185 (UAMM-1) and 0.097 (UAMM-2); 95th percentiles are 1.489 and 0.575. The heavy tail matters for users: a few very wrong steps are noticed more than a uniformly small error.

8.7 Codec micro-benchmarks (E7)

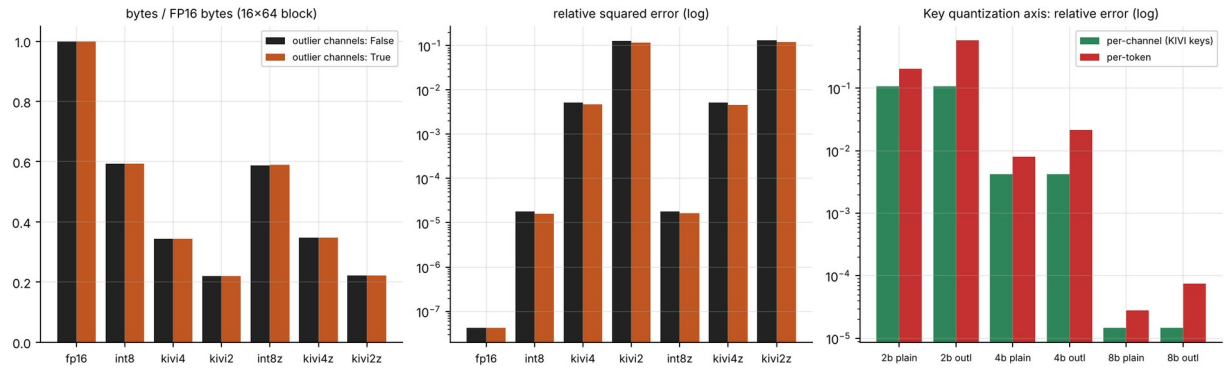


Figure 24. *Left: measured size ratio per codec. Middle: reconstruction error (log scale). Right: per-channel versus per-token key quantisation error.*

Table 27. *Key quantisation axis: relative squared error of keys only (200 random 16 × 64 blocks per row).*

Bits	Outlier channels	Per-channel keys	Per-token keys	Per-token / per-channel
2-bit	without	1.07e-01	2.02e-01	1.9×
2-bit	with	1.07e-01	5.81e-01	5.4×
4-bit	without	4.22e-03	8.04e-03	1.9×
4-bit	with	4.18e-03	2.17e-02	5.2×
8-bit	without	1.47e-05	2.79e-05	1.9×
8-bit	with	1.47e-05	7.52e-05	5.1×

Quantising keys per channel is always better, by a factor of 1.9 without outlier channels and 5.2–5.4× when four of the 64 channels are six times larger, which agrees with the design choice reported for KIVI. Error rises by roughly 25× from 4-bit to 2-bit for the same block (0.0052 → 0.130 in the codec table), which is why 2-bit is a controlled, optional tier rather than a default.

8.8 Attention structure of the workload (E8)

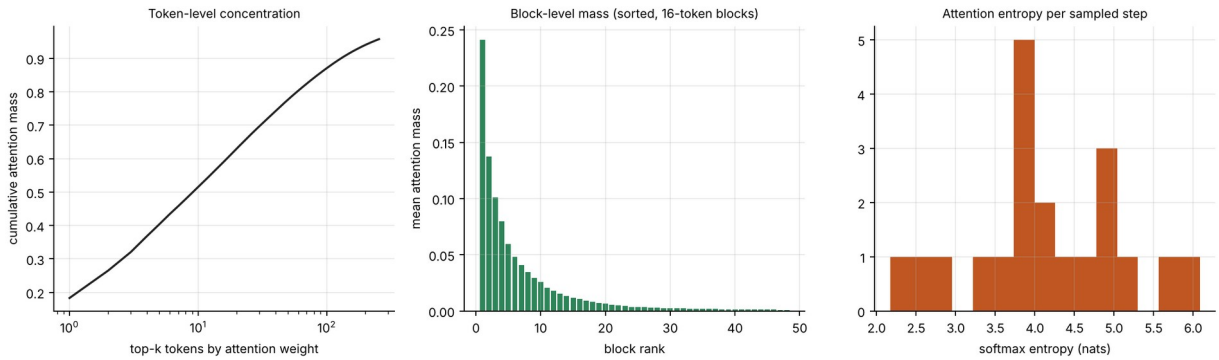


Figure 25. *Left: cumulative attention mass versus number of tokens. Middle: mean attention mass of the blocks sorted by rank. Right: softmax entropy per sampled step.*

On the generator, the top 1 token receives 18 % of the attention, the top 10 receive 51 %, the top 50 receive 78 % and the top 100 receive 87 %; the best block carries 24 % of the mass and the best ten blocks 80 %. Attention is concentrated but not extremely so (mean entropy 4.12 nats, against a maximum of ln(256…1536) ≈ 5.5–7.3). This is a deliberately moderate setting: the heavier the concentration of real attention, the easier the problem is for any selective scheme, so this workload is not an optimistic one for block selection.

8.9 Final placement map

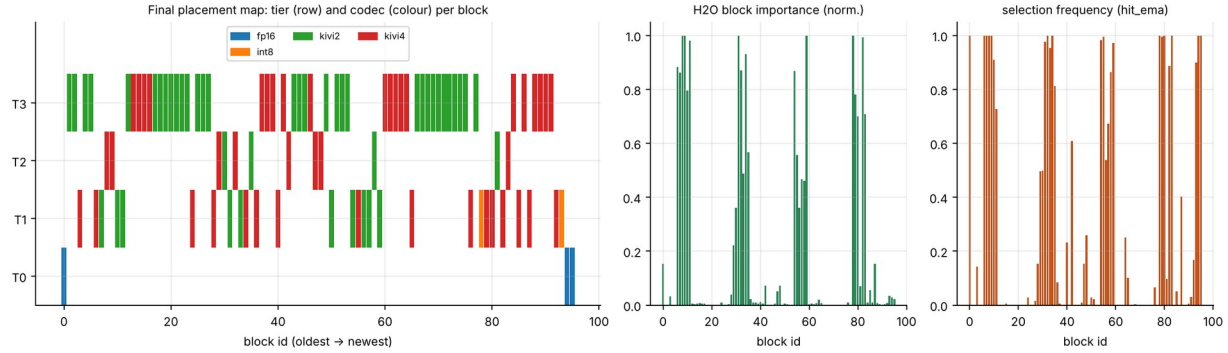


Figure 26. *State of the cache at step 1,536: tier (row) and codec (colour) of every block, with block importance and selection frequency. Older, low-importance blocks occupy flash at 2- or 4-bit; recent blocks are in T0 at FP16.*

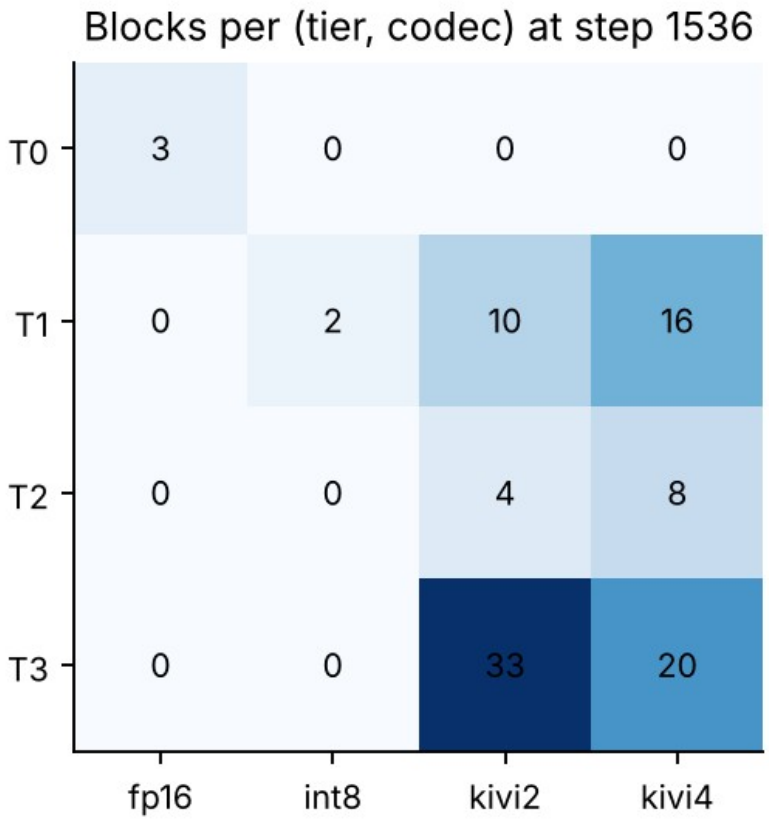


Figure 27. *Blocks per (tier, codec) pair at step 1,536.*

The final state has 96 blocks. Tier T0 holds only FP16 blocks (protected sink and recent window). T1 holds 28 blocks (mainly 4-bit), T2 holds 12 and the flash tier T3 holds 53, all at 4 or 2 bits. No block in T0 or T1 is FP16 beyond the protected set: at this budget the planner has chosen to spend bytes on keeping everything rather than on keeping any unprotected block uncompressed.

8.10 Summary of the simulation findings

Table 28. *Key findings from the simulations.*

Finding	**Evidence**
Joint (tier, codec) placement makes a tight budget feasible without deleting tokens	E1: 0 dropped tokens vs 973; E5b: feasibility requires a 4-bit codec
Quality is bounded and tunable	E4: λ and decay move error between 0.11 and 0.58 along a memory axis
Selective attention delivers most of the saving	E3: 24 of ~96 blocks gives 94.5 % mass recall
Hysteresis removes most churn	E4: 1,871 $\rightarrow$ 630 migrations with the default margin
Replanning cadence matters for flash traffic	E4: 599 $\rightarrow$ 3,326 flash reads from interval 2 $\rightarrow$ 64
Budget exhaustion is graceful, not catastrophic	E2: gradual error growth; overflow reported, not hidden
Some behaviour is non-obvious and needs a floor or a policy	E5: non-monotonic error at 3$\times$ memory; E2: compression when memory is ample

9. Hardware Results: RTL Simulation, Waveforms and Verification

This chapter presents the results for the three Verilog modules: how they were verified, the waveforms recorded during the testbench runs, the lint and mutation results and the synthesis statistics. Every figure showing signals was rendered directly from the value-change-dump (VCD) files written by the Verilator simulations (`docs/wave/trace_*.vcd`).

How to read the waveform figures

The figures are screenshots of the GTKWave waveform viewer (GTK 3 build, run in a virtual X display). Each opens one of the VCD dumps written by Verilator from the testbenches in `hw/tb` (rerun with `+trace`), with the signals, radix, zoom range and marker placed by a Tcl script kept in `docs/gtkwave/`. Buses are shown in hexadecimal or decimal as indicated in the signal values; the vertical line is the marker.

9.1 Verification method

1. **Golden models.** `uamm2/hwmodel.py` re-implements each module's integer arithmetic line for line.
2. **Vector generation.** The same file writes stimulus and expected-output hex files (`hw/vectors/*.hex`), including directed boundary cases and random cases.
3. **Self-checking testbenches.** Each testbench reads the vectors, drives the module, compares every output with the expectation and prints `PASS` or `FAIL` with a count.
4. **Verilator 5.** `make -C hw test` builds and runs all three benches with timing support; `+trace` additionally dumps a VCD.
5. **Lint.** `verilator --lint-only -Wall` on every module.
6. **Mutation testing.** Twelve single-fault variants of the RTL are generated automatically; the regression must fail for every one.
7. **Logic synthesis check.** Yosys `synth` with `check -assert` confirms the designs elaborate to generic gates with no inferred latches.

9.2 Sweep scorer

The scorer test applies four metadata datasets of 64 blocks, each swept at four pressure levels (0, 242, 243, 255), which gives 16 sweeps; a further sweep exercises restart behaviour (17 in total). Dataset 0 is a set of directed boundary cases: for each of the scores 0, 1, 88, 89, 90, 173, 174, 175, 254 and 255 a metadata word is generated whose weighted score equals that value, once unprotected and once

protected, followed by corner cases (all zeros, all 255, single maxima) and random fill. The other three datasets are random with a 20 % chance of the `protected` bit.

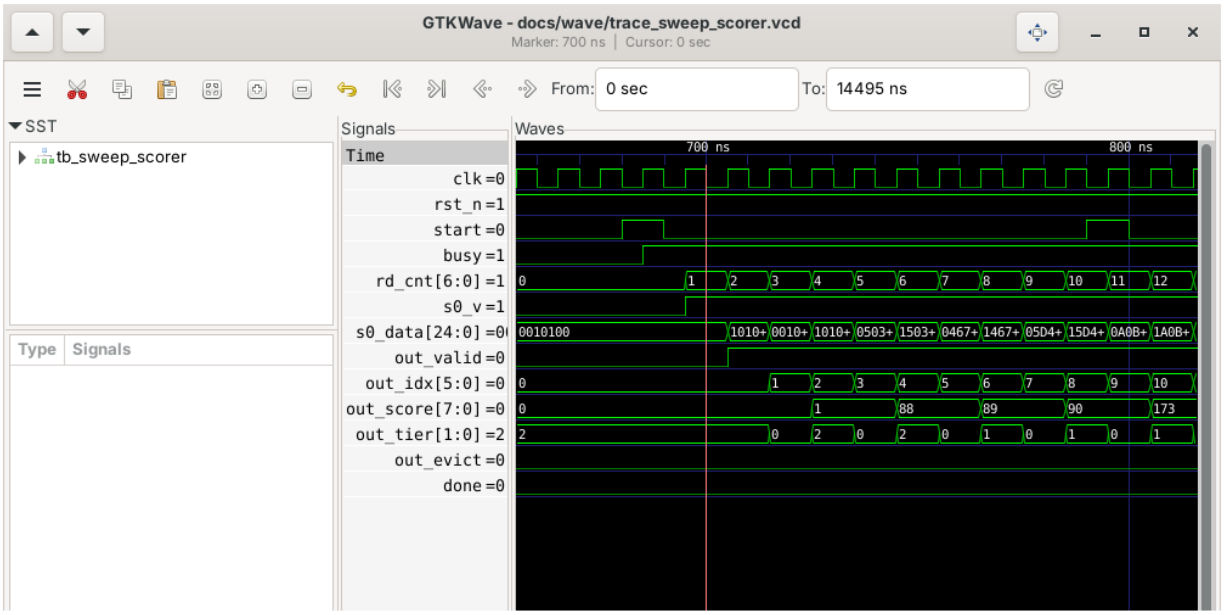


Figure 28. *Sweep start. start is asserted for one clock; busy rises on the next edge; the read counter then increments once per cycle. s0_data shows the metadata words read from the SRAM; the results appear two cycles later as out_valid with the block index, score and tier.*

The first results are the directed boundary cases, which makes the figure self-explanatory: block 0 has score 0 and tier 2 (cold) because it is unprotected; block 1 has the same metadata with `protected` set and is forced to tier 0; block 2 has score 1; blocks 4 and 5 have score 88 (just below the warm threshold of 89, cold when unprotected); blocks 6 and 7 have score 89 (exactly at the warm threshold: warm when unprotected, hot when protected); blocks 8 and 9 have score 90; block 10 has score 173 (one below the hot threshold: warm). The alternation of tier values 2, 0, 2, 0, 2, 0, 1, 0, 1, 0, 1 is the pair-wise unprotected/protected pattern.

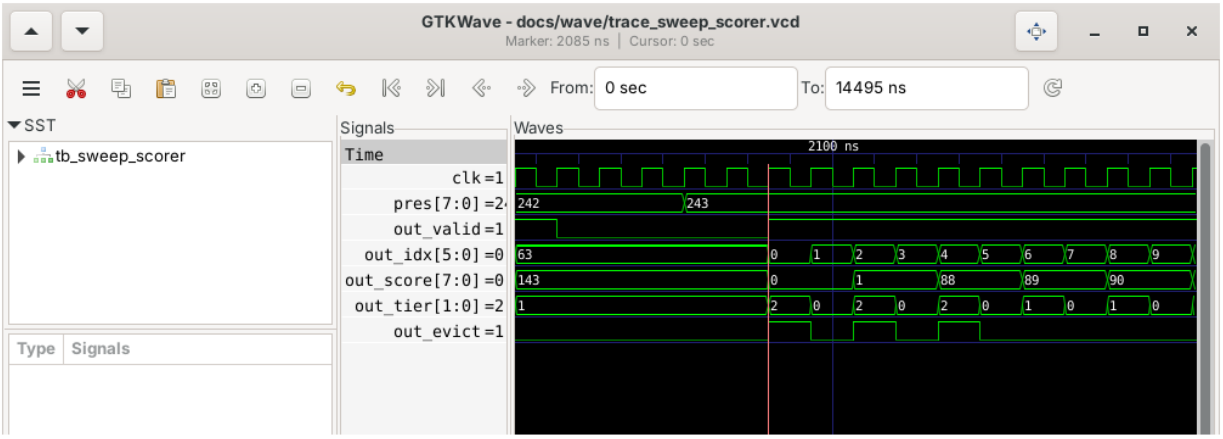


Figure 29. *Eviction under pressure. The pressure latched for the sweep changes from 242 to 243; in the sweep at 243, out_evict is raised for blocks 0, 2 and 4 (unprotected, score below 89) and not for the protected blocks or for blocks 6 and 7 (score 89).*

This is the behaviour the UAMM-1 RTL got wrong (it ignored `protected`) and the main reason for the boundary vectors: at pressure 242 no block is flagged for eviction; at 243 exactly the unprotected blocks whose score is below 89 are flagged.

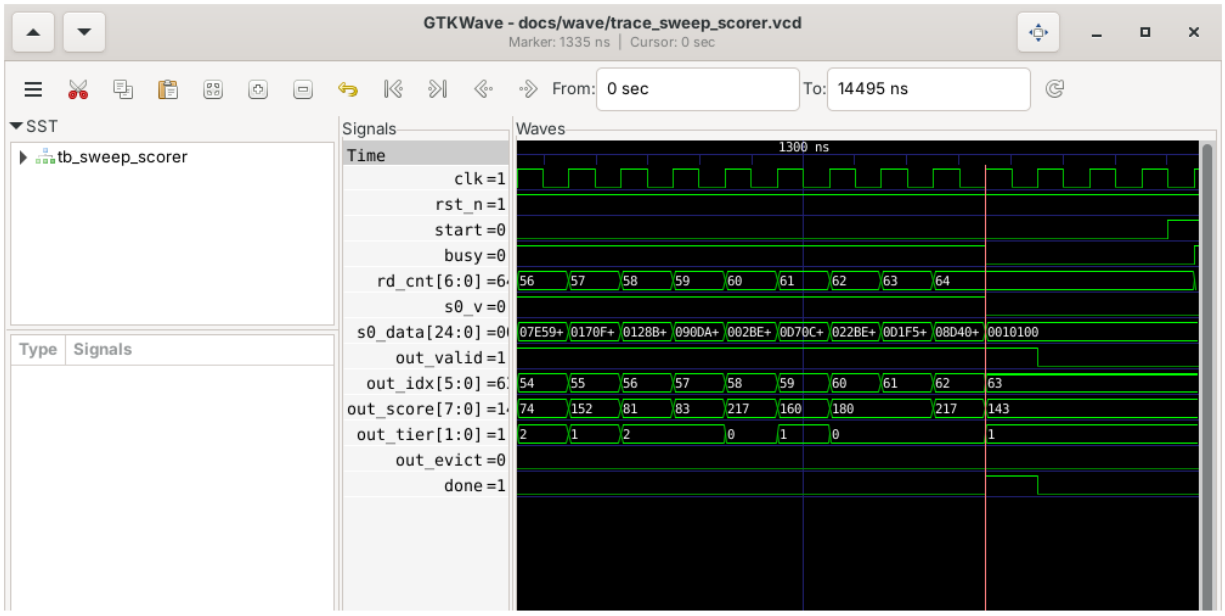


Figure 30. *End of a sweep. The read counter reaches 64 and busy falls; the last result (block 63) is emitted one cycle later together with the done pulse. A new start can follow immediately.*

The latency is as designed: for NBLK = 64 a sweep produces its last result NBLK + 3 cycles after `start`, and it sustains one block per cycle. At a 100 MHz clock a 64-block sweep takes under 0.7 µs; for 4,096 blocks it would take under 41 µs, so the sweep is cheap compared with the planning interval of several decode steps. (These are cycle counts from the simulation multiplied by an assumed clock, not timing-closure results.)

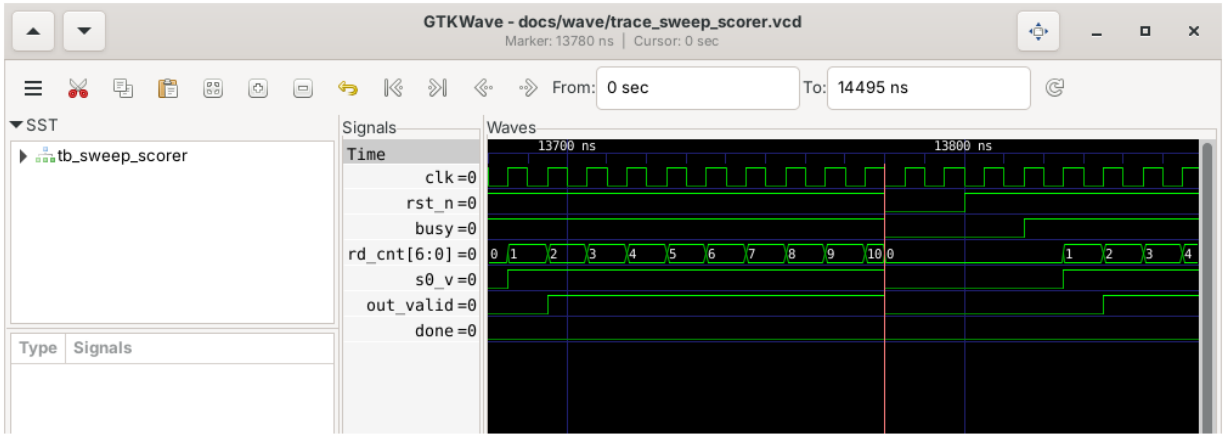


Figure 31. *Asynchronous reset in the middle of a sweep. rst_n is pulled low; busy, s0_v and out_valid drop immediately, the read counter returns to zero and a later start begins a clean sweep.*

Table 29. *Sweep scorer results.*

Check	Count	Result
Sweeps (4 datasets × 4 pressures + restart)	17	PASS
Total per-block comparisons (score, tier, evict, index, valid)	1,088	PASS
`start` while busy ignored	directed	PASS
Reset in the middle of a sweep	directed	PASS

9.3 Codec pipeline

The quantiser and dequantiser testbench feeds 400 vectors of 16 lanes. The vectors are produced in four modes in rotation: full-range random, narrow range around a random centre, constant group (range zero) and extremes (0, 1, 254, 255). Further variations set the zero point inside the data or deliberately **below** the true minimum (stale statistics), which exercises the clamp at code 15. The testbench checks both the 16 packed 4-bit codes and the 16 dequantised bytes.

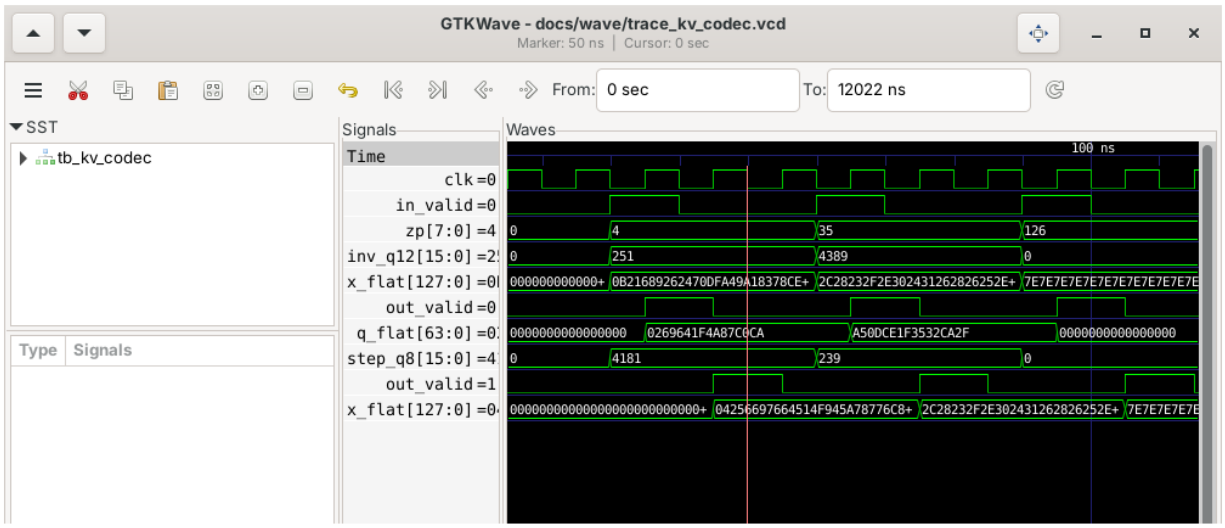


Figure 32. *Two vectors and a constant group through uamm_kv_quant4 and uamm_kv_dequant4. x_flat is the 128-bit input; q_flat is the packed result one cycle after in_valid; the dequantiser is fed the code vector and step and produces x' one cycle later.*

The third vector in the figure is the constant group (`inv_q12 = 0`, `step_q8 = 0`): all lanes are `7e`, every code is zero and the dequantiser reproduces `7e` exactly, as the model predicts. The first two vectors have non-trivial ranges (parameters 251 and 4,389 for inv_q12), show the one-cycle latency of each stage, and match the expected nibble words.

Table 30. *Codec results.*

Check	Count	Result
Codec vectors (quantiser and dequantiser outputs)	400	PASS

9.4 Block table

The block-table testbench performs 6,000 random operations (writes, lookups, and the occasional same-cycle read-and-write to one address) and compares every lookup with a behavioural reference table. It also checks that the valid bits are cleared by reset and that a read in the same cycle as a write returns the old entry.

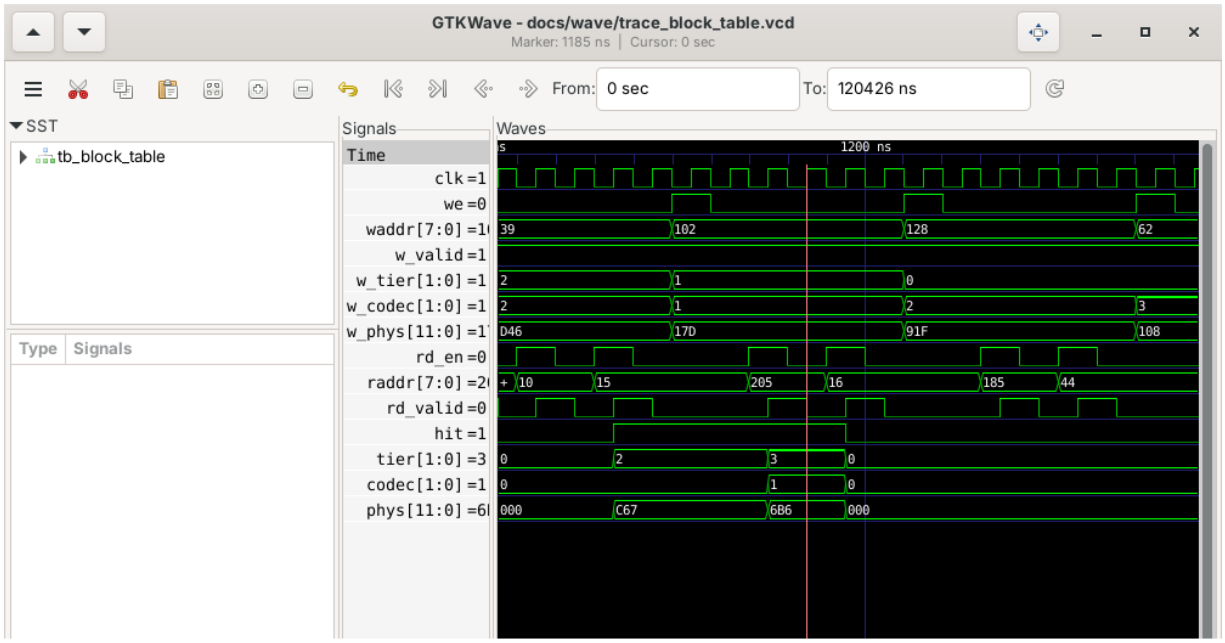


Figure 33. *Random updates and lookups. we pulses write {tier, codec, phys} to the address on waddr; rd_en pulses start a lookup whose result (hit, tier, codec, phys) appears one cycle later through rd_valid. A lookup of a never-written address returns hit = 0; a hit returns the stored fields.*

Table 31. *Block-table results.*

Check	Count	Result
Writes	3,049	PASS
Reads	2,971	PASS
Reads that hit	2,313	PASS

9.5 Regression, lint and mutation results

```
$ make -C hw test

/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --binary --timing -Wno-fatal -Wno-lint -CFLAGS "-std=c++20 -fcoroutines" --top-module
TB_SWEEP_SCORER PASS checks=1088
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --binary --timing -Wno-fatal -Wno-lint -CFLAGS "-std=c++20 -fcoroutines" --top-module
TB_KV_CODEC PASS vectors=400
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --binary --timing -Wno-fatal -Wno-lint -CFLAGS "-std=c++20 -fcoroutines" --top-module
TB_BLOCK_TABLE PASS writes=3049 reads=2971 hits=2313
```

Figure 34. *Output of make -C hw test: all three testbenches print PASS with their check counts.*

```
$ make -C hw lint   (verilator --lint-only -Wall)

/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --lint-only -Wall -Wno-DECLFILENAME --top-module uamm_sweep_scorer rtl/uamm_sweep_scor
- V e r i l a t i o n   R e p o r t:  rev vUNKNOWN-built20260516-4e853d8
- Verilator: Built from 0.035 MB sources in 2 modules, into 0.021 MB in 3 C++ files needing 0.000 MB
- Verilator: Walltime 0.013 s (elab=0.001, cvt=0.008, bld=0.000); cpu 0.009 s on 1 threads; allocated 20.539 MB
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --lint-only -Wall -Wno-DECLFILENAME --top-module uamm_kv_quant4 rtl/uamm_kv_codec.v
- V e r i l a t i o n   R e p o r t:  rev vUNKNOWN-built20260516-4e853d8
- Verilator: Built from 0.034 MB sources in 3 modules, into 0.045 MB in 3 C++ files needing 0.000 MB
- Verilator: Walltime 0.027 s (elab=0.001, cvt=0.011, bld=0.000); cpu 0.013 s on 1 threads; allocated 20.680 MB
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --lint-only -Wall -Wno-DECLFILENAME --top-module uamm_kv_dequant4 rtl/uamm_kv_codec.v
- V e r i l a t i o n   R e p o r t:  rev vUNKNOWN-built20260516-4e853d8
- Verilator: Built from 0.034 MB sources in 3 modules, into 0.037 MB in 3 C++ files needing 0.000 MB
- Verilator: Walltime 0.008 s (elab=0.001, cvt=0.004, bld=0.000); cpu 0.008 s on 1 threads; allocated 20.609 MB
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --lint-only -Wall -Wno-DECLFILENAME --top-module uamm_block_table rtl/uamm_block_table
- V e r i l a t i o n   R e p o r t:  rev vUNKNOWN-built20260516-4e853d8
- Verilator: Built from 0.033 MB sources in 2 modules, into 0.012 MB in 3 C++ files needing 0.000 MB
- Verilator: Walltime 0.006 s (elab=0.001, cvt=0.002, bld=0.000); cpu 0.006 s on 1 threads; allocated 20.418 MB
lint exit=0
```

Figure 35. *Output of make -C hw lint (verilator --lint-only -Wall on every module). Exit status 0, no warnings.*

Mutation testing measures the strength of the testbenches rather than the design: a testbench that cannot detect an injected fault is not protecting anything. Twelve mutants were generated, each a single small change to the RTL source, and the full regression was run on each.

```
$ RTL mutation testing (injected bugs must make a testbench FAIL)

mutation                          result
W_IMP 154 -> 153                  KILLED (testbench FAIL)
W_FRQ 38 -> 40                    KILLED (testbench FAIL)
TH_HOT 174 -> 175                 KILLED (testbench FAIL)
TH_WARM 89 -> 88                  KILLED (testbench FAIL)
EVICT_P 243 -> 244                KILLED (testbench FAIL)
tier ignores protected            KILLED (testbench FAIL)
evict ignores protected           KILLED (testbench FAIL)
quant: drop +2048 rounding        KILLED (testbench FAIL)
quant: remove clamp at 15         KILLED (testbench FAIL)
dequant: drop +128 rounding       KILLED (testbench FAIL)
dequant: remove saturation at 255 KILLED (testbench FAIL)
valid bit write disabled          KILLED (testbench FAIL)

12/12 mutants killed
```

Figure 36. *Mutation results: 12 of 12 injected faults are detected.*

The mutants cover weights, thresholds, rounding, clamping, saturation, the protected-block override and the valid-bit write. During development one mutant (removing the clamp at code 15) initially **survived** because no vector exercised an out-of-range code; the stale-zero-point vectors described in Section 9.3 were added to close that gap, and the mutant is now detected. This is the usual value of mutation testing: it found a hole in the tests that a pass count alone would not have shown.

9.6 Synthesis statistics

Table 32. *Yosys `synth -flatten` generic-cell statistics (technology-independent). Raw reports are in Appendix C.*

Module	Generic cells	Flip-flops	Combinational	Muxes	XOR/ XNOR
Sweep scorer (64 blocks)	4,426	1,667	2,759	1,246	119
Quantiser (16 lanes)	11,292	65	11,227	4	3,252

Module	Generic cells	Flip-flops	Combinational	Muxes	XOR/ XNOR
Dequantiser (16 lanes)	5,241	129	5,112	0	1,364
Block table (256 entries)	12,796	4,370	8,426	3,155	8

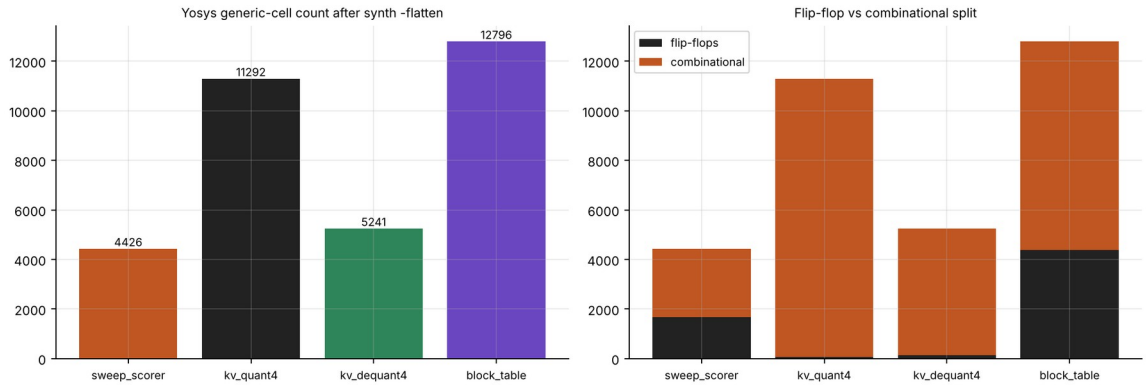


Figure 37. *Cell counts by module and split between sequential and combinational logic.*

- The **block table and the scorer metadata** appear as thousands of flip-flops because the memories are written as generic arrays and Yosys was run without a memory-mapping step. On an FPGA or ASIC flow they would be mapped to block RAM or SRAM macros. The flip-flop counts are therefore an upper bound that reflects storage, not control logic.
- The **quantiser is the largest logic block** (about 11,300 cells), dominated by the sixteen parallel 8×16-bit multipliers. A shift-based reciprocal (power-of-two ranges) or time-multiplexing fewer lanes would shrink it; this is noted as a design trade-off in Chapter 12.
- These are technology-independent counts. **No place-and-route, timing or power analysis has been done.**

10. The Ara Vector Kernel: Results

The vector-processor part of the system is a C kernel (`ara/kv_kernels.c`) that computes attention scores directly on 4-bit packed keys. It exists in two variants that share data layout and algorithm: a scalar path that executes on any host, and an RVV 1.0 path written with the standard vector intrinsics, compiled for a RISC-V vector target.

10.1 Algorithm in detail

For a block of T tokens and d channels with per-channel scale s_c and zero z_c, the key of token t is reconstructed as k_{t,c} = Q_{t,c} · s_c + z_c. The attention logit is therefore:

$$q \cdot k_t = \Sigma_c \; q_c \; (Q_\{t,c\} \; s_c + z_c) = \Sigma_c \; (q_c \; s_c) \; Q_\{t,c\} + \Sigma_c \; q_c \; z_c = \Sigma_c \; \tilde{q}_c \; Q_\{t,c\} + b$$

The pre-scaled query q̃_c = q_c · s_c and the bias b = Σ q_c z_c are computed once per block (`kv_qk_prepare`), and q̃ is de-interleaved into even-channel and odd-channel halves so that each packed byte (low nibble = even channel, high nibble = odd channel) multiplies two contiguous vectors. The per-token loop then does only integer-to-float conversion and fused multiply–add:

Table 33. *Mapping from the scalar algorithm to the RVV instructions.*

Step per token	**Scalar C**	**RVV instruction (observed in the compiled output)**
Load packed bytes	`row[j]`	`vle8.v` (byte load with `vsetvli`)
Extract low / high nibble	`& 0x0F`, `>> 4`	`vand.vx`, `vsrl.vi`
Widen to 32 bits	implicit	`vzext.vf4`
Convert to float	cast	`vfcvt.f.xu.v`
Load pre-scaled query halves	`qs_even[j]`, `qs_odd[j]`	`vle32.v`
Accumulate	`acc += …`	`vfmacc.vv` with tail-undisturbed policy
Reduce across lanes	sequential sum	`vfredusum.vs`
Add bias	`+ bias`	scalar add after `vfmv.f.s`

The tail-undisturbed (`_tu`) form of the multiply–add matters for correctness: when the last strip of a row has fewer than a full vector of elements, the accumulator lanes beyond the active length must keep their previous values until the final reduction. Using the default tail-agnostic form could let those lanes be overwritten with arbitrary values.

10.2 Numerical agreement (host)

```
$ scalar C kernel vs NumPy dequantize-then-dot; RVV compile check

case  tokens  dim  max|C-kernel - numpy dequant.dot|
   0      16   64  5.72e-06
   1      32   64  4.77e-06
   2      16  128  2.00e-05
   3      64   64  6.68e-06
RVV compile check (clang riscv64 rv64gcv): OK -> kv_kernels_rvv.o 2856 bytes (object only, NOT executed)
```

Figure 38. *Scalar C kernel versus NumPy dequantise-then-dot on random blocks, and the RVV compile check.*

On four block shapes the maximum absolute difference between the C kernel and the straightforward NumPy computation (dequantise the whole block, then take the dot product) is between 4.8×10^{-6} and 2.0×10^{-5}, which is floating-point rounding for sums of 64–128 products of magnitude about one. The test suite also checks the pre-scaling algebra and the nibble order.

10.3 RISC-V Vector compile check

The same source was compiled with clang for `riscv64-unknown-elf` with `-march=rv64gcv -O2` (object file only). The inner loop of the compiled code is shown below; it contains the expected vector instructions listed in the mapping table above.

```
$ clang --target=riscv64 -march=rv64gcv -O2 -S ara/kv_kernels.c   (kv_qk_dot_kivi4_rvv)

kv_qk_dot_kivi4_rvv:                    # @kv_qk_dot_kivi4_rvv
# %bb.0:
 vsetvli a6, zero, e32, m1, ta, ma
 blez a3, .LBB2_6
# %bb.1:
 li a6, 0
 srliw a7, a4, 31
 add a7, a7, a4
 sraiw a7, a7, 1
 vmv.v.i v8, 0
 li t0, 2
 vmv.s.x v9, zero
 j .LBB2_3
.LBB2_2:                                #   in Loop: Header=BB2_3 Depth=1
 vsetvli t1, zero, e32, m1, ta, ma
 vfredusum.vs v10, v10, v9
 vfmv.f.s fa5, v10
 fadd.s fa5, fa5, fa0
 slli t1, a6, 2
 add t1, t1, a5
 addi a6, a6, 1
 fsw fa5, 0(t1)
 beq a6, a3, .LBB2_6
.LBB2_3:                                # =>This Loop Header: Depth=1
                                        #     Child Loop BB2_5 Depth 2
 vmv1r.v v10, v8
 blt a4, t0, .LBB2_2
# %bb.4:                                #   in Loop: Header=BB2_3 Depth=1
 li t1, 0
 mul t2, a6, a7
 add t2, t2, a2
 vmv1r.v v10, v8
.LBB2_5:                                #   Parent Loop BB2_3 Depth=1
                                        # =>  This Inner Loop Header: Depth=2
 subw t3, a7, t1
 vsetvli t3, t3, e8, mf4, ta, ma
 add t4, t2, t1
 vle8.v v11, (t4)
 vand.vi v12, v11, 15
 vsrl.vi v11, v11, 4
 vsetvli zero, zero, e32, m1, ta, ma
 vzext.vf4 v13, v12
 vfcvt.f.xu.v v12, v13
 vzext.vf4 v13, v11
 vfcvt.f.xu.v v11, v13
 slli t4, t1, 2
 add t5, a0, t4
 vle32.v v13, (t5)
 add t4, t4, a1
 vle32.v v14, (t4)
 vsetvli zero, zero, e32, m1, tu, ma
 vfmacc.vv v10, v12, v13
 addw t1, t1, t3
 vfmacc.vv v10, v11, v14
 blt t1, a7, .LBB2_5
 j .LBB2_2
.LBB2_6:
 ret
```

Figure 39. *RVV assembly for the 4-bit key kernel (excerpt of the compiler output).*

This demonstrates that the intrinsics lower to the intended instruction sequence for a vector target. **It does not demonstrate that the code runs correctly or fast on Ara:** no Ara, Spike or QEMU execution has been performed (Chapter 12).

10.4 Why a fused kernel: bandwidth arithmetic

An unfused implementation would first dequantise the key block into a floating-point buffer and then compute dot products. If that buffer is staged in memory, each 4-bit key element costs 0.5 bytes to read, 4 bytes to write and 4 bytes to read back (8.5 bytes of traffic) versus 0.5 bytes in the fused kernel, a factor of 17 in traffic for the key stream. If the dequantised tile is kept entirely in registers the memory traffic is the same as the fused kernel's but the kernel then executes the extra dequantisation

instructions (a multiply and add per element) before each multiply–add. The fused form removes both. These are analytical counts from the algorithm, not measurements.

What would make this a measured result

Run the kernel on Spike for functional verification, then on the Ara RTL simulation or an FPGA build to obtain cycles per token for block sizes of 16–64 tokens and head dimensions of 64–128; compare with a dequantise-then-dot baseline written with the same intrinsics, on the same data.

11. Research Papers, Inferences and How We Used Them

This chapter collects, source by source, what we concluded and how it changed the design. The detailed narrative for most entries is in Chapter 2; here they are consolidated so that the provenance of each design decision can be traced.

Table 34. *Sources, inferences and effects on the design.*

Source	Inference we drew	How we used it in UAMM-2	Not used, and why
PagedAttention / vLLM (SOSP 2023)	Blocks are the right unit; block tables are cheap	Fixed 16-token blocks, block table RTL	Prefix sharing / copy-on-write (open item)
H2O (NeurIPS 2023)	Attention is power-law; accumulated attention is a cheap importance signal	Decayed token score → block importance (normalised by p95)	Irreversible eviction as default
Scissorhands (NeurIPS 2023)	Importance persists, but can shift	Workload with topic revisits; ability to promote cold blocks	Eviction-only budgeting
StreamingLLM (ICLR 2024)	Sink tokens + recent window must be kept	Protected blocks (RTL and Python)	Dropping the middle
SnapKV (2024)	A small observation window can seed relevance	Noted as a prefill-time seed (future)	One-shot prompt compression
FastGen / PyramidKV	Budgets should differ by head and layer	Structure for future (layer, head) index	Per-head policies (open item)
Quest (ICML 2024)	Min/max page summaries give a query-aware bound	Selection by Σ `max(q·kmin, q·kmax)`; tested as a true bound	Page-level only; no token-level refinement
InfiniGen (OSDI 2024)	A slow tier works if the fetch set is predicted	Multi-tier model; planner prices tier latency; prefetch listed as next step	Layer-ahead speculation
KIVI (ICML 2024)	Quantise keys per channel and values per token	Codec layer; measured 1.9–5.4× lower key error	Residual full-precision window (we use protected T0 blocks instead)
KVQuant (NeurIPS 2024)	Outliers must be handled explicitly; sub-4-bit needs care	Outlier-aware test data; hardware quantiser kept simple	Non-uniform datatypes, pre-RoPE keys (open item)
Palu (2024)	Low-rank storage composes with quantisation	Left as a future codec family	Needs model weights
FlexGen (ICML 2023)	Placement across tiers can be searched; offload needs compression	Tier + codec search, greedy rather than LP	LP-based optimiser (open item)
LLM in a flash	Flash needs large, rare, sequential reads	Flash holds only cold, entropy-capable blocks; large fixed cost per access	Row–column bundling
MQA / GQA / MLA	Architecture changes shrink K/V at training time	Engine accepts any vector size	No model changes
EvicPress	Compression and eviction should be decided jointly with a utility	Utility planner (λ-weighted error + latency + congestion)	Our additions: hysteresis, ratchet, query-aware reads, RTL
KV-Compress / Zipage	Evict at the paging granularity	Block as unit for every decision	Variable per-head budgets
TensorRT-LLM KV	Compress when pages change tier	Codec change is part of the same	Vendor-specific

Source	Inference we drew	How we used it in UAMM-2	Not used, and why
compression		plan step as the move	kernels
CXL-SpecKV	A hardware cache controller can hide slow-memory latency	RTL block table + sweep scorer as controller pieces	CXL fabric
Ara / Ara2	Open RVV vector unit with AXI memory path	Fused QK kernel; codec RTL placed on the load path	Anything run on Ara itself (open item)
TVM / compiler runtimes	A runtime is an adapter target, not a design dependency	`RuntimeAdapter` seam; skeleton with checklist	Actual integration (open item)
SmoothQuant	Targets weights and activations, not the KV cache	Removed from UAMM-1's list of KV solutions	—
Our UAMM-1	Placement without precision cannot fit a tight budget	Ablation baseline; 10 defects fixed	Threshold policy as default

11.1 Cross-cutting lessons

1. **Combine axes, not just methods.** The ablation shows that placement alone, codecs alone and eviction alone each leave the budget problem unsolved or solved by loss.
2. **Measure, do not assume.** Per-block calibrated error is more reliable than a formula; advertised compression ratios are not measured ones.
3. **Make irreversibility explicit.** The ratchet and opt-in drop make every loss visible and bounded.
4. **Churn is a first-class cost.** Hysteresis and replanning cadence changed migrations by up to 4× and flash reads by up to 5×.
5. **Hardware should accelerate the inner loop and be verified like software.** Golden models, boundary vectors and mutation testing found real defects (an ignored `protected` input, an untested clamp).
6. **Keep seams.** Codec, runtime and kernel boundaries let each piece be replaced as real measurements arrive.

12. Problems UAMM-2 Does Not Yet Address, and Our Solutions

UAMM-2 closes the main gaps of UAMM-1, but several technical problems remain, most of them about quantisation and the precision that attention actually requires. This chapter states each problem, explains why it matters, and gives the solution and the implementation we worked out. The ideas were developed by reasoning about the mathematics of attention and about the structure of the existing code; they indicate where each change would be made in the project of Chapter 6.

12.1 Problems of quantisation and required precision

Problem 1. Key error is amplified by the softmax; value error is not

Problem. UAMM-2 treats keys and values with the same bit width. The two enter attention differently.

Why it matters. A key error Δk changes the logit by $\delta = q \cdot \Delta k$, and the attention weight of that token is scaled by about e^{δ}. Large queries make the softmax sharp, so a small δ can change which token dominates. A value error ΔV changes the output only linearly, by $\Sigma\, w_i \Delta v_i$. Keys therefore need more precision than values for the same output error. On our workload the 2-bit codec has a squared error of 0.130 per block, which is tolerable for values but can reorder attention among keys that have close logits.

Solution and implementation idea. Use separate bit widths for keys and values (a pair such as K4/V2) and let the planner choose the pair. Implementation: extend the codec name to carry two widths; `encode` receives `bits_k` and `bits_v`; `calibrate` measures a *logit-error* metric for keys, E_k = max over sampled queries of $|q \cdot (k - \hat{k})|$, in addition to the value error, and the planner prices the two separately.

```
# codecs.py (sketch)
def encode(k, v, bits_k, bits_v, v_group):
    qk, sk, zk = _asym_quant(k, bits_k, axis=0)          # per-channel keys
    qv, sv, zv = _asym_quant(v_grouped, bits_v, axis=2)   # per-token values
    return pack(qk, bits_k) + pack(qv, bits_v) + meta

# planner cost: separate quality terms
cost = p*read_ms + lam_k*heat*err_logit_k[c] + lam_v*heat*err_v[c] + congestion
```

Basis. KIVI quantises keys and values along different axes; hardware compression engines commonly give keys 4 bits and values 2 bits.

Problem 2. No guaranteed bound on the logit error of a compressed block

Problem. The planner uses a calibrated average error. It does not know whether, for the current query, a compressed block could change the ranking of the top tokens.

Why it matters. With per-channel scales s_c, the dequantisation error of channel c is at most s_c/2. For a query q the logit error of any token in the block is bounded

by $|\delta| \le \tfrac{1}{2} \Sigma_c |q_c| \cdot s_c$. This is cheap (one dot product with the scale vector) and is exactly the kind of quantity the selection step already computes from the min/max summaries.

Solution and implementation idea. At selection time compute $B_{err}(b) = \tfrac{1}{2} \Sigma_c |q_c| s_{b,c}$ for every candidate block. If $B_{err}(b)$ exceeds a tolerance τ times the logit gap to the block that would be ranked next, promote the block (decode it from a higher-precision copy or from FP16 in the fast tier) or exclude it from the approximate path. Implementation: store `scale_k` (already in the payload) as a resident per-block vector, add `err_bound(q, block)` to `attention.py`, and call it inside `select_blocks`.

```
def err_bound(q, scale_k):          # scale_k: per-channel key scales of the block
    return 0.5 * float(np.abs(q) @ scale_k)

# in select_blocks: keep bound ranking, add a risk flag
risky = [b for b in chosen if err_bound(q, b.scale_k) > tau]
```

Basis. Follows from the affine quantiser of KIVI and the bound-based selection of Quest.

Problem 3. Outlier tokens, not only outlier channels

Problem. Per-channel key quantisation handles channels with large magnitude. A single token with an unusually large key or value inside a 16-token block still stretches the min/max range of every channel it touches.

Why it matters. In a block of 16 tokens one outlier can enlarge the range for all channels by a factor of several, so the other 15 tokens use only a few quantisation levels. Heavy-hitter and sink tokens are exactly the tokens most likely to be large and most important to keep accurate.

Solution and implementation idea. Keep a small FP16 side buffer per block for the top-m tokens by key norm ($m = 1$ or 2), exclude them from the min/max computation, and store their indices. The kernel then adds their exact contribution. Implementation: in `_asym_quant`, find the indices of the m largest rows, replace them with the channel median before computing min/max, and append `(index, fp16 key, fp16 value)` records to the payload; `decode` writes the exact rows back. The overhead is $m \times 2d \times 2$ bytes per block (256 B for $m = 1$, $d = 64$).

```
idx = np.argsort(-np.linalg.norm(k, axis=1))[:m]
k_clip = k.copy(); k_clip[idx] = np.median(k, axis=0)
q, s, z = _asym_quant(k_clip, bits, axis=0)
payload += pack_sidecar(idx, k[idx], v[idx])   # exact rows
```

Basis. KVQuant keeps outliers in a separate sparse structure (dense-and-sparse quantisation).

Problem 4. Errors compound when a block is re-encoded repeatedly

Problem. Re-encoding decodes the current payload and encodes it again. The stored error table is measured from the FP16 original, so it does not reflect earlier lossy steps.

Why it matters. A block that moves INT8 → 4-bit → 2-bit has a true error larger than the table's 2-bit entry. The planner then underestimates the cost of its own past decisions. With independent errors e_1, e_2 the accumulated error is approximately $1 - (1 - e_1)(1 - e_2)$.

Solution and implementation idea. Keep an accumulated error per block and use it in the quality term and in the ratchet check; limit the chain length (for example, at most two lossy re-encodes); and for cold blocks keep the FP16 original in flash so that re-encoding always starts from it. Implementation: add `b.err_acc` to `KVBlock`, update it in `reencode` as err_acc = 1 − (1 − err_acc)(1 − err_table[new]), and replace `b.err_table[c]` in the planner by max(err_table[c], err_acc_after_move).

```
b.err_acc = 1 - (1 - b.err_acc) * (1 - b.err_table[codec])      # in reencode()
quality_term = lam * heat * max(b.err_table[c], 1 - (1 - b.err_acc) * (1 - b.err_table[c]))
```

Basis. The ratchet and calibration in Chapter 5; KVQuant calibrates quantisation error before use.

Problem 5. Per-block min/max ranges drift with block size and content

Problem. Asymmetric quantisation uses the exact minimum and maximum of a block. Small blocks have noisy ranges and any outlier sets the step size for the whole block.

Why it matters. Block size 16 gave good results in our sweep, yet a block of 8 tokens had the lowest error (0.110) and a block of 64 the highest (0.341), showing the dependence of error on the range estimate.

Solution and implementation idea. Use clipped ranges: compute the range from the 2nd and 98th percentile of each channel and clamp the extremes to the end codes; select the clipping percentile per block by minimising the calibrated error (the calibration pass already encodes each block, so trying two clipping levels adds one more encode). Combine with Problem 3 so that clipped values are handled by the sidecar.

Basis. Clipping and percentile calibration are standard in post-training quantisation practice and are used in KV-cache quantisers.

Problem 6. Precision is chosen by heat alone, not by sensitivity

Problem. The planner raises or lowers precision according to block heat (attention received). But two blocks with equal heat can differ in how much their error changes the output.

Why it matters. The output error caused by a block is roughly proportional to its attention weight times the norm of the error in its values and to the logit change of its keys. A block with high heat but a very flat value distribution tolerates more error than a block with the same heat and large values.

Solution and implementation idea. Define a sensitivity score S(b) = w̄_b · (‖V_b‖_F · e_v(c) + ‖q‖ · e_k(c)), where w̄_b is the mean attention weight on the block, and let the quality term use S(b) instead of heat × err. Implementation: `h2o.py` already supplies w̄_b through the token scores; `paged_cache.py` can store ‖V_b‖ at sealing; the planner multiplies.

```
S = b.mean_attn * (b.v_norm * err_v[c] + q_norm * err_k[c])
cost = p*read_ms + lam * S + congestion
```

Basis. Sensitivity-based allocation is the principle behind mixed-precision quantisation schemes.

Problem 7. Positional encoding mixes channels before quantisation

Problem. If keys are stored after rotary position embedding (RoPE), each channel pair is rotated by a position-dependent angle, which changes the per-channel statistics and makes the quantisation error position dependent.

Why it matters. Rotation mixes the large-magnitude channels into their pair partner, so outlier channels are no longer isolated and per-channel quantisation loses part of its advantage.

Solution and implementation idea. Store keys before RoPE and apply the rotation inside the attention kernel: since (R_p k) · q = k · (R_p$^{\top}$ q), rotate the query by the inverse of each token's rotation instead of the stored key. For a block of 16 tokens this costs 16 rotated copies of q (16 × d multiply-adds), small compared with reading the keys. Implementation: `kv_qk_prepare` is called per token position, with cosine and sine tables for the block's positions.

Basis. KVQuant quantises keys before the rotary embedding for this reason.

12.2 Problems of selection, placement and planning

Problem 8. No prefetch: returning to an old topic is served from flash on demand

Problem. Blocks that were cold become hot when the query returns to an earlier topic; the first reads must wait for the slow level.

Why it matters. With replanning every 64 steps the experiments show 3,326 flash reads and 0.272 ms per token, against 599 reads and 0.090 ms with replanning every 2 steps. Replanning more often costs migrations (768 against 345).

Solution and implementation idea. Use the bound ranking that selection already computes. After choosing the top B blocks, take ranks B+1 … B+P (for example P =

8) as predicted next reads and promote any that are in flash into DRAM during the current step, overlapped with the attention computation. Implementation: a `prefetch(q, candidates)` function in `attention.py` returns block identifiers; the simulator calls `cache.reencode` and a tier move through the adapter asynchronously; the cost model charges the transfer to a background channel.

```
ranked = np.argsort(-bound)
read     = ranked[:B]
prefetch = [b for b in ranked[B:B+P] if tier[b] == FLASH]   # promote ahead of need
```

Basis. InfiniGen prefetches the entries predicted to be needed from the slow tier.

Problem 9. One flattened vector per token: no layers, heads or grouped-query structure

Problem. The engine stores one vector per token. Real models hold layers × KV heads independent streams, each with its own attention pattern.

Why it matters. An 8B-class geometry has 32 layers × 8 KV heads = 256 streams. A single global fetch budget of 24 blocks starves heads that need broad attention and wastes reads on heads whose attention is sharp.

Solution and implementation idea. Add (layer, head) to the block identity and index the block table by (layer, head, block). Give each stream a budget B_{l,h} from its measured attention entropy H_{l,h}: B_{l,h} ∝ exp(H_{l,h}) normalised to the total budget. Implementation: `KVBlock` gains `layer` and `head`; `H2OImportance` keeps one score array per stream; `select_blocks` is called per stream with its own budget; the RTL block table widens its address by $\log_2(L \cdot H)$ bits.

Basis. PyramidKV assigns layer-dependent budgets; FastGen uses head-dependent policies.

Problem 10. The planner is greedy

Problem. Blocks are visited in heat order and each takes its cheapest feasible slot. A fast level can be filled by moderately hot blocks before a slightly hotter block is considered.

Why it matters. The budget sweep shows non-monotone behaviour: relative error 0.115 at twice the budget but 0.152 at three times, a sign that the heuristic is not always near the best assignment.

Solution and implementation idea. Formulate the assignment as a multiple-choice knapsack: choose one (tier, codec) per block to minimise Σ cost(b, t, c) subject to Σ bytes in tier t ≤ cap_t. Solve it offline on recorded traces with an integer-programming solver to measure the gap; if the gap is material use Lagrangian relaxation (a price per tier, adjusted until capacities are met), which is cheap enough to run online and maps to the `capacity_price` term that already exists.

```
for it in range(20):
    choice = {b: argmin_tc(cost(b,t,c) + price[t]*bytes(b,c)) for b in blocks}
```

```
    for t in tiers:
        price[t] = max(0, price[t] + step * (used(choice,t) - cap[t]))   # dual update
```

Basis. FlexGen formulates placement across memory levels as a linear program.

Problem 11. No quality floor: the planner may lose accuracy it did not need to lose

Problem. The cost function trades accuracy for latency everywhere, even when memory is plentiful.

Why it matters. At 512 tokens the cache fits at FP16 (error 0.0013 under UAMM-1), yet UAMM-2 compresses to 0.54 of FP16 and the error becomes 0.016. At a budget three times larger the error rises from 0.115 to 0.152 because the spare room is spent on speed.

Solution and implementation idea. Gate compression on pressure and add a per-request error budget. A block is demoted only if its tier's fill exceeds a threshold φ (for example 0.85) or if the saving exceeds a margin. The request carries E_max; the planner maintains Σ_b heat(b) $\cdot$ err_acc(b) $\leq$ E_max and refuses a demotion that would break it. Implementation: add `pressure(tier)` to `tiers.py`, a `min_fill_to_demote` parameter to `Config`, and the running sum to the planner.

```
if used[t] / cap[t] < cfg.min_fill_to_demote and codec_rank(c) > b.rank:
    continue                                  # no pressure: keep current precision
if err_sum + heat * new_err > cfg.error_budget:
    continue
```

Basis. KIVI keeps a recent window at full precision for the same reason.

Problem 12. Selection summaries are not charged to memory

Problem. Each block keeps minimum and maximum key vectors resident for selection, but their bytes are not counted in any tier budget.

Why it matters. For 64 channels in FP16 the summaries occupy 256 B per block, about 25 KB for 96 blocks, about 15 % of the 168 KB budget used in the experiments. For d = 128 the summaries grow to 512 B per block while a 2-bit block is about 1.8 KB, so they still add roughly 28 % to the 2-bit footprint.

Solution and implementation idea. Charge the summaries to the fast level; store them in 8 bits using the same affine quantiser, with floor/ceil rounding so the bound remains valid; or keep one summary per group of 8 channels and use the looser bound. Measure the effect on selection recall. Implementation: `kmin`/`kmax` become `int8` arrays with the rule kmin = floor, kmax = ceil on the quantised grid.

Basis. Quest stores two vectors per page and discusses this overhead.

Problem 13. No graceful fallback when the budget is exhausted

Problem. When even 2-bit blocks do not fit, the planner reports overflow and does nothing more.

Why it matters. At 3,072 tokens the all-codec system overflows by 28.5 KB (INT8 and 4-bit only: 132 KB).

Solution and implementation idea. When overflow persists after demotion, evict the lowest-importance tokens inside the least important blocks (token level, not block level), with an explicit counter and an importance threshold, and with the block table recording the surviving token mask. Implementation: a `compact(block, keep_mask)` function re-encodes a block with fewer tokens; `allow_drop` becomes a three-valued policy (never, block, token).

Basis. H2O evicts by accumulated attention; KV-Compress evicts at the paging granularity.

Problem 14. No sharing between sequences

Problem. Every sequence owns its blocks, so a common prompt is stored once per sequence.

Why it matters. An 800-token system prompt shared by four sessions occupies four times the memory of one copy, which is serious when the whole budget is a few hundred megabytes.

Solution and implementation idea. Reference-count blocks and use copy-on-write in the block table. Shared prefix blocks are identified by a hash of their token ids; they are stored at the lowest precision tolerated by the most demanding sharer (the maximum of the sharers' rank floors). Implementation: `KVBlock.refcount`, a dictionary from prefix hash to block id, and a copy on first write to a shared block.

Basis. PagedAttention shares blocks across sequences with copy-on-write.

12.3 Problems of systems and hardware

Problem 15. Migration is not synchronised with attention

Problem. In the simulator a block moves instantaneously. In a real system an attention step may be reading a block while it is being re-encoded.

Why it matters. A block re-encoded from 4-bit to 2-bit while a step reads it would return mixed data and corrupt that step's output.

Solution and implementation idea. Add a version counter and a busy flag to every block-table entry. Migration writes the new payload into a second buffer, then flips the entry (tier, codec, physical address, version + 1) in one table update. An attention step records the version it started with and re-reads if the version changed. This is the same atomic-update idea as double buffering and fits the existing block-table RTL, which already returns the old entry when a read and a write coincide.

Basis. FlexGen overlaps data movement with computation; the block table is the natural place for per-block state.

Problem 16. Flash writes and wear

Problem. Every demotion into flash writes a block. Flash has limited write endurance and large erase units.

Why it matters. A planner that moves blocks back and forth would wear the device. The one-way precision rule already stops promoting precision, but tier moves between DRAM and flash can still alternate.

Solution and implementation idea. Count writes per block and per flash page, add a write cost to the move cost in `tiers.py` (proportional to the wear fraction), and require a larger hysteresis margin for moves into flash. Write blocks to flash in log-structured fashion so that a block is written once per demotion.

Basis. LLM in a flash stresses reading flash in few, large, sequential accesses; the same pattern suits writes.

Problem 17. Hardware blocks do not yet match the software policy

Problem. (a) The codec RTL quantises 8-bit data, whereas the software codecs start from FP16. (b) The scorer implements the original weighted score, not the utility cost. (c) The designs have not been taken through placement and routing.

Why it matters. The quantiser alone is about 11,300 generic gates, mostly sixteen 8×16-bit multipliers; the table and scorer memories appear as flip-flops until mapped to RAM.

Solution and implementation idea. (a) Add a front stage that converts FP16 to a group-normalised 8-bit value, $(x - \min)/(\max - \min) \times 255$, using the group min and max supplied by firmware. (b) Implement the utility cost in fixed point with lookup tables for the read-time and error terms, and verify it against the Python planner with the same golden-model and mutation method used for the current scorer. (c) Map the memories to block RAM and replace the reciprocal multiplier with a shift when the range is rounded up to a power of two, or reduce the number of lanes.

Basis. Hardware KV-compression engines use group quantisers of the same structure.

12.4 Order of work

Table 35. *Suggested order of implementation.*

Step	Goal	Problems
1	Precision control: separate key/value widths, logit-error bound, sidecar, error tracking, sensitivity	1, 2, 3, 4, 5, 6, 7
2	Policy: quality floor, prefetch, per-layer budgets, summary accounting, better optimiser	8, 9, 10, 11, 12
3	Capacity limits and sharing: token-level fallback, copy-on-write	13, 14
4	Systems and hardware: atomic migration, flash wear, FP16 front stage, utility scorer	15, 16, 17

13. Putting the Numbers in a Realistic Frame

Chapter 8 reports results on a small, deliberately tight configuration (168 KB for 1,536 tokens of a 64-channel cache). This chapter extrapolates those *ratios* to the scale of an actual edge deployment, using arithmetic only. These are projections from measured ratios and stated assumptions, not new measurements.

13.1 Why the compression ratios transfer to other geometries

For a block of t tokens and d channels with value group g, the bytes of a b-bit block relative to FP16 are b/16 + 1/t + 1/g (scales and zeros are two FP16 numbers each, per channel for keys and per group for values). The expression does not depend on d. For t = 16 and g = 32 it gives 0.594 (INT8), 0.344 (4-bit) and 0.219 (2-bit), identical to the measured values in Chapter 5. Larger blocks reduce the metadata share (t = 32 gives 0.3125 for 4-bit) but, as Section 8.4.3 shows, enlarge the unit of placement.

13.2 What fits on a phone-class device

Table 36. *Llama-3-8B-like geometry (131,072 B per token in FP16). The mixed rows use the resident ratios measured in E1 and assume the same mix at this scale.*

Storage	Bytes vs FP16	Max tokens in 1 GiB	KV size at 32K tokens
FP16	1.000	8,192	4.00 GiB
INT8	0.594	13,791	2.38 GiB
Mixed int8/4-bit (E1: 0.371)	0.371	22,081	1.48 GiB
Mixed incl. 2-bit (E1: 0.310)	0.310	26,426	1.24 GiB
All 4-bit	0.344	23,814	1.38 GiB
All 2-bit	0.219	37,406	0.88 GiB

In the model, a 1 GiB KV budget that holds about 8,200 tokens in FP16 holds about 26,000 tokens at the 0.310 mix without dropping any: a gain of 3.2×, in line with 1/0.310. A 32K-token context shrinks from 4 GiB to about 1.2 GiB.

13.3 Reads per token

In the 1,536-token run, after step 1,024 the engine read on average 35 % of the blocks per decode step (24 selected plus the protected set). If the same fraction applied at larger scale and the blocks read had the average encoding, bytes read per token would be roughly 0.35 × 0.31 = 0.11 of the dense FP16 read. The fraction would shrink further with longer contexts because the block budget is fixed while the cache grows, but recall would also fall (Section 8.3); the budget has to be tuned against the application's quality needs.

Table 37. *Projected KV-read bound for the 8B-like geometry. For 8,192 tokens the measured read fraction is used; beyond that the fixed 24-block budget plus a 36-token protected set is assumed. Weight reads and compute are ignored, and the bound is not a prediction of end-to-end speed.*

Context	Dense FP16 KV read / token	KV-only bound (tok/s, 12.8 GB/s)	Selective + mixed codecs read / token	KV-only bound (tok/s)
8,192	1.00 GiB	11.9	112 MiB	109.3
32,768	4.00 GiB	3.0	16 MiB	750.0
131,072	16.00 GiB	0.7	16 MiB	750.0

At 32K tokens the dense FP16 read bound is about 3 tokens per second on 12.8 GB/s memory; with selective reads and mixed codecs the KV-only bound is far higher, which would make the weights, not the cache, the limit again. This is the sense in which the design aims to restore the bottleneck to where it belongs. Whether recall at 24 blocks is sufficient at that length for a real model is exactly the question that real-model validation (Chapter 12, gap 1) must answer.

13.4 What would convince a sceptic

Table 38. *Evidence available and evidence needed for each claim.*

Claim	Evidence in this report	Evidence still needed
The joint policy beats placement alone in a tight budget	E1, E2, E5 on the synthetic trace	Same ablation on real traces, with task accuracy
Hysteresis and cadence control churn	E4	Energy measured on a device
Hardware blocks implement the specified arithmetic	Bit-exact RTL regression, mutation tests, lint	Place-and-route, timing, power, integration with a bus
The kernel is correct	Host agreement with NumPy; RVV compiles	Execution on Spike/Ara
The design speeds up an edge device	Modelled fetch time	Measured tokens/s on target hardware

13.5 Deployment sketch

1. Pin an open 1–3B model, a runtime build and a target (FPGA with CVA6+Ara, or an Arm SoC for the first measurements).
2. Implement the adapter and replace the cost constants with measurements.
3. Run perplexity and long-context benchmarks at FP16, uniform 4-bit and UAMM-2, at equal memory budgets.
4. Use results to set λ, block budget and plan interval per device class.
5. Move the scorer and codec to hardware only if profiling shows the software planner or codec to be a bottleneck.

14. Conclusion

Running language models on edge processors is limited by memory: the KV cache grows with every token, must be read in full by standard attention, and competes with the weights for a few gigabytes of shared DRAM. The research community has produced strong but partial answers: block paging, attention-based eviction, query-aware selection, low-bit quantisation, offloading and compression engines. Each leaves a dimension of the problem open.

UAMM-2 combines them around one idea: a block's placement is a (tier, codec) pair chosen together from measured importance, measured error and a priced cost, with a precision ratchet, hysteresis and query-aware reads. It adds three verified RTL blocks, a fused vector kernel for the Ara vector processor and a runtime seam for compiler-based runtimes, and keeps the codec interface open for external compression engines.

On a controlled synthetic workload with assumed memory costs, the combined design keeps every token within a budget that makes the earlier threshold policy delete 63 % of them, retains 0.94 of the attention mass instead of 0.76, and reduces modelled fetch time by about 8× at a cost of bounded quantisation error. It degrades gracefully as the budget is stretched, and its sensitivity to each knob has been measured. The RTL is bit-exact against golden models and every injected fault was caught.

What remains is to replace the assumptions with measurements: real model activations, real memory costs, execution of the vector kernel on Ara or a simulator and integration with a deployed runtime. Chapter 12 lists these steps with a proposed order. The design, the code and the experiments are organised so that each of those substitutions is a local change.

Code availability

The complete source code of UAMM-1 and UAMM-2, the experiment scripts and raw results, the RTL with its testbenches and waveform dumps, and the scripts that regenerate every figure and table in this report are available at https://github.com/IshanviPrasad29/uamm2-kv-cache-manager.

References

1. Kwon, W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180.
2. Zhang, Z. et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. NeurIPS 2023. arXiv:2306.14048.
3. Liu, Z. et al. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. NeurIPS 2023. arXiv:2305.17118.
4. Xiao, G. et al. Efficient Streaming Language Models with Attention Sinks. ICLR 2024. arXiv:2309.17453.
5. Li, Y. et al. SnapKV: LLM Knows What You are Looking for Before Generation. 2024. arXiv:2404.14469.
6. Ge, S. et al. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs (FastGen). 2023. arXiv:2310.01801.
7. Cai, Z. et al. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. 2024. arXiv:2406.02069.
8. Tang, J. et al. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. ICML 2024. arXiv:2406.10774.
9. Lee, W. et al. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management. OSDI 2024. arXiv:2406.19707.
10. Liu, Z. et al. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. ICML 2024. arXiv:2402.02750.
11. Hooper, C. et al. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. NeurIPS 2024. arXiv:2401.18079.
12. Chang, C.-C. et al. Palu: Compressing KV-Cache with Low-Rank Projection. 2024. arXiv:2407.21118.
13. Sheng, Y. et al. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. ICML 2023. arXiv:2303.06865.
14. Alizadeh, K. et al. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. 2023. arXiv:2312.11514.
15. Shazeer, N. Fast Transformer Decoding: One Write-Head is All You Need. 2019. arXiv:1911.02150.
16. Ainslie, J. et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. 2023. arXiv:2305.13245.
17. DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. arXiv:2405.04434.
18. Dao, T. et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022. arXiv:2205.14135.

19. Xiao, G. et al. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. ICML 2023. arXiv:2211.10438.

20. Cavalcante, M. et al. Ara: A 1-GHz+ Scalable and Energy-Efficient RISC-V Vector Processor with Multiprecision Floating-Point Support in 22-nm FD-SOI. IEEE TVLSI, 2020.

21. Perotti, M. et al. Ara2: Exploring Single- and Multi-Core Vector Processing with an Efficient RVV 1.0 Compliant Open-Source Processor. 2023. arXiv:2311.07493.

22. Chen, T. et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. OSDI 2018. arXiv:1802.04799.

23. EvicPress (arXiv:2512.14946); Ye et al., KV-Compress (arXiv:2410.00161); PagedEviction (arXiv:2509.04377); DiffKV (arXiv:2412.03131); TensorRT-LLM KV-cache compression framework; CXL-SpecKV; HiKV (arXiv:2607.22389); Titanus (arXiv:2505.17787); Kelle (arXiv:2510.16040). Recent preprints; details to be re-verified against the latest versions before formal citation.

24. RISC-V International. RISC-V Vector Extension, version 1.0.

25. Verilator, Yosys and GTKWave open-source tools used for simulation, synthesis checking and waveform viewing.

Appendix A. Glossary

Table 39. *Glossary.*

Term	Meaning
KV cache	Stored keys and values of all previous tokens, per layer and head
Block	Fixed group of 16 consecutive tokens; unit of placement
Tier	Memory level (T0 SRAM, T1/T2 DRAM, T3 flash)
Codec	Storage format of a block (fp16, int8, kivi4, kivi2, optionally entropy-coded)
Heavy hitter	Token that accumulates a large share of attention
Sink	Initial tokens that attract attention irrespective of content
Ratchet	Rule that precision may only decrease
Hysteresis	Rule that suppresses a move unless it is clearly better
Recall (attention mass)	Share of the dense attention probability falling on tokens that were read
RVV	RISC-V Vector extension
VCD	Value change dump: waveform file written by simulators, viewable in GTKWave

Appendix B. Raw Experiment Tables

Mean ± standard deviation over seeds. Columns: resident / FP16, dropped tokens, mean relative L2 error, 95th-percentile error, attention-mass recall, modelled fetch ms per token, flash reads, migrations, re-encodes, overflow bytes.

Table 40. *E1 ablation (5 seeds, 1,536 tokens)*

setting	res/FP16	drop	err	err p95	recall	fetch ms	flash	migr	reenc	overflow B
v1_threshold_fp16	0.3667 ± 0.01019	972.8 ± 15.68	0.4789 ± 0.03459	1.288 ± 0.1304	0.7581 ± 0.01638	0.9229 ± 0.02585	1.261e+04 ± 374	744.8 ± 35.78	0 ± 0	0 ± 0
v2_planner_fp16	1 ± 0	0 ± 0	0.09209 ± 0.01912	0.3312 ± 0.07139	0.9389 ± 0.0117	1.041 ± 0.005997	1.436e+04 ± 86.17	628.6 ± 18.78	0 ± 0	2.58e+05 ± 1832
v2_planner_int8_kivi4	0.3711 ± 0.002651	0 ± 0	0.1224 ± 0.01366	0.3614 ± 0.06096	0.9389 ± 0.0117	0.1156 ± 0.002161	969.6 ± 32.11	698.6 ± 25.26	178.8 ± 1.6	0 ± 0
v2_planner_all_codecs	0.3101 ± 0.001326	0 ± 0	0.1662 ± 0.02025	0.5709 ± 0.1013	0.9389 ± 0.0117	0.1117 ± 0.002028	915 ± 30.09	644.4 ± 21.27	225.6 ± 1.625	0 ± 0
v2_all_codecs_with_drop	0.3101 ± 0.001326	0 ± 0	0.1662 ± 0.02025	0.5709 ± 0.1013	0.9389 ± 0.0117	0.1117 ± 0.002028	915 ± 30.09	644.4 ± 21.27	225.6 ± 1.625	0 ± 0

Table 41. *E2 context scaling: combined system, all codecs (3 seeds)*

setting	res/FP16	drop	err	err p95	recall	fetch ms	flash	migr	reenc	overflow B
512	0.5381 ± 0.006369	0 ± 0	0.01606 ± 0.004446	0.06127 ± 0.01477	0.9994 ± 0.0001219	0.03473 ± 8.165e-06	0 ± 0	37 ± 0.8165	36.67 ± 1.247	0 ± 0
768	0.4447 ± 0.008532	0 ± 0	0.04193 ± 0.004352	0.1447 ± 0.0109	0.9897 ± 0.0003442	0.04879 ± 0.0005987	42 ± 4.243	92.33 ± 2.055	74.67 ± 3.399	0 ± 0
1024	0.3758 ± 0.008014	0 ± 0	0.07125 ± 0.005684	0.2138 ± 0.01402	0.9757 ± 0.00282	0.07163 ± 0.001114	242 ± 10.8	216.7 ± 1.7	122.7 ± 4.784	0 ± 0
1536	0.3096 ± 0.001061	0 ± 0	0.1592 ± 0.02339	0.5361 ± 0.1069	0.9454 ± 0.01103	0.1112 ± 0.00247	907.7 ± 36.75	629.7 ± 14.52	226.3 ± 1.7	0 ± 0
2048	0.266 ± 0.003044	0 ± 0	0.2704 ± 0.03453	0.8487 ± 0.0891	0.9129 ± 0.01327	0.1518 ± 0.002232	2005 ± 44.97	1236 ± 4.028	343.3 ± 2.625	0 ± 0
3072	0.2412 ± 0.001112	0 ± 0	0.5148 ± 0.03828	1.348 ± 0.05351	0.8303 ± 0.01601	0.1783 ± 0.004526	3822 ± 135.8	3127 ± 39.6	550.7 ± 1.247	2.85e+04 ± 3786

Table 42. *E2 context scaling: original threshold policy (3 seeds)*

setting	res/FP16	drop	err	err p95	recall	fetch ms	flash	migr	reenc	overflow B
512	1 ± 0	0 ± 0	0.001343 ± 0.0002953	0.00823 ± 0.001455	0.9994 ± 0.0001219	0.4732 ± 0.03644	1929 ± 160.4	133.3 ± 4.643	0 ± 0	0 ± 0
768	0.6806 ± 0.04282	245.3 ± 32.88	0.0998 ± 0.04309	0.5857 ± 0.1558	0.9539 ± 0.01951	0.728 ± 0.02277	4737 ± 157.5	283.7 ± 4.19	0 ± 0	0 ± 0
1024	0.5156 ± 0.01274	496 ± 13.06	0.2407 ± 0.0561	1.062 ± 0.07522	0.8815 ± 0.02411	0.8461 ± 0.02636	7536 ± 248.6	453 ± 4.243	0 ± 0	0 ± 0
1536	0.3681 ± 0.009805	970.7 ± 15.08	0.5055 ± 0.01385	1.359 ± 0.1236	0.7502 ± 0.01583	0.9191 ± 0.02101	1.256e+04 ± 303.9	756.7 ± 17.59	0 ± 0	0 ± 0
2048	0.2865 ± 0.003724	1461 ± 7.542	0.6835 ± 0.0417	1.545 ± 0.01908	0.6567 ± 0.02954	0.9484 ± 0.02693	1.746e+04 ± 525.1	1108 ± 36.43	0 ± 0	0 ± 0
3072	0.1962 ± 0.009805	2469 ± 30.17	0.9853 ± 0.02037	1.98 ± 0.05004	0.5319 ± 0.0208	0.9456 ± 0.01344	2.634e+04 ± 397.4	1772 ± 35.44	0 ± 0	0 ± 0

Table 43. *E3 attention block budget*

setting	res/FP16	drop	err	err p95	recall	fetch ms	flash	migr	reenc	overflow B
4	0.2688 ± 0.001226	0 ± 0	1.101 ± 0.01643	2.532 ± 0.1898	0.5679 ± 0.0129	0.03566 ± 0.003463	366 ± 51.46	468 ± 17.96	261.3 ± 0.9428	0 ± 0
8	0.2722 ± 0.003412	0 ± 0	0.6631 ± 0.01471	1.759 ± 0.1425	0.7439 ± 0.01728	0.06222 ± 0.001582	633.3 ± 23.61	573.7 ± 21.01	258.3 ± 1.7	0 ± 0
12	0.2766 ± 0.001226	0 ± 0	0.4433 ± 0.027	1.284 ± 0.1678	0.8411 ± 0.01875	0.08708 ± 0.0009615	881 ± 14.31	643.7 ± 21.75	254.7 ± 0.4714	0 ± 0
16	0.2848 ± 0.002123	0 ± 0	0.3076 ± 0.02236	0.9817 ± 0.1632	0.8948 ± 0.01659	0.09848 ± 0.002588	933.7 ± 38.32	642.7 ± 21.51	246 ± 0.8165	0 ± 0
24	0.3096 ± 0.001061	0 ± 0	0.1592 ± 0.02339	0.5361 ± 0.1069	0.9454 ± 0.01103	0.1112 ± 0.00247	907.7 ± 36.75	629.7 ± 14.52	226.3 ± 1.7	0 ± 0
32	0.3369 ± 0.003677	0 ± 0	0.1011 ± 0.007664	0.2905 ± 0.05086	0.9662 ± 0.006626	0.1322 ± 0.004088	1036 ± 62.25	752.3 ± 48.79	206 ± 5.657	0 ± 0
48	0.376 ± 0.002123	0 ± 0	0.07004 ± 0.002308	0.1659 ± 0.01421	0.9864 ± 0.002735	0.278 ± 0.02397	2931 ± 354.2	808.3 ± 60.42	177 ± 5.099	0 ± 0

Table 44. *E4 quality weight lambda*

setting	res/FP16	drop	err	err p95	recall	fetch ms	flash	migr	reenc	overflow B
0.0	0.2432 ± 0	0 ± 0	0.5772 ± 0.01988	1.043 ± 0.07619	0.9454 ± 0.01103	0.09789 ± 0.002523	723.7 ± 37.56	572.7 ± 6.944	93 ± 0	0 ± 0
0.25	0.264 ± 0.001838	0 ± 0	0.2893 ± 0.01638	0.7877 ± 0.06544	0.9454 ± 0.01103	0.101 ± 0.001069	763 ± 16.08	610.3 ± 18.35	175.7 ± 1.7	0 ± 0
0.5	0.2835 ± 0.002123	0 ± 0	0.2003 ± 0.02808	0.6013 ± 0.1195	0.9454 ± 0.01103	0.1062 ± 0.002955	836.3 ± 43.76	631 ± 10.98	188.3 ± 2.494	0 ± 0
1.0	0.2948 ± 0.0006128	0 ± 0	0.1815 ± 0.03109	0.5715 ± 0.1199	0.9454 ± 0.01103	0.1079 ± 0.003259	859.7 ± 48.4	639.3 ± 16.94	213.7 ± 3.3	0 ± 0
2.0	0.3096 ± 0.001061	0 ± 0	0.1592 ± 0.02339	0.5361 ± 0.1069	0.9454 ± 0.01103	0.1112 ± 0.00247	907.7 ± 36.75	629.7 ± 14.52	226.3 ± 1.7	0 ± 0
4.0	0.3152 ± 0.002671	0 ± 0	0.1523 ± 0.02454	0.5262 ± 0.1153	0.9454 ± 0.01103	0.1973 ± 0.03053	2189 ± 455.1	567 ± 5.715	226.3 ± 2.867	0 ± 0
8.0	0.3252 ± 0.003184	0 ± 0	0.1382 ± 0.01213	0.4728 ± 0.07563	0.9454 ± 0.01103	0.2453 ± 0.003006	2912 ± 44.85	567.3 ± 24.28	222.7 ± 1.247	0 ± 0
16.0	0.3434 ± 0.003677	0 ± 0	0.1341 ± 0.01333	0.4557 ± 0.06814	0.9454 ± 0.01103	0.237 ± 0.004147	2792 ± 60.42	613.7 ± 20.29	214.7 ± 1.886	0 ± 0
32.0	0.3656 ± 0.006457	0 ± 0	0.127 ± 0.008716	0.4326 ± 0.05711	0.9454 ± 0.01103	0.2535 ± 0.006791	3036 ± 101.6	726 ± 52.71	205 ± 3.559	0 ± 0

Table 45. *E5 capacity factor, combined system*

setting	res/FP16	drop	err	err p95	recall	fetch ms	flash	migr	reenc	overflow B
0.5	0.2878 ± 0.008107	0 ± 0	0.2204 ± 0.0104	0.6296 ± 0.03874	0.9454 ± 0.01103	0.3011 ± 0.01844	3729 ± 272.7	863.7 ± 35.46	200.7 ± 5.437	3.311e+04 ± 3205
0.75	0.2935 ± 0.002671	0 ± 0	0.1801 ± 0.02752	0.5765 ± 0.1167	0.9454 ± 0.01103	0.1415 ± 0.005091	1360 ± 76.25	804.7 ± 40.14	229.7 ± 1.7	0 ± 0
1.0	0.3096 ± 0.001061	0 ± 0	0.1592 ± 0.02339	0.5361 ± 0.1069	0.9454 ± 0.01103	0.1112 ± 0.00247	907.7 ± 36.75	629.7 ± 14.52	226.3 ± 1.7	0 ± 0
1.5	0.3226 ± 0.004246	0 ± 0	0.1306 ± 0.009824	0.4271 ± 0.06095	0.9454 ± 0.01103	0.09922 ± 0.002136	728 ± 30.69	594.3 ± 9.672	221.3 ± 3.3	0 ± 0
2.0	0.33 ± 0.001621	0 ± 0	0.115 ± 0.01631	0.3613 ± 0.06399	0.9454 ± 0.01103	0.08823 ± 0.002131	566.3 ± 29.51	548.3 ± 15.54	217 ± 0.8165	0 ± 0
3.0	0.3239 ± 0.002808	0 ± 0	0.1518 ± 0.02296	0.5418 ± 0.09728	0.9454 ± 0.01103	0.04853 ± 0.0003229	3.333 ± 4.028	162 ± 6.532	225.7 ± 1.247	0 ± 0

Table 46. *E5 capacity factor, original policy*

setting	res/FP16	drop	err	err p95	recall	fetch ms	flash	migr	reenc	overflow B
0.5	0.2188 ± 0	1200 ± 0	0.8552 ± 0.03184	1.855 ± 0.1433	0.5875 ± 0.01813	0.7566 ± 0.002511	1.04e+04 ± 36.37	463.3 ± 30.44	0 ± 0	2048 ± 0
0.75	0.2986 ± 0.01302	1077 ± 19.96	0.6606 ± 0.01781	1.532 ± 0.2061	0.6683 ± 0.008979	1.001 ± 0.007869	1.376e+04 ± 117.6	577 ± 27.76	0 ± 0	0 ± 0
1.0	0.3681 ± 0.009805	970.7 ± 15.08	0.5055 ± 0.01385	1.359 ± 0.1236	0.7502 ± 0.01583	0.9191 ± 0.02101	1.256e+04 ± 303.9	756.7 ± 17.59	0 ± 0	0 ± 0
1.5	0.5208 ± 0.008492	736 ± 13.06	0.2803 ± 0.02885	0.9541 ± 0.142	0.8519 ± 0.01029	0.7331 ± 0.007656	9869 ± 110.8	853.3 ± 18.26	0 ± 0	0 ± 0
2.0	0.625 ± 0	576 ± 0	0.1709 ± 0.01651	0.6773 ± 0.08573	0.9041 ± 0.008442	0.6124 ± 0.02139	8123 ± 309.4	928.7 ± 11.09	0 ± 0	0 ± 0
3.0	0.8299 ± 0.009852	261.3 ± 15.08	0.08563 ± 0.01555	0.3117 ± 0.06408	0.9438 ± 0.009977	0.5019 ± 0.02393	6524 ± 346.1	932.3 ± 5.437	0 ± 0	0 ± 0

Appendix C. Core Source Listings

The complete project is supplied as a separate archive; the listings below are the parts the report describes most closely.

hw/rtl/uamm_sweep_scorer.v

```
// UAMM-2 sweep scorer: hardware "heat classifier" for the block table.
//
// Replaces the single-shot uamm_controller of UAMM-1:
//   * no divider: weights are integers summing to 256 (154/64/38 ~= 60%/25%/15%), score = sum >> 8
//   * block metadata lives in a small SRAM; one sweep streams every block through a 2-stage pipeline
//     (1 block/cycle) so thousands of blocks are scored per planner interval
//   * `protected` (sink / recent window) is honoured in hardware, matching the Python planner
//
// Metadata word (25 bits): {protected, frequency[7:0], recency[7:0], importance[7:0]}
// Output tier code: 2'b00 hot (T0/T1), 2'b01 warm (T2), 2'b10 cold (T3).
// Synthesizable Verilog-2001.
module uamm_sweep_scorer #(
    parameter NBLK = 64,
    parameter AW   = 6            // AW = log2(NBLK)
) (
    input  wire          clk,
    input  wire          rst_n,
    // metadata write port
    input  wire          we,
    input  wire [AW-1:0] waddr,
    input  wire [24:0]   wdata,
    // sweep control
    input  wire          start,       // ignored while busy
    input  wire [7:0]    pressure,    // memory pressure, latched at start
    output reg           busy,
    output reg           done,        // pulses with the last out_valid of a sweep
    // result stream
    output reg           out_valid,
    output reg  [AW-1:0] out_idx,
    output reg  [7:0]    out_score,
    output reg  [1:0]    out_tier,
    output reg           out_evict
);
    localparam [7:0] W_IMP   = 8'd154;
    localparam [7:0] W_REC   = 8'd64;
    localparam [7:0] W_FRQ   = 8'd38;
    localparam [7:0] TH_HOT  = 8'd174;
    localparam [7:0] TH_WARM = 8'd89;
    localparam [7:0] EVICT_P = 8'd243;

    reg [24:0] mem [0:NBLK-1];
    always @(posedge clk) if (we) mem[waddr] <= wdata;

    reg [AW:0]    rd_cnt;
    reg [7:0]     pres;
    reg           s0_v;
    reg [AW-1:0]  s0_idx;
    reg [24:0]    s0_data;

    // synchronous SRAM read, address = issue counter
    always @(posedge clk) s0_data <= mem[rd_cnt[AW-1:0]];

    wire [7:0]  imp   = s0_data[7:0];
    wire [7:0]  rec   = s0_data[15:8];
    wire [7:0]  frq   = s0_data[23:16];
    wire        prot  = s0_data[24];
    /* verilator lint_off UNUSEDSIGNAL */
    wire [15:0] wsum  = (imp * W_IMP) + (rec * W_REC) + (frq * W_FRQ);  // <= 255*256
    /* verilator lint_on UNUSEDSIGNAL */
    wire [7:0]  score = wsum[15:8];   // wsum[7:0] intentionally dropped (>> 8)
    wire [1:0]  tier  = prot ? 2'b00 :
                        (score >= TH_HOT)  ? 2'b00 :
                        (score >= TH_WARM) ? 2'b01 : 2'b10;
    wire        evict = (!prot) && (pres >= EVICT_P) && (score < TH_WARM);
    localparam [31:0] LAST_IDX = NBLK - 1;
    wire        last  = s0_v && ({{(32-AW){1'b0}}, s0_idx} == LAST_IDX);
```

```
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            busy <= 1'b0;  done <= 1'b0;  rd_cnt <= {(AW+1){1'b0}};  pres <= 8'd0;
            s0_v <= 1'b0;  s0_idx <= {AW{1'b0}};
            out_valid <= 1'b0;  out_idx <= {AW{1'b0}};  out_score <= 8'd0;
            out_tier <= 2'b10;  out_evict <= 1'b0;
        end else begin
            // ---- stage 1: score the block read last cycle
            out_valid <= s0_v;
            done      <= last;
            if (s0_v) begin
                out_idx   <= s0_idx;
                out_score <= score;
                out_tier  <= tier;
                out_evict <= evict;
            end
            // ---- stage 0: issue reads
            if (start && !busy) begin
                busy <= 1'b1;  rd_cnt <= {(AW+1){1'b0}};  pres <= pressure;  s0_v <= 1'b0;
            end else if (busy && (rd_cnt < NBLK)) begin
                s0_v   <= 1'b1;
                s0_idx <= rd_cnt[AW-1:0];
                rd_cnt <= rd_cnt + 1'b1;
            end else begin
                s0_v <= 1'b0;
            end
            if (last) busy <= 1'b0;
        end
    end
endmodule
```

hw/rtl/uamm_kv_codec.v

```
// UAMM-2 KV codec datapath: 8-bit -> 4-bit group quantizer and the inverse dequantizer.
// One beat = LANES values. Intended to sit on the AXI-stream path between DRAM/flash and Ara's
// vector load/store unit so (de)quantization happens in the load path, not in vector instructions.
//
// Per-group parameters are computed by software/firmware from the group's min/max (range = max-min):
//   zp       = min                       (8-bit)
//   inv_q12  = round(15*4096 / range)    (Q4.12, 16-bit; 0 when range == 0)
//   step_q8  = round(range*256 / 15)     (Q8.8,  16-bit; 0 when range == 0)
// Quantize:   q  = clamp( ((x - zp) * inv_q12 + 2048) >> 12 , 0, 15 )   (x < zp clamps to q = 0)
// Dequantize: x' = min( 255, zp + ((q * step_q8 + 128) >> 8) )
// Nibble i of the packed word is lane i (bits [4i+3:4i]) - the same packing as uamm2/codecs.py.
// Synthesizable Verilog-2001, one pipeline register.

module uamm_kv_quant4 #(parameter LANES = 16) (
    input  wire                 clk,
    input  wire                 rst_n,
    input  wire                 in_valid,
    input  wire [LANES*8-1:0]   x_flat,
    input  wire [7:0]           zp,
    input  wire [15:0]          inv_q12,
    output reg                  out_valid,
    output reg  [LANES*4-1:0]   q_flat
);
    reg [LANES*4-1:0] q_next;
    reg [7:0]  xi;
    reg [7:0]  dx;
    reg [31:0] prod;
    reg [31:0] r;
    integer i;
    always @* begin
        q_next = {(LANES*4){1'b0}};
        for (i = 0; i < LANES; i = i + 1) begin
            xi   = x_flat[i*8 +: 8];
            dx   = (xi >= zp) ? (xi - zp) : 8'd0;
            prod = dx * inv_q12;
            r    = (prod + 32'd2048) >> 12;
            q_next[i*4 +: 4] = (r > 32'd15) ? 4'd15 : r[3:0];
        end
    end
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
```

```
            out_valid <= 1'b0;
            q_flat    <= {(LANES*4){1'b0}};
        end else begin
            out_valid <= in_valid;
            if (in_valid) q_flat <= q_next;
        end
    end
endmodule

module uamm_kv_dequant4 #(parameter LANES = 16) (
    input  wire                 clk,
    input  wire                 rst_n,
    input  wire                 in_valid,
    input  wire [LANES*4-1:0]   q_flat,
    input  wire [7:0]           zp,
    input  wire [15:0]          step_q8,
    output reg                  out_valid,
    output reg  [LANES*8-1:0]   x_flat
);
    reg [LANES*8-1:0] x_next;
    reg [3:0]  qi;
    reg [31:0] prod;
    reg [31:0] sum;
    integer i;
    always @* begin
        x_next = {(LANES*8){1'b0}};
        for (i = 0; i < LANES; i = i + 1) begin
            qi   = q_flat[i*4 +: 4];
            prod = qi * step_q8;
            sum  = {24'd0, zp} + ((prod + 32'd128) >> 8);
            x_next[i*8 +: 8] = (sum > 32'd255) ? 8'd255 : sum[7:0];
        end
    end
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            out_valid <= 1'b0;
            x_flat    <= {(LANES*8){1'b0}};
        end else begin
            out_valid <= in_valid;
            if (in_valid) x_flat <= x_next;
        end
    end
endmodule
```

hw/rtl/uamm_block_table.v

```
// UAMM-2 block-table translation unit (logical block -> tier / codec / physical slot).
// Inspired by the address-translation stage of FPGA KV-cache engines; deliberately minimal:
// direct-mapped table, 1-cycle synchronous lookup, no TLB.
//   entry = {tier[1:0], codec[1:0], phys[PW-1:0]} plus a separate valid bit (reset to 0).
// A read issued in the same cycle as a write to the same address returns the OLD entry.
// Synthesizable Verilog-2001.
module uamm_block_table #(
    parameter AW = 8,
    parameter PW = 12
) (
    input  wire          clk,
    input  wire          rst_n,
    // update port
    input  wire          we,
    input  wire [AW-1:0] waddr,
    input  wire          w_valid,
    input  wire [1:0]    w_tier,
    input  wire [1:0]    w_codec,
    input  wire [PW-1:0] w_phys,
    // lookup port
    input  wire          rd_en,
    input  wire [AW-1:0] raddr,
    output reg           rd_valid,
    output reg           hit,
    output reg  [1:0]    tier,
    output reg  [1:0]    codec,
    output reg  [PW-1:0] phys
);
    localparam DEPTH = (1 << AW);
```

```
    reg [PW+3:0]    mem   [0:DEPTH-1];
    reg [DEPTH-1:0] vbits;

    always @(posedge clk) if (we) mem[waddr] <= {w_tier, w_codec, w_phys};

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            vbits    <= {DEPTH{1'b0}};
            rd_valid <= 1'b0;
            hit      <= 1'b0;
            tier     <= 2'd0;
            codec    <= 2'd0;
            phys     <= {PW{1'b0}};
        end else begin
            if (we) vbits[waddr] <= w_valid;
            rd_valid <= rd_en;
            if (rd_en) begin
                hit   <= vbits[raddr];
                tier  <= mem[raddr][PW+3:PW+2];
                codec <= mem[raddr][PW+1:PW];
                phys  <= mem[raddr][PW-1:0];
            end
        end
    end
endmodule
```

ara/kv_kernels.c

```
#include "kv_kernels.h"

#if defined(__riscv_vector)
#include <riscv_vector.h>
#endif

float kv_qk_prepare(const float *q, const float *scale, const float *zero, int d,
                    float *qs_even, float *qs_odd) {
    float bias = 0.0f;
    for (int c = 0; c < d; c++) {
        bias += q[c] * zero[c];
        if ((c & 1) == 0) qs_even[c >> 1] = q[c] * scale[c];
        else              qs_odd[c >> 1]  = q[c] * scale[c];
    }
    return bias;
}

void kv_qk_dot_kivi4_scalar(const float *qs_even, const float *qs_odd, float bias,
                            const uint8_t *packed, int T, int d, float *out) {
    const int half = d / 2;
    for (int t = 0; t < T; t++) {
        const uint8_t *row = packed + (long)t * half;
        float acc = 0.0f;
        for (int j = 0; j < half; j++) {
            acc += (float)(row[j] & 0x0F) * qs_even[j];
            acc += (float)(row[j] >> 4)   * qs_odd[j];
        }
        out[t] = acc + bias;
    }
}

#if defined(__riscv_vector)
void kv_qk_dot_kivi4_rvv(const float *qs_even, const float *qs_odd, float bias,
                         const uint8_t *packed, int T, int d, float *out) {
    const int half = d / 2;
    const size_t vlmax = __riscv_vsetvlmax_e32m1();
    for (int t = 0; t < T; t++) {
        const uint8_t *row = packed + (long)t * half;
        vfloat32m1_t acc = __riscv_vfmv_v_f_f32m1(0.0f, vlmax);
        int j = 0;
        while (j < half) {
            size_t vl = __riscv_vsetvl_e32m1((size_t)(half - j));
            vuint8mf4_t b   = __riscv_vle8_v_u8mf4(row + j, vl);
            vuint8mf4_t lo8 = __riscv_vand_vx_u8mf4(b, 0x0F, vl);
            vuint8mf4_t hi8 = __riscv_vsrl_vx_u8mf4(b, 4, vl);
            vfloat32m1_t lo = __riscv_vfcvt_f_xu_v_f32m1(__riscv_vzext_vf4_u32m1(lo8, vl), vl);
            vfloat32m1_t hi = __riscv_vfcvt_f_xu_v_f32m1(__riscv_vzext_vf4_u32m1(hi8, vl), vl);
```

```
            vfloat32m1_t qe = __riscv_vle32_v_f32m1(qs_even + j, vl);
            vfloat32m1_t qo = __riscv_vle32_v_f32m1(qs_odd + j, vl);
            acc = __riscv_vfmacc_vv_f32m1_tu(acc, lo, qe, vl);   /* _tu: keep tail lanes of acc */
            acc = __riscv_vfmacc_vv_f32m1_tu(acc, hi, qo, vl);
            j += (int)vl;
        }
        vfloat32m1_t zero = __riscv_vfmv_s_f_f32m1(0.0f, 1);
        vfloat32m1_t sum  = __riscv_vfredusum_vs_f32m1_f32m1(acc, zero, vlmax);
        out[t] = __riscv_vfmv_f_s_f32m1_f32(sum) + bias;
    }
}
#endif

void kv_qk_dot_kivi4(const float *qs_even, const float *qs_odd, float bias,
                     const uint8_t *packed, int T, int d, float *out) {
#if defined(__riscv_vector)
    kv_qk_dot_kivi4_rvv(qs_even, qs_odd, bias, packed, T, d, out);
#else
    kv_qk_dot_kivi4_scalar(qs_even, qs_odd, bias, packed, T, d, out);
#endif
}
```

uamm2/codecs.py

```
"""KIVI-style KV block codecs (real quantization, real byte payloads).

Per block of T tokens with d channels:
  * keys   : asymmetric, quantized PER-CHANNEL (min/max over the block's tokens)
  * values : asymmetric, quantized PER-TOKEN in channel groups of `v_group`
This follows the asymmetric scheme described in the KIVI paper. Scales/zeros are
stored as fp16. Codec names:
  fp16 | int8 | kivi4 | kivi2   and entropy-coded variants  int8z | kivi4z | kivi2z
The "z" variants run zlib over the serialized blob (used on the flash tier).
Precision is a one-way ratchet: re-encoding a lossy block can only lower precision.
"""
import zlib
from dataclasses import dataclass
from typing import Dict, Tuple

import numpy as np

CODEC_RANK = {"fp16": 0, "int8": 1, "kivi4": 2, "kivi2": 3}
FAMILY_BITS = {"fp16": 16, "int8": 8, "kivi4": 4, "kivi2": 2}


def parse_codec(name: str) -> Tuple[str, bool]:
    """Return (family, entropy_coded)."""
    entropy = name.endswith("z")
    family = name[:-1] if entropy else name
    if family not in FAMILY_BITS or (entropy and family == "fp16"):
        raise ValueError(f"unknown codec {name!r}")
    return family, entropy


def codec_rank(name: str) -> int:
    return CODEC_RANK[parse_codec(name)[0]]


@dataclass
class Payload:
    codec: str
    n_tokens: int
    d: int
    v_group: int
    blob: bytes

    @property
    def nbytes(self) -> int:
        return len(self.blob)


# ---------------------------------------------------------------- primitives
def _asym_quant(x: np.ndarray, bits: int, axis: int):
    levels = (1 << bits) - 1
    mn = x.min(axis=axis, keepdims=True)
```

```
    mx = x.max(axis=axis, keepdims=True)
    zero = mn.astype(np.float16)
    scale = np.maximum((mx - mn) / levels, 2.0 ** -10).astype(np.float16)
    q = np.clip(np.rint((x - zero.astype(np.float32)) / scale.astype(np.float32)), 0, levels)
    return q.astype(np.uint8), scale, zero


def _asym_dequant(q: np.ndarray, scale: np.ndarray, zero: np.ndarray) -> np.ndarray:
    return q.astype(np.float32) * scale.astype(np.float32) + zero.astype(np.float32)


def pack_bits(q: np.ndarray, bits: int) -> bytes:
    """Pack uint8 values (< 2**bits) little-endian within each byte (element 0 = low bits)."""
    flat = q.reshape(-1).astype(np.uint8)
    if bits == 8:
        return flat.tobytes()
    per = 8 // bits
    pad = (-len(flat)) % per
    if pad:
        flat = np.concatenate([flat, np.zeros(pad, np.uint8)])
    m = flat.reshape(-1, per)
    out = np.zeros(m.shape[0], np.uint8)
    for i in range(per):
        out |= (m[:, i] << (bits * i)).astype(np.uint8)
    return out.tobytes()


def unpack_bits(buf: bytes, bits: int, count: int) -> np.ndarray:
    arr = np.frombuffer(buf, np.uint8)
    if bits == 8:
        return arr[:count].copy()
    per = 8 // bits
    mask = (1 << bits) - 1
    out = np.empty((len(arr), per), np.uint8)
    for i in range(per):
        out[:, i] = (arr >> (bits * i)) & mask
    return out.reshape(-1)[:count]


def _packed_len(count: int, bits: int) -> int:
    return count if bits == 8 else -(-count * bits // 8)


# ---------------------------------------------------------------- encode
def encode(k: np.ndarray, v: np.ndarray, codec: str, v_group: int) -> Payload:
    t, d = k.shape
    if v.shape != (t, d):
        raise ValueError("k and v must have the same shape (T, d)")
    if d % v_group:
        raise ValueError("d must be divisible by v_group")
    family, entropy = parse_codec(codec)
    if family == "fp16":
        blob = k.astype("<f2").tobytes() + v.astype("<f2").tobytes()
    else:
        bits = FAMILY_BITS[family]
        qk, sk, zk = _asym_quant(k.astype(np.float32), bits, axis=0)              # per-channel
        v3 = v.astype(np.float32).reshape(t, d // v_group, v_group)
        qv, sv, zv = _asym_quant(v3, bits, axis=2)                                 # per-token
        blob = (pack_bits(qk, bits) + pack_bits(qv, bits) +
                sk.tobytes() + zk.tobytes() + sv.tobytes() + zv.tobytes())
    if entropy:
        blob = zlib.compress(blob, 6)
    return Payload(codec, t, d, v_group, blob)


# ---------------------------------------------------------------- decode
def decode(p: Payload) -> Tuple[np.ndarray, np.ndarray]:
    family, entropy = parse_codec(p.codec)
    blob = zlib.decompress(p.blob) if entropy else p.blob
    t, d, g = p.n_tokens, p.d, p.v_group
    if family == "fp16":
        n = t * d * 2
        k = np.frombuffer(blob[:n], "<f2").astype(np.float32).reshape(t, d)
        v = np.frombuffer(blob[n:2 * n], "<f2").astype(np.float32).reshape(t, d)
        return k, v
    bits = FAMILY_BITS[family]
```

```
    nk = _packed_len(t * d, bits)
    off = 0
    qk = unpack_bits(blob[off:off + nk], bits, t * d).reshape(t, d); off += nk
    qv = unpack_bits(blob[off:off + nk], bits, t * d).reshape(t, d // g, g); off += nk
    ng = t * (d // g)
    sk = np.frombuffer(blob[off:off + 2 * d], "<f2").reshape(1, d); off += 2 * d
    zk = np.frombuffer(blob[off:off + 2 * d], "<f2").reshape(1, d); off += 2 * d
    sv = np.frombuffer(blob[off:off + 2 * ng], "<f2").reshape(t, d // g, 1); off += 2 * ng
    zv = np.frombuffer(blob[off:off + 2 * ng], "<f2").reshape(t, d // g, 1); off += 2 * ng
    k = _asym_dequant(qk, sk, zk)
    v = _asym_dequant(qv, sv, zv).reshape(t, d)
    return k, v


def rel_error(k, v, k2, v2) -> float:
    num = float(((k - k2) ** 2).sum() + ((v - v2) ** 2).sum())
    den = float((k ** 2).sum() + (v ** 2).sum()) + 1e-12
    return num / den


def calibrate(k: np.ndarray, v: np.ndarray, codec_names, v_group: int):
    """Encode `k,v` with every codec once; return (err_table, size_table).

    Done when a block is sealed (originals available), like a background calibration pass.
    """
    errs: Dict[str, float] = {}
    sizes: Dict[str, int] = {}
    for name in codec_names:
        p = encode(k, v, name, v_group)
        k2, v2 = decode(p)
        errs[name] = rel_error(k, v, k2, v2)
        sizes[name] = p.nbytes
    return errs, sizes


def pertoken_key_roundtrip(k: np.ndarray, bits: int) -> np.ndarray:
    """Per-TOKEN key quantization (the alternative KIVI argues against); used in tests."""
    q, s, z = _asym_quant(k.astype(np.float32), bits, axis=1)
    return _asym_dequant(q, s, z)


def perchannel_key_roundtrip(k: np.ndarray, bits: int) -> np.ndarray:
    q, s, z = _asym_quant(k.astype(np.float32), bits, axis=0)
    return _asym_dequant(q, s, z)
```

uamm2/planner.py

```
"""Placement planners.

UtilityPlanner (UAMM-2): chooses a (tier, codec) PAIR per block by minimizing expected cost

    cost(b,t,c) = p_access(b) * [read_ms(t,c,bytes) + decode_ms(c,bytes)]     (latency if fetched)
                + lambda * heat(b) * rel_err(b,c)                              (quality risk)
                + capacity_price * (bytes / cap_t) * (1 + 4 * fill_t)          (congestion)

greedily in descending "heat" order under per-tier byte capacities. Hysteresis (minimum residency and
a relative improvement margin) stops promote/demote thrashing. Precision is a one-way ratchet
(see codecs.py), so a block can only move to a codec of equal or lower precision.
This is a greedy heuristic, not an optimal knapsack solution.

ThresholdPlanner reproduces the original UAMM-1 policy (0.60/0.25/0.15 score, fixed thresholds,
fp16 only, cascade demotion, drop when nothing fits) for ablation.
"""
from typing import Dict, List, Tuple

from .codecs import CODEC_RANK, codec_rank, parse_codec
from .config import Config
from .paged_cache import KVBlock, all_codec_names
from .tiers import DROP, FLASH, read_cost_ms

Plan = Dict[int, Tuple[int, str]]   # block_id -> (tier, codec) ; tier == DROP means evict


def codec_allowed(tier: int, codec: str) -> bool:
    family, entropy = parse_codec(codec)
```

```
    if entropy and tier != FLASH:
        return False
    if tier == 2 and CODEC_RANK[family] < CODEC_RANK["kivi4"]:
        return False
    return True


def _fixed_usage(cfg: Config, blocks: List[KVBlock]) -> List[int]:
    used = [0] * len(cfg.tiers)
    for b in blocks:
        if not b.dropped and (b.protected or not b.sealed):
            used[b.tier] += b.nbytes(cfg.head_dim)
    return used


class UtilityPlanner:
    def __init__(self, cfg: Config):
        self.cfg = cfg
        self.cands = [(t, c) for t in range(len(cfg.tiers)) for c in all_codec_names(cfg)
                      if codec_allowed(t, c)]

    @staticmethod
    def heat(b: KVBlock) -> float:
        return 0.5 * b.hit_ema + 0.5 * b.importance

    def _cost(self, b, t, c, nbytes, used, cap) -> float:
        cfg = self.cfg
        p = max(b.hit_ema, 0.01)
        fill = used[t] / max(cap[t], 1.0)
        return (p * read_cost_ms(cfg, t, c, nbytes)
                + cfg.lambda_quality * self.heat(b) * b.err_table[c]
                + cfg.capacity_price * (nbytes / cfg.tiers[t].capacity_bytes) * (1.0 + 4.0 * fill))

    def plan(self, blocks: List[KVBlock], step: int) -> Plan:
        cfg = self.cfg
        cap = [s.capacity_bytes * (1.0 - cfg.capacity_headroom) for s in cfg.tiers]
        used = _fixed_usage(cfg, blocks)
        movable = [b for b in blocks if b.sealed and not b.protected and not b.dropped]
        movable.sort(key=lambda b: -self.heat(b))
        plan: Plan = {}
        for b in movable:
            best, best_cost, cur, cur_cost = None, float("inf"), None, float("inf")
            for t, c in self.cands:
                if codec_rank(c) < b.rank:
                    continue
                nbytes = b.payload.nbytes if c == b.codec else b.size_table[c]
                if used[t] + nbytes > cap[t]:
                    continue
                cost = self._cost(b, t, c, nbytes, used, cap)
                if cost < best_cost:
                    best, best_cost = (t, c, nbytes), cost
                if (t, c) == (b.tier, b.codec):
                    cur, cur_cost = (t, c, nbytes), cost
            choice = best
            if cur is not None and best is not None:
                too_soon = (step - b.last_move_step) < cfg.min_residency_steps
                close = cur_cost <= best_cost + cfg.hysteresis_margin * max(best_cost, 1e-4)
                if too_soon or close:
                    choice = cur
            if choice is None:
                if cfg.allow_drop and b.importance < cfg.drop_importance:
                    plan[b.block_id] = (DROP, b.codec)
                else:      # nothing fits: stay put (counted as overflow by the simulator)
                    used[b.tier] += b.nbytes(cfg.head_dim)
                continue
            t, c, nbytes = choice
            used[t] += nbytes
            plan[b.block_id] = (t, c)
        return plan


class ThresholdPlanner:
    """Original UAMM-1 policy (fp16 only)."""

    def __init__(self, cfg: Config):
        self.cfg = cfg
```

```
    def plan(self, blocks: List[KVBlock], step: int) -> Plan:
        cfg = self.cfg
        cap = [s.capacity_bytes for s in cfg.tiers]
        used = _fixed_usage(cfg, blocks)
        movable = [b for b in blocks if b.sealed and not b.protected and not b.dropped]
        if not movable:
            return {}
        max_age = max(1, max(step - b.last_access for b in movable))
        max_cnt = max(1, max(b.access_count for b in movable))

        def score(b):
            rec = 1.0 - (step - b.last_access) / max_age
            return cfg.alpha * b.importance + cfg.beta * rec + cfg.gamma * b.access_count / max_cnt

        plan: Plan = {}
        for b in sorted(movable, key=lambda x: -score(x)):
            s = score(b)
            target = 1 if s >= cfg.t1_threshold else (2 if s >= cfg.t2_threshold else 3)
            nbytes = b.payload.nbytes
            placed = False
            for t in range(target, len(cfg.tiers)):
                if used[t] + nbytes <= cap[t]:
                    used[t] += nbytes
                    plan[b.block_id] = (t, b.codec)
                    placed = True
                    break
            if not placed:
                plan[b.block_id] = (DROP, b.codec)    # UAMM-1 behaviour: data loss when full
        return plan


def make_planner(cfg: Config):
    return UtilityPlanner(cfg) if cfg.policy == "utility" else ThresholdPlanner(cfg)
```

uamm2/attention.py

```
"""Block-sparse attention over the tiered cache + dense FP32 oracle for quality measurement."""
from dataclasses import dataclass
from typing import List

import numpy as np

from .paged_cache import KVBlock, PagedKVCache


def softmax(x: np.ndarray) -> np.ndarray:
    z = x - x.max()
    e = np.exp(z)
    return e / e.sum()


@dataclass
class AttnResult:
    out: np.ndarray
    token_idx: np.ndarray
    weights: np.ndarray
    read_blocks: List[KVBlock]


def select_blocks(q: np.ndarray, candidates: List[KVBlock], budget: int) -> List[KVBlock]:
    """Query-aware selection using per-block key min/max bounds.

    bound(b) = sum_c max(q_c*kmin_c, q_c*kmax_c) is an upper bound on q.k over the block, so blocks
    that cannot contain a high-scoring key are skipped without fetching (or decoding) them.
    """
    if budget >= len(candidates):
        return list(candidates)
    kmin = np.stack([b.kmin for b in candidates])
    kmax = np.stack([b.kmax for b in candidates])
    bound = np.maximum(q * kmin, q * kmax).sum(axis=1)
    top = np.argsort(-bound, kind="stable")[:budget]
    return [candidates[i] for i in sorted(top)]
```

```
def sparse_attention(q: np.ndarray, cache: PagedKVCache, read_blocks: List[KVBlock]) -> AttnResult:
    d = q.shape[0]
    ks, vs, idx = [], [], []
    for b in read_blocks:
        k, v = cache.get_kv(b)
        ks.append(k)
        vs.append(v)
        idx.append(np.arange(b.start, b.start + b.n_tokens))
    K, V, tok = np.concatenate(ks), np.concatenate(vs), np.concatenate(idx)
    w = softmax(K @ q / np.sqrt(d))
    return AttnResult(w @ V, tok, w, read_blocks)


def dense_oracle(q: np.ndarray, K: np.ndarray, V: np.ndarray):
    w = softmax(K @ q / np.sqrt(q.shape[0]))
    return w @ V, w
```

uamm2/h2o.py

```
"""H2O-style heavy-hitter importance, fed by attention weights the engine already computed.

Token score = decayed running sum of attention weight received. Only tokens that were actually
attended (their block was fetched) receive weight; everything decays each step. Block importance
aggregates token scores (mean|max|sum) and is normalized by the 95th percentile of block scores,
so the (huge) attention-sink score does not squash every other block toward zero.

Cost: one vectorized decay over N tokens per step (O(N) in numpy, not a Python loop).
Note: this signal drives PLACEMENT (lossless tiering). Permanent token dropping, as in the original
H2O paper, is a separate opt-in (Config.allow_drop).
"""
from typing import List

import numpy as np

from .config import Config
from .paged_cache import KVBlock


class H2OImportance:
    def __init__(self, cfg: Config, capacity: int = 4096):
        self.cfg = cfg
        self.tok = np.zeros(capacity, np.float64)

    def _ensure(self, n: int) -> None:
        if n > len(self.tok):
            self.tok = np.concatenate([self.tok, np.zeros(max(n, 2 * len(self.tok)) - len(self.tok))])

    def update(self, token_idx: np.ndarray, weights: np.ndarray, n_tokens: int) -> None:
        self._ensure(n_tokens)
        self.tok[:n_tokens] *= self.cfg.h2o_decay
        np.add.at(self.tok, token_idx, np.maximum(weights, 0.0))

    def refresh(self, blocks: List[KVBlock]) -> None:
        if not blocks:
            return
        bs = self.cfg.block_size
        n = blocks[-1].end
        nb = len(blocks)
        grid = np.zeros(nb * bs)
        grid[:n] = self.tok[:n]
        grid = grid.reshape(nb, bs)
        counts = np.array([b.n_tokens for b in blocks], dtype=np.float64)
        agg_mode = self.cfg.h2o_aggregation
        if agg_mode == "sum":
            agg = grid.sum(axis=1)
        elif agg_mode == "max":
            agg = grid.max(axis=1)
        else:
            agg = grid.sum(axis=1) / np.maximum(counts, 1)
        scale = float(np.percentile(agg, 95)) + 1e-12
        norm = np.clip(agg / scale, 0.0, 1.0)
        for b, x in zip(blocks, norm):
            b.importance = float(x)
```

uamm2/paged_cache.py

```
"""Paged KV cache holding REAL tensors (as encoded payloads), not just metadata."""
import zlib
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple

import numpy as np

from .codecs import Payload, calibrate, codec_rank, decode, encode, parse_codec
from .config import Config


def all_codec_names(cfg: Config) -> List[str]:
    names: List[str] = []
    for fam in cfg.codecs:
        names.append(fam)
        if cfg.use_entropy_coding and fam != "fp16":
            names.append(fam + "z")
    return names


@dataclass
class KVBlock:
    block_id: int
    start: int
    n_tokens: int = 0
    sealed: bool = False
    k_stage: Optional[List[np.ndarray]] = field(default_factory=list)   # open block only
    v_stage: Optional[List[np.ndarray]] = field(default_factory=list)
    payload: Optional[Payload] = None
    tier: int = 0
    codec: str = "fp16"
    kmin: Optional[np.ndarray] = None       # key summaries, kept resident (tiny)
    kmax: Optional[np.ndarray] = None
    err_table: Dict[str, float] = field(default_factory=dict)
    size_table: Dict[str, int] = field(default_factory=dict)
    importance: float = 0.0                 # H2O-derived, normalized to [0,1]
    hit_ema: float = 1.0                    # how often attention actually fetched this block
    last_access: int = 0
    access_count: int = 0
    last_move_step: int = 0
    protected: bool = False
    dropped: bool = False

    @property
    def end(self) -> int:
        return self.start + self.n_tokens

    @property
    def rank(self) -> int:
        return codec_rank(self.codec)

    def nbytes(self, d: int) -> int:
        if self.dropped:
            return 0
        if self.payload is not None:
            return self.payload.nbytes
        return self.n_tokens * d * 2 * 2     # open block staged as fp16-equivalent


class PagedKVCache:
    def __init__(self, cfg: Config):
        self.cfg = cfg
        self.blocks: List[KVBlock] = []
        self._codec_names = all_codec_names(cfg)

    @property
    def n_tokens(self) -> int:
        return sum(b.n_tokens for b in self.blocks)

    def append_token(self, k: np.ndarray, v: np.ndarray) -> Tuple[KVBlock, bool]:
        """Append one token. Returns (block, sealed_now)."""
        cfg = self.cfg
        if not self.blocks or self.blocks[-1].sealed:
            self.blocks.append(KVBlock(block_id=len(self.blocks), start=self.n_tokens))
```

```
        b = self.blocks[-1]
        b.k_stage.append(np.asarray(k, np.float32))
        b.v_stage.append(np.asarray(v, np.float32))
        b.n_tokens += 1
        sealed = False
        if b.n_tokens == cfg.block_size:
            self._seal(b)
            sealed = True
        return b, sealed

    def _seal(self, b: KVBlock) -> None:
        cfg = self.cfg
        k = np.stack(b.k_stage).astype(np.float32)
        v = np.stack(b.v_stage).astype(np.float32)
        payload = encode(k, v, "fp16", cfg.v_group)
        k16, v16 = decode(payload)           # fp16-rounded originals = ground truth from here on
        b.kmin, b.kmax = k16.min(axis=0), k16.max(axis=0)
        b.err_table, b.size_table = calibrate(k16, v16, self._codec_names, cfg.v_group)
        b.payload, b.sealed = payload, True
        b.k_stage = b.v_stage = None
        b.tier, b.codec = 0, "fp16"

    def get_kv(self, b: KVBlock) -> Tuple[np.ndarray, np.ndarray]:
        if b.dropped:
            raise ValueError("block was dropped")
        if not b.sealed:
            return np.stack(b.k_stage), np.stack(b.v_stage)
        return decode(b.payload)

    def reencode(self, b: KVBlock, codec: str) -> None:
        """Change codec. Decodes the current (possibly lossy) payload first, so error compounds."""
        new_family, new_entropy = parse_codec(codec)
        old_family, old_entropy = parse_codec(b.codec)
        if new_family == old_family:
            # same precision: only the entropy wrapper changes -> lossless blob transform
            raw = zlib.decompress(b.payload.blob) if old_entropy else b.payload.blob
            blob = zlib.compress(raw, 6) if new_entropy else raw
            b.payload = Payload(codec, b.payload.n_tokens, b.payload.d, b.payload.v_group, blob)
        else:
            k, v = decode(b.payload)
            b.payload = encode(k, v, codec, self.cfg.v_group)
        b.codec = codec

    def drop(self, b: KVBlock) -> None:
        b.dropped, b.payload = True, None
```

uamm2/simulator.py

```
"""End-to-end UAMM-2 reference simulator.

Real: KV tensors are quantized/packed/entropy-coded, attention runs on the decoded tensors, and
quality is measured against a dense FP32 oracle. Simulated (assumed constants): tier latencies,
bandwidths and decode costs. No LLM is run; the q/k/v stream is synthetic (see workload.py).
"""
from collections import Counter
from typing import Dict

import numpy as np

from .attention import dense_oracle, select_blocks, sparse_attention
from .config import Config
from .h2o import H2OImportance
from .paged_cache import PagedKVCache
from .planner import Plan, make_planner
from .tiers import DROP, move_cost_ms, read_cost_ms, usage, decode_cost_ms
from .workload import SyntheticWorkload


class Simulator:
    def __init__(self, cfg: Config, seed: int = 7):
        self.cfg = cfg
        self.cache = PagedKVCache(cfg)
        self.h2o = H2OImportance(cfg)
        self.planner = make_planner(cfg)
        self.work = SyntheticWorkload(cfg.head_dim, seed)
```

```
    self.stats = Counter()
    self.migration_ms = 0.0
    self.overflow = [0] * len(cfg.tiers)
    self.plan_log = []
    self.trace = []          # per-step records (filled by run)

# ------------------------------------------------------------------
def _update_protection(self, t: int) -> None:
    cfg = self.cfg
    lo = t + 1 - cfg.recent_window_tokens
    for b in self.cache.blocks:
        b.protected = (b.start < cfg.sink_tokens) or (b.end > lo) or (not b.sealed)

def _apply_plan(self, plan: Plan, step: int) -> None:
    cfg, cache = self.cfg, self.cache
    for bid, (tier, codec) in plan.items():
        b = cache.blocks[bid]
        if tier == DROP:
            cache.drop(b)
            self.stats["drops"] += 1
            self.stats["dropped_tokens"] += b.n_tokens
            continue
        moved = False
        if codec != b.codec:
            before = b.payload.nbytes
            cache.reencode(b, codec)
            self.migration_ms += (decode_cost_ms(cfg, b.codec, before) +
                                  decode_cost_ms(cfg, b.codec, b.payload.nbytes))
            self.stats["reencodes"] += 1
            moved = True
        if tier != b.tier:
            self.migration_ms += move_cost_ms(cfg, b.tier, tier, b.payload.nbytes)
            b.tier = tier
            self.stats["migrations"] += 1
            moved = True
        if moved:
            b.last_move_step = step
    used = usage(cfg, cache.blocks)
    for t, u in enumerate(used):
        self.overflow[t] = max(self.overflow[t], max(0, u - cfg.tiers[t].capacity_bytes))

# ------------------------------------------------------------------
def run(self, n_tokens: int) -> Dict:
    if n_tokens <= 0:
        raise ValueError("n_tokens must be positive")
    cfg, cache, d = self.cfg, self.cache, self.cfg.head_dim
    K_or = np.zeros((n_tokens, d), np.float32)
    V_or = np.zeros((n_tokens, d), np.float32)
    rel_l2, cosine, recall, fetch_ms, flash_reads = [], [], [], [], 0
    warmup = 2 * cfg.recent_window_tokens

    for t in range(n_tokens):
        q, k, v = self.work.step(t)
        k = k.astype(np.float16).astype(np.float32)
        v = v.astype(np.float16).astype(np.float32)
        K_or[t], V_or[t] = k, v
        cache.append_token(k, v)
        self._update_protection(t)

        prot = [b for b in cache.blocks if b.protected and not b.dropped]
        cands = [b for b in cache.blocks if b.sealed and not b.protected and not b.dropped]
        chosen = select_blocks(q, cands, cfg.attn_block_budget)
        read = sorted(prot + chosen, key=lambda b: b.block_id)
        res = sparse_attention(q, cache, read)
        ref, w_or = dense_oracle(q, K_or[:t + 1], V_or[:t + 1])

        step_rec = {"t": t,
                    "fetch_ms": sum(read_cost_ms(cfg, b.tier, b.codec, b.nbytes(d)) for b in read),
                    "recall": float(w_or[res.token_idx].sum()),
                    "rel_l2": float(np.linalg.norm(res.out - ref) / (np.linalg.norm(ref) + 1e-12)),
                    "blocks_read": len(read),
                    "flash_reads": sum(1 for b in read if b.tier == len(cfg.tiers) - 1)}
        self.trace.append(step_rec)
        if t >= warmup:
            rel_l2.append(float(np.linalg.norm(res.out - ref) / (np.linalg.norm(ref) + 1e-12)))
            cosine.append(float(res.out @ ref / (np.linalg.norm(res.out) * np.linalg.norm(ref) + 1e-12)))
```

```
                recall.append(float(w_or[res.token_idx].sum()))
                fetch_ms.append(sum(read_cost_ms(cfg, b.tier, b.codec, b.nbytes(d)) for b in read))
                flash_reads += sum(1 for b in read if b.tier == len(cfg.tiers) - 1)

            h = cfg.hit_ema
            chosen_ids = {b.block_id for b in chosen}
            for b in cands:
                if b.block_id in chosen_ids:
                    b.hit_ema = h * b.hit_ema + (1 - h)
                    b.last_access, b.access_count = t, b.access_count + 1
                else:
                    b.hit_ema *= h
            for b in prot:
                b.last_access, b.access_count = t, b.access_count + 1
            self.h2o.update(res.token_idx, res.weights, t + 1)

            if (t + 1) % cfg.plan_interval == 0 and any(b.sealed for b in cache.blocks):
                self.h2o.refresh(cache.blocks)
                plan = self.planner.plan(cache.blocks, t)
                self._apply_plan(plan, t)
                step_rec["tier_bytes"] = usage(cfg, cache.blocks)
                step_rec["codecs"] = dict(Counter(b.codec for b in cache.blocks if not b.dropped))
                step_rec["migrations"] = self.stats["migrations"]
                step_rec["reencodes"] = self.stats["reencodes"]
                step_rec["dropped_tokens"] = self.stats["dropped_tokens"]

        return self._summary(n_tokens, rel_l2, cosine, recall, fetch_ms, flash_reads)

    # ------------------------------------------------------------------
    def _summary(self, n_tokens, rel_l2, cosine, recall, fetch_ms, flash_reads) -> Dict:
        cfg, d = self.cfg, self.cfg.head_dim
        blocks = self.cache.blocks
        live = [b for b in blocks if not b.dropped]
        used = usage(cfg, blocks)
        resident = sum(used)
        fp16_total = n_tokens * d * 2 * 2
        tier_blocks = Counter(b.tier for b in live)
        codecs = Counter(b.codec for b in live)
        p95 = float(np.percentile(rel_l2, 95)) if rel_l2 else 0.0
        return {
            "policy": cfg.policy,
            "tokens": n_tokens,
            "blocks": len(blocks),
            "fp16_kv_bytes": fp16_total,
            "resident_bytes": resident,
            "resident_vs_fp16": round(resident / fp16_total, 4),
            "tier_bytes": {cfg.tiers[i].name: used[i] for i in range(len(used))},
            "tier_blocks": {cfg.tiers[i].name: tier_blocks.get(i, 0) for i in range(len(used))},
            "codec_blocks": dict(sorted(codecs.items())),
            "attn_rel_l2_mean": round(float(np.mean(rel_l2)), 5) if rel_l2 else None,
            "attn_rel_l2_p95": round(p95, 5),
            "attn_cosine_mean": round(float(np.mean(cosine)), 6) if cosine else None,
            "attn_mass_recall_mean": round(float(np.mean(recall)), 5) if recall else None,
            "sim_fetch_ms_per_token": round(float(np.mean(fetch_ms)), 5) if fetch_ms else None,
            "flash_block_reads": flash_reads,
            "migrations": self.stats["migrations"],
            "reencodes": self.stats["reencodes"],
            "migration_ms_total_sim": round(self.migration_ms, 3),
            "dropped_blocks": self.stats["drops"],
            "dropped_tokens": self.stats["dropped_tokens"],
            "capacity_overflow_bytes_max": {cfg.tiers[i].name: self.overflow[i]
                                            for i in range(len(used))},
            "note": "synthetic workload + assumed latency constants; not a hardware measurement",
        }
```

uamm2/config.py

```
"""Configuration for UAMM-2.

All latency / bandwidth / decode-cost numbers below are ASSUMED placeholders for an
edge SoC. They are not measurements. Replace them with numbers measured on your
target (Ara FPGA build, LPDDR, flash) before drawing any performance conclusion.
"""
from dataclasses import dataclass, field
from typing import Dict, Tuple
```

```
@dataclass(frozen=True)
class TierSpec:
    name: str
    capacity_bytes: int
    base_latency_ms: float      # fixed cost per block access
    bandwidth_gbps: float       # GB/s


def _default_tiers() -> Tuple[TierSpec, ...]:
    return (
        TierSpec("T0_sram_vrf", 24 * 1024, 0.0005, 100.0),   # near-compute, protected set
        TierSpec("T1_dram_hot", 48 * 1024, 0.0020, 12.8),    # DRAM, light codecs
        TierSpec("T2_dram_warm", 32 * 1024, 0.0020, 12.8),   # DRAM, 4/2-bit codecs only
        TierSpec("T3_flash", 64 * 1024, 0.1000, 1.0),        # flash, entropy-coded
    )


@dataclass
class Config:
    # ---- geometry ----------------------------------------------------
    head_dim: int = 64              # channels of the flattened per-token K (and V) vector
    block_size: int = 16            # tokens per KV block (paged)
    v_group: int = 32               # channel group size for per-token value quantization
    sink_tokens: int = 4            # attention-sink tokens (always protected)
    recent_window_tokens: int = 32  # recent window (always protected)

    # ---- sparse attention (block selection) ----------------------------
    attn_block_budget: int = 24     # max non-protected blocks fetched per decode step

    # ---- H2O-style importance ------------------------------------------
    h2o_decay: float = 0.95
    h2o_aggregation: str = "mean"   # mean | max | sum

    # ---- planner -------------------------------------------------------
    policy: str = "utility"         # "utility" (UAMM-2) | "threshold" (original UAMM-1)
    plan_interval: int = 8          # replan every N decode steps
    hysteresis_margin: float = 0.25
    min_residency_steps: int = 16
    lambda_quality: float = 2.0     # ms-equivalent penalty per unit (heat * rel_err); tuned on synthetic seeds
        7-9
    capacity_price: float = 0.05    # ms-equivalent congestion price (see planner)
    capacity_headroom: float = 0.03
    hit_ema: float = 0.9            # EMA factor for per-block selection frequency
    allow_drop: bool = False        # H2O-style permanent eviction (lossy), off by default
    drop_importance: float = 0.02

    # ---- codecs --------------------------------------------------------
    codecs: Tuple[str, ...] = ("fp16", "int8", "kivi4", "kivi2")  # enabled families
    use_entropy_coding: bool = True                               # zlib variants on flash

    # ---- original UAMM-1 threshold policy (kept for ablation) -----------
    alpha: float = 0.60
    beta: float = 0.25
    gamma: float = 0.15
    t1_threshold: float = 0.68
    t2_threshold: float = 0.35

    # ---- cost model (ASSUMED) ------------------------------------------
    tiers: Tuple[TierSpec, ...] = field(default_factory=_default_tiers)
    decode_ns_per_byte: Dict[str, float] = field(default_factory=lambda: {
        "fp16": 0.0, "int8": 0.05, "kivi4": 0.12, "kivi2": 0.20})
    zlib_ns_per_byte: float = 1.0

    def __post_init__(self):
        if self.head_dim % self.v_group != 0:
            raise ValueError("head_dim must be divisible by v_group")
        if self.h2o_aggregation not in ("mean", "max", "sum"):
            raise ValueError("h2o_aggregation must be mean|max|sum")
        if self.policy not in ("utility", "threshold"):
            raise ValueError("policy must be utility|threshold")
        if not 0.0 <= self.h2o_decay <= 1.0:
            raise ValueError("h2o_decay must be in [0,1]")
        if self.block_size <= 0:
            raise ValueError("block_size must be positive")
```

uamm2/hwmodel.py

```
"""Bit-exact Python golden models of the RTL in hw/rtl, plus test-vector generation.

Everything here mirrors the integer arithmetic in the Verilog line for line. The RTL testbenches
compare against vectors written by `write_vectors`.
"""
from pathlib import Path
from typing import List, Tuple

import numpy as np

W_IMP, W_REC, W_FRQ = 154, 64, 38          # sums to 256 (~ 60% / 25% / 15%)
TH_HOT, TH_WARM, EVICT_P = 174, 89, 243
NBLK, LANES = 64, 16
PRESSURES = (0, 242, 243, 255)


# ---------------------------------------------------------------- scorer
def pack_meta(imp: int, rec: int, frq: int, prot: int) -> int:
    return (prot << 24) | (frq << 16) | (rec << 8) | imp


def scorer_model(imp: int, rec: int, frq: int, prot: int, pressure: int) -> Tuple[int, int, int]:
    """Returns (score, tier, evict). tier: 0 hot, 1 warm, 2 cold."""
    score = (imp * W_IMP + rec * W_REC + frq * W_FRQ) >> 8
    if prot or score >= TH_HOT:
        tier = 0
    elif score >= TH_WARM:
        tier = 1
    else:
        tier = 2
    evict = int((not prot) and pressure >= EVICT_P and score < TH_WARM)
    return score, tier, evict


def _score_to_meta_for(target_score: int, rng) -> Tuple[int, int, int]:
    """Find (imp, rec, frq) whose score == target_score (for boundary vectors)."""
    lo, hi = max(0, target_score - 20), min(255, target_score + 20)
    for _ in range(200000):
        imp = int(rng.integers(lo, hi + 1))
        frq = int(rng.integers(lo, hi + 1))
        base = W_IMP * imp + W_FRQ * frq
        recs = [r for r in range(256) if ((base + W_REC * r) >> 8) == target_score]
        if recs:
            return imp, int(rng.choice(recs)), frq
    raise RuntimeError(f"no vector found for score {target_score}")


def scorer_datasets(seed: int = 3) -> List[List[Tuple[int, int, int, int]]]:
    rng = np.random.default_rng(seed)
    ds = []
    # dataset 0: directed boundaries around every threshold, with and without `protected`
    d0 = []
    for tgt in (0, 1, 88, 89, 90, 173, 174, 175, 254, 255):
        imp, rec, frq = _score_to_meta_for(tgt, rng)
        d0.append((imp, rec, frq, 0))
        d0.append((imp, rec, frq, 1))
    corners = [(0, 0, 0), (255, 255, 255), (255, 0, 0), (0, 255, 0), (0, 0, 255), (1, 1, 1)]
    for c in corners:
        d0.append((*c, 0))
    while len(d0) < NBLK:
        d0.append((*(int(x) for x in rng.integers(0, 256, 3)), 0))
    ds.append(d0[:NBLK])
    # datasets 1..3: random
    for _ in range(3):
        ds.append([(*(int(x) for x in rng.integers(0, 256, 3)), int(rng.random() < 0.2))
                   for _ in range(NBLK)])
    return ds


# ---------------------------------------------------------------- codec
def codec_params(mn: int, mx: int) -> Tuple[int, int, int]:
    rng = mx - mn
```

```
    if rng == 0:
        return mn, 0, 0
    inv = min(65535, int(round(15 * 4096 / rng)))
    step = min(65535, int(round(rng * 256 / 15)))
    return mn, inv, step


def quant4_model(x: List[int], zp: int, inv: int) -> List[int]:
    out = []
    for xi in x:
        dx = xi - zp if xi >= zp else 0
        r = (dx * inv + 2048) >> 12
        out.append(min(r, 15))
    return out


def dequant4_model(q: List[int], zp: int, step: int) -> List[int]:
    return [min(255, zp + ((qi * step + 128) >> 8)) for qi in q]


def _pack_nibbles(q: List[int]) -> int:
    return sum(v << (4 * i) for i, v in enumerate(q))


def _pack_bytes(x: List[int]) -> int:
    return sum(v << (8 * i) for i, v in enumerate(x))


def codec_vectors(n: int = 400, seed: int = 5):
    rng = np.random.default_rng(seed)
    rows = []
    for i in range(n):
        mode = i % 4
        if mode == 0:       # full range
            x = [int(v) for v in rng.integers(0, 256, LANES)]
        elif mode == 1:     # narrow range around a random center
            c = int(rng.integers(0, 230))
            x = [int(c + v) for v in rng.integers(0, 16, LANES)]
        elif mode == 2:     # constant group (range == 0)
            x = [int(rng.integers(0, 256))] * LANES
        else:               # extremes
            x = [int(v) for v in rng.choice([0, 255, 1, 254], LANES)]
        mn, mx = min(x), max(x)
        zp, inv, step = codec_params(mn, mx)
        # also exercise zp larger than some x (clamp path) on a subset
        if i % 7 == 0 and mx > mn:
            zp = min(255, mn + int(rng.integers(0, max(1, (mx - mn) // 2) + 1)))
        # zp BELOW the true min (stale / conservative statistics): exercises the q>15 clamp
        if i % 5 == 0 and mn > 0:
            zp = max(0, mn - int(rng.integers(1, min(mn, 40) + 1)))
        q = quant4_model(x, zp, inv)
        deq = dequant4_model(q, zp, step)
        rows.append((zp, inv, step, x, q, deq))
    return rows


def write_vectors(out_dir) -> None:
    out = Path(out_dir)
    out.mkdir(parents=True, exist_ok=True)
    ds = scorer_datasets()
    with open(out / "scorer_meta.hex", "w") as f:
        for d in ds:
            for imp, rec, frq, prot in d:
                f.write(f"{pack_meta(imp, rec, frq, prot):07x}\n")
    with open(out / "scorer_exp.hex", "w") as f:
        for p in PRESSURES:
            for d in ds:
                for imp, rec, frq, prot in d:
                    s, t, e = scorer_model(imp, rec, frq, prot, p)
                    f.write(f"{(e << 10) | (t << 8) | s:03x}\n")
    rows = codec_vectors()
    with open(out / "codec_in.hex", "w") as fi, open(out / "codec_exp.hex", "w") as fe:
        for zp, inv, step, x, q, deq in rows:
            fi.write(f"{(zp << 160) | (inv << 144) | (step << 128) | _pack_bytes(x):042x}\n")
            fe.write(f"{(_pack_bytes(deq) << 64) | _pack_nibbles(q):048x}\n")
```

```
    with open(out / "meta.txt", "w") as f:
        f.write(f"NBLK={NBLK}\nDS={len(ds)}\nNPRESS={len(PRESSURES)}\nNCODEC={len(rows)}\n")
```

hw/tb/tb_sweep_scorer.v

```
`timescale 1ns/1ps
// Bit-exact check of uamm_sweep_scorer against vectors from uamm2/hwmodel.py.
// Run from the hw/ directory (vectors are read from vectors/).
module tb_sweep_scorer;
    localparam NBLK = 64, AW = 6, DS = 4, NP = 4;
    reg clk = 0;
    always #5 clk = ~clk;
    reg rst_n = 0, we = 0, start = 0;
    reg [AW-1:0] waddr = 0;
    reg [24:0]   wdata = 0;
    reg [7:0]    pressure = 0;
    wire busy, done, out_valid, out_evict;
    wire [AW-1:0] out_idx;
    wire [7:0] out_score;
    wire [1:0] out_tier;

    uamm_sweep_scorer #(.NBLK(NBLK), .AW(AW)) dut (
        .clk(clk), .rst_n(rst_n), .we(we), .waddr(waddr), .wdata(wdata),
        .start(start), .pressure(pressure), .busy(busy), .done(done),
        .out_valid(out_valid), .out_idx(out_idx), .out_score(out_score),
        .out_tier(out_tier), .out_evict(out_evict));

    reg [24:0] meta [0:DS*NBLK-1];
    reg [10:0] expv [0:NP*DS*NBLK-1];
    reg [7:0]  plist [0:NP-1];
    integer errors = 0, got = 0, base = 0, checks = 0;
    reg checking = 0;

    always @(posedge clk) begin
        if (checking && out_valid) begin
            checks = checks + 1;
            got = got + 1;
            if ({out_evict, out_tier, out_score} !== expv[base + out_idx]) begin
                errors = errors + 1;
                if (errors <= 10)
                    $display("MISMATCH idx=%0d got evict=%b tier=%b score=%0d  exp=%03h",
                             out_idx, out_evict, out_tier, out_score, expv[base + out_idx]);
            end
        end
    end

    task load_dataset(input integer ds);
        integer i;
        begin
            for (i = 0; i < NBLK; i = i + 1) begin
                @(negedge clk); we = 1; waddr = i; wdata = meta[ds*NBLK + i];
            end
            @(negedge clk); we = 0;
        end
    endtask

    task sweep(input integer ds, input integer pi, input integer poke);
        begin
            base = pi*DS*NBLK + ds*NBLK; got = 0; checking = 1;
            @(negedge clk); pressure = plist[pi]; start = 1;
            @(negedge clk); start = 0;
            if (poke) begin
                repeat (10) @(negedge clk);
                pressure = 8'd255; start = 1;       // must be ignored while busy
                @(negedge clk); start = 0;
            end
            wait (done === 1'b1);
            repeat (3) @(posedge clk);
            checking = 0;
            if (got != NBLK) begin
                errors = errors + 1;
                $display("COUNT ERROR ds=%0d pi=%0d got=%0d", ds, pi, got);
            end
            if (busy !== 1'b0) begin
                errors = errors + 1;
```

```
                $display("busy still high after sweep");
            end
        end
    endtask

    integer ds, pi;
    initial begin
        $readmemh("vectors/scorer_meta.hex", meta);
        $readmemh("vectors/scorer_exp.hex", expv);
        plist[0] = 0; plist[1] = 242; plist[2] = 243; plist[3] = 255;
        repeat (3) @(posedge clk);
        rst_n = 1;
        for (ds = 0; ds < DS; ds = ds + 1) begin
            load_dataset(ds);
            for (pi = 0; pi < NP; pi = pi + 1)
                sweep(ds, pi, (ds == 0 && pi == 0));
        end
        // reset in the middle of a sweep, then verify clean recovery
        @(negedge clk); pressure = 8'd255; start = 1;
        @(negedge clk); start = 0;
        repeat (10) @(negedge clk);
        rst_n = 0;
        #2;
        if (busy !== 1'b0 || out_valid !== 1'b0 || done !== 1'b0) begin
            errors = errors + 1; $display("reset did not clear control state");
        end
        repeat (2) @(negedge clk);
        rst_n = 1;
        sweep(3, 3, 0);
        if (errors == 0) $display("TB_SWEEP_SCORER PASS checks=%0d", checks);
        else             $display("TB_SWEEP_SCORER FAIL errors=%0d", errors);
        $finish;
    end

    initial begin
        #5000000;
        $display("TB_SWEEP_SCORER FAIL timeout");
        $finish;
    end
    initial if ($test$plusargs("trace")) begin
        $dumpfile("trace_sweep_scorer.vcd");
        $dumpvars(0, tb_sweep_scorer);
    end
endmodule
```

hw/tb/tb_kv_codec.v

```
`timescale 1ns/1ps
// Bit-exact check of uamm_kv_quant4 -> uamm_kv_dequant4 against uamm2/hwmodel.py vectors.
module tb_kv_codec;
    localparam LANES = 16, N = 400;
    reg clk = 0;
    always #5 clk = ~clk;
    reg rst_n = 0, in_valid = 0;
    reg [LANES*8-1:0] x_in = 0;
    reg [7:0]  zp = 0;
    reg [15:0] inv = 0, step = 0;
    wire q_valid, x_valid;
    wire [LANES*4-1:0] q_flat;
    wire [LANES*8-1:0] x_out;

    uamm_kv_quant4 #(.LANES(LANES)) uq (.clk(clk), .rst_n(rst_n), .in_valid(in_valid),
        .x_flat(x_in), .zp(zp), .inv_q12(inv), .out_valid(q_valid), .q_flat(q_flat));
    uamm_kv_dequant4 #(.LANES(LANES)) ud (.clk(clk), .rst_n(rst_n), .in_valid(q_valid),
        .q_flat(q_flat), .zp(zp), .step_q8(step), .out_valid(x_valid), .x_flat(x_out));

    reg [167:0] vin  [0:N-1];
    reg [191:0] vexp [0:N-1];
    integer i, errors = 0, sawq, sawx;

    initial begin
        $readmemh("vectors/codec_in.hex", vin);
        $readmemh("vectors/codec_exp.hex", vexp);
        repeat (3) @(posedge clk);
        rst_n = 1;
```

```
        for (i = 0; i < N; i = i + 1) begin
            @(negedge clk);
            {zp, inv, step, x_in} = vin[i];
            in_valid = 1;
            @(negedge clk);
            in_valid = 0;
            @(negedge clk);
            if ({x_out, q_flat} !== vexp[i]) begin
                errors = errors + 1;
                if (errors <= 10) $display("MISMATCH vec %0d got=%048h exp=%048h", i, {x_out, q_flat}, vexp[i]);
            end
        end
        rst_n = 0;
        #2;
        if (q_valid !== 1'b0 || x_valid !== 1'b0 || q_flat !== 0 || x_out !== 0) begin
            errors = errors + 1; $display("reset did not clear codec outputs");
        end
        if (errors == 0) $display("TB_KV_CODEC PASS vectors=%0d", N);
        else             $display("TB_KV_CODEC FAIL errors=%0d", errors);
        $finish;
    end
    initial if ($test$plusargs("trace")) begin
        $dumpfile("trace_kv_codec.vcd");
        $dumpvars(0, tb_kv_codec);
    end
endmodule
```

hw/tb/tb_block_table.v

```
`timescale 1ns/1ps
// Randomized check of uamm_block_table against a behavioural reference model.
module tb_block_table;
    localparam AW = 8, PW = 12, NOPS = 6000;
    reg clk = 0;
    always #5 clk = ~clk;
    reg rst_n = 0, we = 0, w_valid = 0, rd_en = 0;
    reg [AW-1:0] waddr = 0, raddr = 0;
    reg [1:0] w_tier = 0, w_codec = 0;
    reg [PW-1:0] w_phys = 0;
    wire rd_valid, hit;
    wire [1:0] tier, codec;
    wire [PW-1:0] phys;

    uamm_block_table #(.AW(AW), .PW(PW)) dut (.clk(clk), .rst_n(rst_n), .we(we), .waddr(waddr),
        .w_valid(w_valid), .w_tier(w_tier), .w_codec(w_codec), .w_phys(w_phys), .rd_en(rd_en),
        .raddr(raddr), .rd_valid(rd_valid), .hit(hit), .tier(tier), .codec(codec), .phys(phys));

    reg [PW+3:0] rmem [0:(1<<AW)-1];
    reg          rv   [0:(1<<AW)-1];
    integer n, i, a, errors = 0, reads = 0, writes = 0, hits = 0;
    reg [1:0] t, c;
    reg [PW-1:0] ph;
    reg vv;

    task do_read(input integer addr);
        begin
            @(negedge clk); rd_en = 1; raddr = addr; we = 0;
            @(posedge clk); #1;
            reads = reads + 1;
            if (rd_valid !== 1'b1) begin errors = errors + 1; $display("rd_valid low"); end
            if (hit !== rv[addr]) begin
                errors = errors + 1; $display("HIT MISMATCH addr=%0d got=%b exp=%b", addr, hit, rv[addr]);
            end else if (rv[addr]) begin
                hits = hits + 1;
                if ({tier, codec, phys} !== rmem[addr]) begin
                    errors = errors + 1;
                    $display("DATA MISMATCH addr=%0d got=%0h exp=%0h", addr, {tier, codec, phys}, rmem[addr]);
                end
            end
            @(negedge clk); rd_en = 0;
            @(posedge clk); #1;
            if (rd_valid !== 1'b0) begin errors = errors + 1; $display("rd_valid stuck high"); end
        end
    endtask
```

```
    initial begin
        for (i = 0; i < (1<<AW); i = i + 1) begin rv[i] = 0; rmem[i] = 0; end
        repeat (3) @(posedge clk);
        rst_n = 1;
        for (i = 0; i < 20; i = i + 1) do_read($urandom % (1<<AW));    // all must miss after reset
        for (n = 0; n < NOPS; n = n + 1) begin
            if (($urandom % 4) < 2) begin
                a = $urandom % (1<<AW);
                t = $urandom; c = $urandom; ph = $urandom; vv = (($urandom % 8) != 0);
                @(negedge clk);
                we = 1; waddr = a; w_valid = vv; w_tier = t; w_codec = c; w_phys = ph; rd_en = 0;
                rmem[a] = {t, c, ph}; rv[a] = vv; writes = writes + 1;
                @(negedge clk); we = 0;
            end else begin
                do_read($urandom % (1<<AW));
            end
        end
        if (errors == 0) $display("TB_BLOCK_TABLE PASS writes=%0d reads=%0d hits=%0d", writes, reads, hits);
        else             $display("TB_BLOCK_TABLE FAIL errors=%0d", errors);
        $finish;
    end
    initial if ($test$plusargs("trace")) begin
        $dumpfile("trace_block_table.vcd");
        $dumpvars(0, tb_block_table);
    end
endmodule
```

uamm2/tiers.py

```
"""Tier accounting and (assumed) cost model. No real memory movement happens here."""
from typing import List

from .codecs import parse_codec
from .config import Config
from .paged_cache import KVBlock

NUM_TIERS = 4
FLASH = 3
DROP = -1


def usage(cfg: Config, blocks: List[KVBlock]) -> List[int]:
    used = [0] * len(cfg.tiers)
    for b in blocks:
        if not b.dropped:
            used[b.tier] += b.nbytes(cfg.head_dim)
    return used


def decode_cost_ms(cfg: Config, codec: str, nbytes: int) -> float:
    family, entropy = parse_codec(codec)
    ns = cfg.decode_ns_per_byte[family] + (cfg.zlib_ns_per_byte if entropy else 0.0)
    return nbytes * ns * 1e-6


def read_cost_ms(cfg: Config, tier: int, codec: str, nbytes: int) -> float:
    spec = cfg.tiers[tier]
    xfer_ms = nbytes / (spec.bandwidth_gbps * 1e9) * 1e3
    return spec.base_latency_ms + xfer_ms + decode_cost_ms(cfg, codec, nbytes)


def move_cost_ms(cfg: Config, src: int, dst: int, nbytes: int) -> float:
    a, b = cfg.tiers[src], cfg.tiers[dst]
    return (a.base_latency_ms + b.base_latency_ms +
            nbytes / (a.bandwidth_gbps * 1e9) * 1e3 + nbytes / (b.bandwidth_gbps * 1e9) * 1e3)
```

uamm2/workload.py

```
"""Synthetic attention workload (NOT a real LLM).

Produces per-step (q, k, v) with structure that exercises the system:
  * a few attention-sink tokens and sporadic heavy-hitter tokens,
  * outlier key channels (the per-channel-key-quantization motivation),
  * topic phases whose blocks become hot again when the query returns to an old topic
```

```
    (so cold blocks must be promoted back),
  * local recency in the query.
Replace with real model activations for any real conclusion.
"""
import numpy as np


class SyntheticWorkload:
    def __init__(self, d: int, seed: int = 7, n_topics: int = 4, phase_len: int = 96,
                 sharpness: float = 4.0):
        self.d = d
        self.rng = np.random.default_rng(seed)
        r = self.rng
        self.channel_scale = np.ones(d)
        self.channel_scale[r.choice(d, size=max(1, d // 16), replace=False)] = 6.0   # key outliers
        self.topics = r.normal(size=(n_topics, d))
        self.topics /= np.linalg.norm(self.topics, axis=1, keepdims=True)
        self.heavy_dir = r.normal(size=d)
        self.heavy_dir /= np.linalg.norm(self.heavy_dir)
        self.phase_len = phase_len
        self.sharpness = sharpness   # scales query norm -> softmax peakedness
        self.n_topics = n_topics
        self._keys = []

    def topic_of(self, t: int) -> int:
        return (t // self.phase_len) % self.n_topics

    def query_topic(self, t: int) -> int:
        phase = t // self.phase_len
        # revisit older topics periodically so old blocks turn hot again
        return (phase if phase % 3 else max(0, phase - 2)) % self.n_topics

    def step(self, t: int):
        r, d = self.rng, self.d
        heavy = t < 4 or r.random() < 0.03
        k = r.normal(size=d) * 0.5 * self.channel_scale
        k = k + 3.0 * np.sqrt(d) / 8 * self.topics[self.topic_of(t)]
        if heavy:
            k = k + 4.0 * np.sqrt(d) / 8 * self.heavy_dir
        v = r.normal(size=d)
        q = 0.3 * r.normal(size=d) + 2.0 * np.sqrt(d) / 8 * self.topics[self.query_topic(t)]
        q = q + 1.0 * np.sqrt(d) / 8 * self.heavy_dir
        if self._keys:
            j = int(r.integers(1, min(len(self._keys), 12) + 1))
            q = q + 0.5 * self._keys[-j] / np.linalg.norm(self._keys[-j]) * np.sqrt(d) / 8
        self._keys.append(k)
        q = q * self.sharpness
        return q.astype(np.float32), k.astype(np.float32), v.astype(np.float32)
```

uamm2/runtime.py

```
"""Runtime adapter layer: the seam between UAMM-2 and a real inference engine.

ReferenceAdapter is fully functional against the NumPy reference engine in this package.
CompilerRuntimeAdapter is a documented SKELETON: it is NOT connected to a compiler-based runtime, and nothing in
     it has been
run against a real runtime build. What I verified about a compiler-based runtime (from its docs): it compiles
    models via
Apache TVM Unity, targets CUDA/Vulkan/Metal GPUs plus ARM/x86 CPUs through LLVM, supports dynamic
shapes and low-bit weight quantization, and exposes an OpenAI-style API. I did NOT verify how its
paged KV cache is exposed or whether a RISC-V/RVV (Ara) target works end to end; both need
validation against a pinned TVM version before this adapter can be filled in.
"""
from abc import ABC, abstractmethod
from typing import Dict, Tuple

import numpy as np

from .config import Config
from .paged_cache import PagedKVCache


class RuntimeAdapter(ABC):
    @abstractmethod
    def write_token(self, k: np.ndarray, v: np.ndarray) -> int:
```

```
        """Append one token's K/V; return its block id."""

    @abstractmethod
    def read_block(self, block_id: int) -> Tuple[np.ndarray, np.ndarray]:
        """Return (K, V) float32 for a block, decoding whatever codec it is stored in."""

    @abstractmethod
    def migrate_block(self, block_id: int, tier: int, codec: str) -> None:
        """Move/re-encode a block. Real runtimes must synchronize with in-flight attention."""

    @abstractmethod
    def attention_importance(self) -> Dict[int, float]:
        """Return {token_id: attention_weight} for the last step, if the backend can expose it."""


class ReferenceAdapter(RuntimeAdapter):
    def __init__(self, cfg: Config):
        self.cfg = cfg
        self.cache = PagedKVCache(cfg)
        self._last: Dict[int, float] = {}

    def write_token(self, k, v) -> int:
        b, _ = self.cache.append_token(k, v)
        return b.block_id

    def read_block(self, block_id: int):
        return self.cache.get_kv(self.cache.blocks[block_id])

    def migrate_block(self, block_id: int, tier: int, codec: str) -> None:
        b = self.cache.blocks[block_id]
        if not b.sealed:
            raise ValueError("cannot migrate the open block")
        if not 0 <= tier < len(self.cfg.tiers):
            raise ValueError("bad tier")
        if codec != b.codec:
            self.cache.reencode(b, codec)
        b.tier = tier

    def attention_importance(self) -> Dict[int, float]:
        return dict(self._last)

    def record_attention(self, token_idx, weights) -> None:
        self._last = {int(t): float(w) for t, w in zip(token_idx, weights)}


class CompilerRuntimeAdapter(RuntimeAdapter):
    """SKELETON - see module docstring. Integration checklist (all unverified):

    1. write_token / read_block: map to the engine's paged KV cache pages. Requires reading the
       pinned TVM version's KV-cache API; storage there is engine-owned tensors, not NumPy.
    2. attention_importance: fused attention kernels normally do not expose softmax weights.
       Either add a row-sum output to the attention kernel or accumulate scores in a side kernel
       (on Ara: a vfredusum loop over the already-computed logits).
    3. migrate_block: needs real, synchronized tensor movement and a codec kernel (see ara/).
    4. Target: confirm TVM's LLVM path can emit RVV 1.0 code for the model's kernels, and what
       runtime (bare-metal / Linux on CVA6) hosts the TVM runtime.
    """

    def _todo(self):
        raise NotImplementedError("CompilerRuntimeAdapter is a skeleton; see its docstring for the checklist")

    def write_token(self, k, v):
        self._todo()

    def read_block(self, block_id):
        self._todo()

    def migrate_block(self, block_id, tier, codec):
        self._todo()

    def attention_importance(self):
        self._todo()
```

tests/test_system.py

```
from dataclasses import replace

import numpy as np
import pytest

from uamm2 import Config, Simulator
from uamm2.runtime import CompilerRuntimeAdapter, ReferenceAdapter
from uamm2.tiers import usage

N = 1024     # fp16 KV = 262 KB > total tier budget (168 KB): compression or dropping is mandatory


@pytest.fixture(scope="module")
def results():
    base = Config()
    v1 = replace(base, policy="threshold", codecs=("fp16",), use_entropy_coding=False)
    v2 = replace(base)
    out = {}
    for name, cfg in (("v1", v1), ("v2", v2)):
        sim = Simulator(cfg, seed=7)
        out[name] = (sim, sim.run(N))
    return out


def test_deterministic():
    a = Simulator(Config(), seed=3).run(256)
    b = Simulator(Config(), seed=3).run(256)
    assert a == b


def test_v2_fits_budget_without_losing_data(results):
    _, r = results["v2"]
    assert r["dropped_tokens"] == 0
    assert sum(r["capacity_overflow_bytes_max"].values()) == 0
    assert r["resident_vs_fp16"] < 0.5


def test_v1_must_drop_data_when_fp16_does_not_fit(results):
    _, r = results["v1"]
    assert r["dropped_tokens"] > 0


def test_v2_beats_v1_on_quality_in_this_synthetic_setup(results):
    _, r1 = results["v1"]
    _, r2 = results["v2"]
    assert r2["attn_rel_l2_mean"] < r1["attn_rel_l2_mean"]
    assert r2["attn_mass_recall_mean"] > r1["attn_mass_recall_mean"]


def test_tier_capacities_respected_on_exact_size_tiers(results):
    sim, _ = results["v2"]
    used = usage(sim.cfg, sim.cache.blocks)
    for t in (0, 1, 2):
        assert used[t] <= sim.cfg.tiers[t].capacity_bytes
    assert used[3] <= sim.cfg.tiers[3].capacity_bytes * 1.05       # entropy sizes are estimated


def test_protected_blocks_stay_hot_and_lossless(results):
    sim, _ = results["v2"]
    for b in sim.cache.blocks:
        if b.protected and not b.dropped:
            assert b.tier == 0 and b.codec == "fp16"


def test_precision_never_increases_across_plans():
    """One-way ratchet: instrument every applied plan and assert no block regains precision."""
    class Checked(Simulator):
        violations = 0
        changed = 0

        def _apply_plan(self, plan, step):
            before = {b.block_id: b.rank for b in self.cache.blocks}
            super()._apply_plan(plan, step)
            for b in self.cache.blocks:
```

```
                if b.block_id in before and not b.dropped:
                    if b.rank < before[b.block_id]:
                        Checked.violations += 1
                    if b.rank > before[b.block_id]:
                        Checked.changed += 1

    Checked.violations = Checked.changed = 0
    Checked(Config(), seed=7).run(768)
    assert Checked.violations == 0
    assert Checked.changed > 0          # the test really saw precision being lowered


def test_hysteresis_reduces_churn():
    base = Config()
    low = Simulator(replace(base, hysteresis_margin=0.0, min_residency_steps=0), 7).run(768)
    high = Simulator(base, 7).run(768)
    churn_hi = high["migrations"] + high["reencodes"]
    churn_lo = low["migrations"] + low["reencodes"]
    assert churn_hi < 0.6 * churn_lo          # observed ~0.3x on this workload


def test_drop_only_when_enabled():
    cfg = replace(Config(), codecs=("fp16",), use_entropy_coding=False)     # cannot fit -> overflow
    off = Simulator(cfg, 7).run(1024)
    on = Simulator(replace(cfg, allow_drop=True), 7).run(1024)
    assert off["dropped_tokens"] == 0 and sum(off["capacity_overflow_bytes_max"].values()) > 0
    assert on["dropped_tokens"] > 0
    assert sum(on["capacity_overflow_bytes_max"].values()) < sum(off["capacity_overflow_bytes_max"].values())


def test_reference_adapter_roundtrip_and_migrate():
    cfg = Config()
    ad = ReferenceAdapter(cfg)
    r = np.random.default_rng(0)
    for _ in range(16):
        ad.write_token(r.normal(size=64).astype(np.float32), r.normal(size=64).astype(np.float32))
    k0, _ = ad.read_block(0)
    ad.migrate_block(0, 2, "kivi4")
    k1, _ = ad.read_block(0)
    assert ad.cache.blocks[0].tier == 2 and np.abs(k1 - k0).max() < 1.0
    with pytest.raises(ValueError):
        ad.migrate_block(0, 9, "kivi4")


def test_compiler_adapter_is_an_honest_skeleton():
    with pytest.raises(NotImplementedError):
        CompilerRuntimeAdapter().write_token(None, None)


def test_invalid_run_args():
    with pytest.raises(ValueError):
        Simulator(Config()).run(0)
```

tests/test_codecs.py

```
import numpy as np
import pytest

from uamm2.codecs import (CODEC_RANK, calibrate, codec_rank, decode, encode, pack_bits,
                          parse_codec, perchannel_key_roundtrip, pertoken_key_roundtrip,
                          rel_error, unpack_bits)


def _kv(t=16, d=64, seed=0, outliers=True):
    r = np.random.default_rng(seed)
    scale = np.ones(d)
    if outliers:
        scale[r.choice(d, 4, replace=False)] = 6.0
    return (r.normal(size=(t, d)) * scale).astype(np.float32), r.normal(size=(t, d)).astype(np.float32)


@pytest.mark.parametrize("bits", [2, 4, 8])
@pytest.mark.parametrize("count", [1, 7, 64, 1000])
def test_pack_unpack_roundtrip(bits, count):
    r = np.random.default_rng(bits * 1000 + count)
```

```
    q = r.integers(0, 1 << bits, count).astype(np.uint8)
    assert np.array_equal(unpack_bits(pack_bits(q, bits), bits, count), q)


def test_pack_nibble_order_low_first():
    assert pack_bits(np.array([0x3, 0xA], np.uint8), 4) == bytes([0xA3])   # element 0 -> low nibble


def test_fp16_roundtrip_is_fp16_exact():
    k, v = _kv()
    k2, v2 = decode(encode(k, v, "fp16", 32))
    assert np.array_equal(k2, k.astype(np.float16).astype(np.float32))
    assert np.array_equal(v2, v.astype(np.float16).astype(np.float32))


def test_error_and_size_ordering():
    k, v = _kv()
    errs, sizes = calibrate(k, v, ["fp16", "int8", "kivi4", "kivi2"], 32)
    assert errs["fp16"] < errs["int8"] < errs["kivi4"] < errs["kivi2"]
    assert sizes["fp16"] > sizes["int8"] > sizes["kivi4"] > sizes["kivi2"]
    assert sizes["kivi4"] / sizes["fp16"] < 0.36
    assert sizes["kivi2"] / sizes["fp16"] < 0.25
    assert errs["int8"] < 1e-3 and errs["kivi4"] < 0.05


@pytest.mark.parametrize("fam", ["int8", "kivi4", "kivi2"])
def test_entropy_variant_is_lossless_wrt_base(fam):
    k, v = _kv(seed=3)
    a = decode(encode(k, v, fam, 32))
    b = decode(encode(k, v, fam + "z", 32))
    assert np.array_equal(a[0], b[0]) and np.array_equal(a[1], b[1])


def test_per_channel_keys_beat_per_token_on_outlier_channels():
    k, _ = _kv(seed=4, outliers=True)
    e_ch = float(((perchannel_key_roundtrip(k, 4) - k) ** 2).sum() / (k ** 2).sum())
    e_tok = float(((pertoken_key_roundtrip(k, 4) - k) ** 2).sum() / (k ** 2).sum())
    assert e_ch < e_tok


def test_constant_input_does_not_blow_up():
    k = np.full((16, 64), 3.0, np.float32)
    v = np.full((16, 64), -2.0, np.float32)
    for name in ("int8", "kivi4", "kivi2"):
        k2, v2 = decode(encode(k, v, name, 32))
        assert np.isfinite(k2).all() and np.abs(k2 - k).max() < 0.01


def test_partial_block_shapes():
    k, v = _kv(t=5)
    k2, v2 = decode(encode(k, v, "kivi4z", 32))
    assert k2.shape == (5, 64) and v2.shape == (5, 64)


def test_bad_inputs():
    with pytest.raises(ValueError):
        parse_codec("fp16z")
    with pytest.raises(ValueError):
        parse_codec("bogus")
    k, v = _kv()
    with pytest.raises(ValueError):
        encode(k, v[:, :32], "int8", 32)
    with pytest.raises(ValueError):
        encode(k, v, "int8", 48)       # d not divisible by v_group


def test_rank_ordering():
    assert codec_rank("fp16") < codec_rank("int8") < codec_rank("kivi4z") <= CODEC_RANK["kivi2"]
```

Appendix D. UAMM-1 Source Code (Original Prototype)

This appendix reproduces, unchanged, the prototype that Chapter 3 analyses: the Python modules (configuration, paged cache, controller, simulator) followed by the Verilog controller and its testbench. The concatenated listings are also stored in the archive under `uamm1_original/`. They are the authors' own earlier code and are included so that the comparison in Chapters 4 and 8 can be checked against the actual implementation.

D.1 UAMM-1 Python modules

```
from dataclasses import dataclass

@dataclass
class Config:
    block_size: int = 16
    gpu_capacity: int = 8
    cpu_capacity: int = 16
    nvme_capacity: int = 64
    recent_window_tokens: int = 32
    alpha: float = 0.60
    beta: float = 0.25
    gamma: float = 0.15
    gpu_threshold: float = 0.68
    cpu_threshold: float = 0.35
    simulated_gpu_read_ms: float = 0.02
    simulated_cpu_read_ms: float = 0.20
    simulated_nvme_read_ms: float = 4.0

from dataclasses import dataclass
from .config import Config
from .paged_cache import KVBlock

@dataclass
class Decision:
    score: float
    target_tier: str
    evict: bool = False

class UnifiedMemoryController:
    def __init__(self, config: Config):
        self.cfg = config

    @staticmethod
    def _norm(value: float, maximum: float) -> float:
        return max(0.0, min(1.0, value / maximum if maximum > 0 else 0.0))

    def score(self, block: KVBlock, now: int, max_age: int, max_frequency: int) -> float:
        importance = self._norm(block.importance, 1.0)
        age = max(0, now - block.last_access)
        recency = 1.0 - self._norm(age, max(1, max_age))
        frequency = self._norm(block.access_count, max(1, max_frequency))
        return (self.cfg.alpha * importance +
                self.cfg.beta * recency +
                self.cfg.gamma * frequency)

    def decide(self, block: KVBlock, now: int, max_age: int, max_frequency: int,
               pressure: float) -> Decision:
        score = self.score(block, now, max_age, max_frequency)
        if score >= self.cfg.gpu_threshold:
            tier = "gpu"
        elif score >= self.cfg.cpu_threshold:
            tier = "cpu"
        else:
            tier = "nvme"
        evict = pressure >= 0.95 and score < self.cfg.cpu_threshold and not block.protected
        return Decision(score, tier, evict)
```

```
from collections import defaultdict
from typing import Dict, Iterable
from .paged_cache import PagedKVCache

class H2OImportanceTracker:
    """H2O-inspired heavy-hitter tracker using supplied attention weights."""
    def __init__(self, cache: PagedKVCache, decay: float = 0.9, aggregation: str = "mean"):
        if not 0.0 <= decay <= 1.0:
            raise ValueError("decay must be in [0,1]")
        if aggregation not in {"mean", "sum", "max"}:
            raise ValueError("aggregation must be mean, sum or max")
        self.cache, self.decay, self.aggregation = cache, decay, aggregation
        self.token_scores: Dict[int, float] = defaultdict(float)

    def update(self, attention_by_token: Dict[int, float]) -> None:
        for token_id in list(self.token_scores):
            self.token_scores[token_id] *= self.decay
        for token_id, weight in attention_by_token.items():
            if token_id in self.cache.token_to_block:
                self.token_scores[token_id] += max(0.0, float(weight))

        grouped = defaultdict(list)
        for token_id, block_id in self.cache.token_to_block.items():
            grouped[block_id].append(self.token_scores[token_id])
        for block_id, values in grouped.items():
            block = self.cache.get_block(block_id)
            if not values:
                block.importance = 0.0
            elif self.aggregation == "sum":
                block.importance = sum(values)
            elif self.aggregation == "max":
                block.importance = max(values)
            else:
                block.importance = sum(values) / len(values)

    def block_importance(self, block_id: int) -> float:
        return self.cache.get_block(block_id).importance

import argparse
import json
from .config import Config
from .simulator import UAMMSimulator

def main():
    p = argparse.ArgumentParser(description="UAMM reference simulator")
    p.add_argument("--tokens", type=int, default=512)
    p.add_argument("--block-size", type=int, default=16)
    p.add_argument("--gpu-blocks", type=int, default=8)
    p.add_argument("--cpu-blocks", type=int, default=16)
    p.add_argument("--nvme-blocks", type=int, default=64)
    args = p.parse_args()
    cfg = Config(block_size=args.block_size, gpu_capacity=args.gpu_blocks,
                 cpu_capacity=args.cpu_blocks, nvme_capacity=args.nvme_blocks)
    sim = UAMMSimulator(cfg)
    metrics = sim.run(args.tokens)
    print("UAMM simulation complete (metadata simulator; not real LLM inference)")
    print("Tier occupancy:", json.dumps(sim.tiers.snapshot(), indent=2))
    print("Metrics:", json.dumps(metrics.as_dict(), indent=2))

if __name__ == "__main__":
    main()

from collections import defaultdict

class MemoryTiers:
    """Capacity and transfer-cost simulator; no real GPU/CPU/NVMe data movement."""
    def __init__(self, gpu_capacity: int, cpu_capacity: int, nvme_capacity: int):
        self.capacity = {"gpu": gpu_capacity, "cpu": cpu_capacity, "nvme": nvme_capacity}
        self.blocks = {"gpu": set(), "cpu": set(), "nvme": set()}
        self.transfer_count = 0
        self.eviction_count = 0

    def tier_of(self, block_id: int):
        for tier, ids in self.blocks.items():
            if block_id in ids:
```

```
                return tier
        return None

    def occupancy(self, tier: str) -> int:
        return len(self.blocks[tier])

    def has_space(self, tier: str) -> bool:
        return self.occupancy(tier) < self.capacity[tier]

    def place(self, block_id: int, tier: str) -> bool:
        if tier not in self.blocks:
            raise ValueError(f"unknown tier {tier}")
        current = self.tier_of(block_id)
        if current == tier:
            return True
        if not self.has_space(tier):
            return False
        if current:
            self.blocks[current].remove(block_id)
            self.transfer_count += 1
        self.blocks[tier].add(block_id)
        return True

    def remove(self, block_id: int):
        tier = self.tier_of(block_id)
        if tier:
            self.blocks[tier].remove(block_id)
            self.eviction_count += 1

    def all_ids(self):
        return set().union(*self.blocks.values())

    def snapshot(self):
        return {tier: len(ids) for tier, ids in self.blocks.items()}

from dataclasses import dataclass, asdict

@dataclass
class Metrics:
    simulated_ttft_ms: float
    simulated_itl_ms: float
    simulated_throughput_tokens_s: float
    peak_gpu_blocks: int
    cpu_nvme_transfers: int
    evictions: int

    def as_dict(self):
        return asdict(self)

from dataclasses import dataclass, field
from typing import Dict, List

@dataclass
class KVBlock:
    block_id: int
    token_ids: List[int] = field(default_factory=list)
    tier: str = "gpu"
    importance: float = 0.0
    recency: float = 0.0
    frequency: float = 0.0
    last_access: int = 0
    access_count: int = 0
    protected: bool = False

class PagedKVCache:
    """Logical fixed-size KV block allocator; stores metadata, not actual tensors."""
    def __init__(self, block_size: int):
        if block_size <= 0:
            raise ValueError("block_size must be positive")
        self.block_size = block_size
        self.blocks: Dict[int, KVBlock] = {}
        self.token_to_block: Dict[int, int] = {}
        self._next_id = 0

    def append_token(self, token_id: int) -> KVBlock:
        if token_id in self.token_to_block:
```

```
            raise ValueError(f"duplicate token id: {token_id}")
        if not self.blocks or len(next(reversed(self.blocks.values())).token_ids) >= self.block_size:
            block = KVBlock(self._next_id)
            self.blocks[self._next_id] = block
            self._next_id += 1
        else:
            block = next(reversed(self.blocks.values()))
        block.token_ids.append(token_id)
        self.token_to_block[token_id] = block.block_id
        return block

    def get_block(self, block_id: int) -> KVBlock:
        return self.blocks[block_id]

    def remove_block(self, block_id: int) -> None:
        block = self.blocks.pop(block_id)
        for token_id in block.token_ids:
            self.token_to_block.pop(token_id, None)

    def __len__(self):
        return len(self.blocks)

"""Extension points for real inference runtimes.

The simulator intentionally does not pretend to patch vLLM internals. A production
adapter must be written against the exact pinned vLLM version and its supported
interfaces. Attention-score access and KV block migration are backend-dependent.
"""

class RuntimeAdapter:
    def load_model(self):
        raise NotImplementedError

      def allocate_kv_block(self, sequence_id, logical_block_id):
          raise NotImplementedError

      def get_attention_importance(self, sequence_id):
          """Return {token_id: attention_weight}; backend must expose this safely."""
          raise NotImplementedError

      def migrate_kv_block(self, block_id, source_tier, destination_tier):
          """Perform real, synchronized tensor movement and update ownership metadata."""
          raise NotImplementedError

class SmoothQuantAdapter:
    """Hook point for a calibrated SmoothQuant INT8 model artifact."""
    def load_quantized_model(self, model_path):
        raise NotImplementedError("Connect a tested SmoothQuant-compatible runtime here.")

import random
from .paged_cache import PagedKVCache
from .h2o import H2OImportanceTracker
from .controller import UnifiedMemoryController
from .memory_tiers import MemoryTiers
from .metrics import Metrics

class UAMMSimulator:
    def __init__(self, config):
        self.cfg = config
        self.cache = PagedKVCache(config.block_size)
        self.h2o = H2OImportanceTracker(self.cache)
        self.controller = UnifiedMemoryController(config)
        self.tiers = MemoryTiers(config.gpu_capacity, config.cpu_capacity, config.nvme_capacity)
        self.tick = 0
        self.peak_gpu = 0
        self.latency_ms = 0.0

      def _ensure_capacity(self, target: str, incoming_id: int):
          if self.tiers.has_space(target):
              return True
          # Demote the least valuable, unprotected resident block to a slower tier.
          candidates = []
          for block_id in self.tiers.blocks[target]:
              b = self.cache.get_block(block_id)
              if not b.protected and block_id != incoming_id:
                  candidates.append((b.importance + b.recency * 0.01, block_id))
```

```
            if not candidates:
                return False
            _, victim_id = min(candidates)
            fallback = "cpu" if target == "gpu" else "nvme"
            if fallback == "nvme" and not self.tiers.has_space("nvme"):
                self.tiers.remove(victim_id)
                self.cache.remove_block(victim_id)
                return self.tiers.has_space(target)
            if self.tiers.place(victim_id, fallback):
                self.cache.get_block(victim_id).tier = fallback
        return self.tiers.has_space(target)
    return False

def _place(self, block, target):
    current = self.tiers.tier_of(block.block_id)
    if current == target:
        return
    if not self._ensure_capacity(target, block.block_id):
        # Try slower tier as a safe fallback.
        for fallback in (("cpu", "nvme") if target == "gpu" else ("nvme",)):
            if self._ensure_capacity(fallback, block.block_id) and self.tiers.place(block.block_id, fallback):
                block.tier = fallback
                return
        return
    self.tiers.place(block.block_id, target)
    block.tier = target

def run(self, token_count: int):
    if token_count <= 0:
        raise ValueError("token_count must be positive")
    random.seed(7)
    for token in range(token_count):
        self.tick += 1
        block = self.cache.append_token(token)
        if self.tiers.tier_of(block.block_id) is None:
            self._place(block, "gpu")
        # Simulated attention: recent tokens plus a few persistent heavy hitters.
        attention = {}
        start = max(0, token - self.cfg.recent_window_tokens)
        for old_token in range(start, token + 1):
            attention[old_token] = random.random() * (1.0 if old_token > token - 8 else 0.15)
        if token > 0:
            attention[0] = attention.get(0, 0.0) + 0.8
        self.h2o.update(attention)

        for b in self.cache.blocks.values():
            b.protected = bool(b.token_ids and b.token_ids[-1] >= token - self.cfg.recent_window_tokens)
            b.recency = 1.0 / (1.0 + max(0, token - b.last_access))
        # Touch blocks in the current attention window.
        for tok in attention:
            bid = self.cache.token_to_block.get(tok)
            if bid is not None:
                b = self.cache.get_block(bid)
                b.access_count += 1
                b.last_access = token
                tier = self.tiers.tier_of(bid)
                if tier == "cpu":
                    self.latency_ms += self.cfg.simulated_cpu_read_ms
                elif tier == "nvme":
                    self.latency_ms += self.cfg.simulated_nvme_read_ms
                else:
                    self.latency_ms += self.cfg.simulated_gpu_read_ms

        pressure = self.tiers.occupancy("gpu") / max(1, self.cfg.gpu_capacity)
        max_freq = max((b.access_count for b in self.cache.blocks.values()), default=1)
        for b in list(self.cache.blocks.values()):
            decision = self.controller.decide(b, token, max(1, token_count), max_freq, pressure)
            if not b.protected:
                self._place(b, decision.target_tier)
    self.peak_gpu = max(self.peak_gpu, self.tiers.occupancy("gpu"))

# Simulated aggregate metrics only; no real model tokens were generated.
ttft = 5.0 + self.cfg.block_size * 0.03
itl = self.latency_ms / token_count if token_count else 0.0
elapsed = max(0.001, ttft + self.latency_ms)
return Metrics(ttft, itl, token_count / (elapsed / 1000.0),
```

```
            self.peak_gpu, self.tiers.transfer_count, self.tiers.eviction_count)
```

D.2 UAMM-1 Verilog controller and testbench

```
`timescale 1ns/1ps
module tb_uamm_controller;
    reg clk = 0;
    always #5 clk = ~clk;
    reg rst_n = 0, valid = 0;
    reg [7:0] importance, recency, frequency, memory_pressure;
    wire out_valid, evict;
    wire [7:0] score;
    wire [1:0] tier;

    uamm_controller dut(.clk(clk), .rst_n(rst_n), .valid(valid),
        .importance(importance), .recency(recency), .frequency(frequency),
        .memory_pressure(memory_pressure), .out_valid(out_valid),
        .score(score), .tier(tier), .evict(evict));

    task apply;
        input [7:0] h, r, f, p;
        input [1:0] expected_tier;
        input expected_evict;
        begin
            @(negedge clk);
            valid=1; importance=h; recency=r; frequency=f; memory_pressure=p;
            @(posedge clk); #1;
            if (!out_valid || tier !== expected_tier || evict !== expected_evict) begin
                $display("FAIL h=%0d r=%0d f=%0d p=%0d score=%0d tier=%b evict=%b",
                         h,r,f,p,score,tier,evict);
                $fatal(1);
            end
        end
    endtask

    initial begin
        importance=0; recency=0; frequency=0; memory_pressure=0;
        repeat(2) @(posedge clk);
        rst_n=1;
        apply(8'd255,8'd255,8'd255,8'd0,2'b00,1'b0);
        apply(8'd150,8'd100,8'd80,8'd0,2'b01,1'b0);
        apply(8'd10,8'd10,8'd10,8'd255,2'b10,1'b1);
        $display("PASS: controller decisions");
        $finish;
    end
endmodule

// UAMM policy decision engine - synthesizable Verilog-2001.
// Inputs are normalized unsigned 8-bit values (0..255).
module uamm_controller (
    input wire        clk,
    input wire        rst_n,
    input wire        valid,
    input wire [7:0] importance,
    input wire [7:0] recency,
    input wire [7:0] frequency,
    input wire [7:0] memory_pressure,
    output reg        out_valid,
    output reg [7:0] score,
    output reg [1:0] tier,        // 00 GPU, 01 CPU, 10 NVMe
    output reg        evict
);
    // Weighted score: 60% importance + 25% recency + 15% frequency.
    // Sum of weights is 100; divide by 100 to retain 8-bit scale.
    wire [15:0] weighted_sum =
        (importance * 8'd60) +
        (recency    * 8'd25) +
        (frequency * 8'd15);
    wire [7:0] computed_score = weighted_sum / 16'd100;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            out_valid <= 1'b0;
            score       <= 8'd0;
            tier        <= 2'b10;
```

```
            evict       <= 1'b0;
        end else begin
            out_valid <= valid;
            if (valid) begin
                 score <= computed_score;
                 if (computed_score >= 8'd174)
                      tier <= 2'b00; // GPU
                 else if (computed_score >= 8'd89)
                      tier <= 2'b01; // CPU
                 else
                      tier <= 2'b10; // NVMe

                  evict <= (memory_pressure >= 8'd243) &&
                           (computed_score < 8'd89);
              end else begin
                  evict <= 1'b0;
              end
        end
    end
endmodule
```

RTL regression log

```
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --binary --timing -Wno-fatal -Wno-lint -CFLAGS
    "-std=c++20 -fcoroutines" --top-module tb_sweep_scorer -Mdir build/sweep_scorer rtl/uamm_sweep_scorer.v
    tb/tb_sweep_scorer.v -o sim >build/sweep_scorer.log 2>&1 || (cat build/sweep_scorer.log; exit 1)
TB_SWEEP_SCORER PASS checks=1088
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --binary --timing -Wno-fatal -Wno-lint -CFLAGS
    "-std=c++20 -fcoroutines" --top-module tb_kv_codec -Mdir build/kv_codec rtl/uamm_kv_codec.v tb/tb_kv_codec.v
     -o sim >build/kv_codec.log 2>&1 || (cat build/kv_codec.log; exit 1)
TB_KV_CODEC PASS vectors=400
/usr/local/lib/python3.13/site-packages/verilator/bin/verilator --binary --timing -Wno-fatal -Wno-lint -CFLAGS
    "-std=c++20 -fcoroutines" --top-module tb_block_table -Mdir build/block_table rtl/uamm_block_table.v
    tb/tb_block_table.v -o sim >build/block_table.log 2>&1 || (cat build/block_table.log; exit 1)
TB_BLOCK_TABLE PASS writes=3049 reads=2971 hits=2313
```